

JSON Mastery: Structured Outputs & JSON Schema untuk LLM

Teknik Interaksi Database Modern dengan JSON, Python, n8n, dan Open Source LLM

Onno W. Purbo
onno@indo.net.id
onno@itts.ac.id
Twitter onnowpurbo

Institut Teknologi Tangerang Selatan (ITTS)
XecureIT
OnnoCenter
2026

Lisensi: Karya ini dilisensikan di bawah **Creative Commons Attribution (CC BY) 4.0 International**. Anda bebas menggunakan, membagikan, dan mengadaptasi karya ini untuk tujuan apa pun, termasuk komersial, dengan tetap mencantumkan atribusi yang layak kepada penulis.

Disclaimer: Buku ini disusun untuk tujuan edukasi, dokumentasi, dan refleksi publik berdasarkan sumber-sumber yang dianggap kredibel pada saat penulisan. Penulis telah berupaya menjaga akurasi, namun sangat mungkin masih terdapat kekurangan atau kekeliruan; karena itu, buku ini tidak dimaksudkan sebagai satu-satunya rujukan yang final, dan penulis terbuka terhadap koreksi serta masukan yang membangun

LLM tidak boleh dipercaya sebelum output-nya divalidasi, distrukturkan, disimpan, dan dapat diaudit.

Kata Pengantar

Buku **Structured Outputs & JSON Schema untuk LLM** disusun untuk membantu mahasiswa memahami bahwa AI modern tidak cukup hanya mampu menjawab dalam bentuk teks. Dalam sistem nyata, output AI harus dapat **distrukturkan, divalidasi, disimpan, dicari, dan diaudit**. Karena itu, buku ini mengajak pembaca bergerak dari konsep dasar JSON, JSON Schema, Python, Pydantic, open source LLM, n8n, hingga database JSON secara bertahap dan praktis.

Pendekatan buku ini sederhana: **LLM tidak boleh langsung dipercaya sebelum output-nya lolos pemeriksaan struktur**. Mahasiswa akan belajar membuat schema, membangun validator Python, menjalankan LLM lokal, menyimpan output ke database, membuat workflow otomatis, menangani error, dan menyusun proyek akhir yang layak menjadi portofolio.

Buku ini juga disusun dengan semangat *open source*, agar mahasiswa dapat belajar secara mandiri, membangun sistem sendiri, dan memahami fondasi *AI engineering* secara lebih kritis. Sebagian proses penyusunan dibantu oleh AI, namun tanggung jawab isi, arah, dan penyuntingan tetap berada pada penulis.

Semoga buku ini menjadi bekal praktis bagi mahasiswa untuk membangun aplikasi AI yang bukan hanya terlihat pintar, tetapi juga **rapi, terukur, aman, dan dapat dipertanggungjawabkan**.

Jakarta, Mei 2026

Onno W. Purbo

Executive Summary

Buku **Structured Outputs & JSON Schema untuk LLM** diposisikan sebagai buku praktis *AI engineering* untuk mahasiswa yang ingin bergerak dari sekadar memakai LLM sebagai mesin penjawab menuju kemampuan membangun **pipeline data AI yang valid, terstruktur, tersimpan, dan dapat diaudit**. Revisi buku ini menajamkan arah lama dengan menjadikan **database sebagai pusat sistem**, bukan hanya tempat akhir penyimpanan. Karena fokusnya adalah teknik interaksi database modern, mahasiswa perlu memahami bagaimana output AI disimpan dalam kolom JSON, diekstrak melalui query, di filter berdasarkan entitas, dioptimalkan dengan *generated column* dan indeks, serta dilacak melalui *audit trail* dan *schema versioning*. Buku ini juga memperjelas pilihan teknologi *open source* agar pembelajaran tidak bergantung pada layanan tertutup: **Ollama** untuk eksperimen lokal, **vLLM** untuk *serving* skala lebih besar, **llama.cpp** untuk model GGUF dan grammar, serta **Instructor** dan **Outlines** untuk *constrained generation* yang lebih stabil. Namun pesan strategis buku ini lebih dalam: *structured output* bukan sekadar meminta LLM menulis JSON, melainkan membangun sistem yang tahu kapan harus menerima, menolak, mengulang, memperbaiki, atau meminta verifikasi manusia. Karena itu, reliability menjadi benang merah: JSON harus divalidasi, Pydantic dipakai sebagai pagar Python, error harus dicatat, output mentah harus disimpan, dan kualitas harus diukur melalui *valid JSON rate*, *schema compliance rate*, *precision*, dan *recall*. Dengan pendekatan 100 halaman, 10 bab, dan orientasi praktik, buku ini membimbing mahasiswa membangun sistem nyata: ekstraksi berita, NER, form intelligence, invoice, chatbot intent, RAG metadata, hingga workflow n8n. Hasil akhirnya bukan hanya pemahaman konsep, tetapi **portofolio mini-project AI berbasis database** yang siap didemokan, diuji, dikembangkan, dan dipertanggungjawabkan.

Daftar Isi

Kata Pengantar.....	4
Executive Summary.....	5
Daftar Isi.....	6
Bab 1 — Dari Teks Bebas ke Data Terstruktur: Mengapa Structured Output Penting..	18
1.1 Masalah Utama LLM: Pintar, tetapi Tidak Selalu Stabil.....	18
1.2 Mengapa Database Membutuhkan Data yang Machine-Readable.....	19
1.3 Dari JSON Biasa ke JSON Schema.....	20
1.4 Perbedaan Teks Bebas, JSON Biasa, JSON Mode, JSON Schema, dan Structured Output.....	21
1. Teks Bebas.....	21
2. JSON Biasa.....	21
3. JSON Mode.....	22
4. JSON Schema.....	22
5. Structured Output.....	22
1.5 Mengapa Structured Output Penting untuk Mahasiswa.....	23
1.6 Masalah Data: Ambiguitas, Celah Pengetahuan, dan Blind Spot.....	24
Ambiguitas.....	24
Celah Pengetahuan.....	24
Blind Spot.....	24
1.7 Pipeline Besar Buku: Dari Input Teks ke Sistem yang Dapat Diaudit.....	25
1. Input Teks.....	25
2. LLM.....	25
3. JSON Schema.....	25
4. Validasi.....	25
5. Database.....	25
6. Workflow Otomatis.....	26
7. Audit.....	26
1.8 Peran Python dalam Ekosistem Structured Output.....	26
1.9 Peran Open Source LLM dalam Kemandirian Teknologi.....	27
1.10 Use Case Mahasiswa.....	28
Ekstraksi Data Berita.....	28
Form Otomatis.....	28
NER.....	29
Invoice Sederhana.....	29
Survei.....	29
Chatbot Database.....	30
1.11 Risiko Jika LLM Langsung Dihubungkan ke Database.....	30
1.12 Cara Berpikir Baru: Dari Prompt Engineering ke Data Engineering.....	31
1.13 Prinsip Desain Awal Structured Output.....	32
1. Mulai dari Output yang Dibutuhkan.....	32
2. Schema Harus Sederhana.....	32
3. Validasi Wajib Ada.....	32

4. Error Harus Dicatat.....	32
5. Human-in-the-Loop untuk Data Sensitif.....	33
1.14 Peta Praktik yang Akan Dibangun di Bab Berikutnya.....	33
1.15 Ringkasan Bab.....	34
Bab 2 — Fondasi JSON dan JSON Schema untuk Aplikasi LLM.....	35
2.1 Mengapa JSON Menjadi Bahasa Antara Manusia, LLM, dan Database.....	35
2.2 Review JSON: Object, Array, String, Number, Boolean, Null.....	36
2.2.1 Object.....	36
2.2.2 Array.....	36
2.2.3 String.....	37
2.2.4 Number.....	37
2.2.5 Boolean.....	37
2.2.6 Null.....	37
2.3 Masalah JSON Biasa: Valid secara Bentuk, Belum Tentu Benar secara Isi.....	38
2.4 Apa Itu JSON Schema.....	38
2.5 Struktur Dasar JSON Schema.....	39
2.6 Constraint Penting dalam JSON Schema.....	40
2.6.1 enum.....	40
2.6.2 minimum dan maximum.....	40
2.6.3 minLength dan maxLength.....	41
2.6.4 pattern.....	41
2.7 Nested Object dan Array.....	42
2.8 Schema Sederhana vs Schema Terlalu Kompleks.....	43
2.9 Kesalahan Umum Saat Membuat Schema untuk LLM.....	44
1. Field Terlalu Banyak.....	44
2. Nama Field Ambigu.....	45
3. Tipe Data Tidak Konsisten.....	45
4. Semua Field Dibuat Opsional.....	45
5. Tidak Menolak Field Tambahan.....	46
2.10 Prinsip Desain Schema untuk LLM.....	47
1. Singkat.....	47
2. Eksplisit.....	47
3. Ketat.....	47
4. Mudah Diuji.....	47
5. Cocok untuk Database.....	48
Bagian Praktik.....	49
2.11 Praktik 1 — Membuat JSON Data Mahasiswa.....	49
2.12 Praktik 2 — Membuat Schema Data Berita.....	51
2.13 Praktik 3 — Membuat Schema Data Transaksi Sederhana.....	53
2.14 Praktik 4 — Validasi JSON Menggunakan Python.....	55
Instalasi pustaka.....	55
Script validasi sederhana.....	55
2.15 Praktik 5 — Menguji Data yang Salah.....	57
2.16 Praktik 6 — Menyimpan Schema dan Data dalam File.....	58

2.17 Hubungan JSON Schema dengan Database JSON.....	60
2.18 Catatan Keamanan: JSON Bukan Otomatis Aman.....	60
2.19 Menghubungkan Bab 2 ke Bab 3.....	61
2.20 Checklist Desain JSON Schema untuk Mahasiswa.....	61
2.21 Latihan Mandiri.....	62
Latihan 1 — Biodata Mahasiswa.....	62
Latihan 2 — Berita Kampus.....	62
Latihan 3 — Transaksi Sederhana.....	63
2.22 Ringkasan Bab.....	63
Bab 3 — Python, Pydantic, dan Validasi Output LLM.....	64
3.1 Mengapa Python Menjadi Alat Utama Eksperimen Structured Output.....	64
3.2 Dari JSON Schema ke Pydantic.....	65
3.3 Pydantic sebagai Model Data.....	66
3.4 Mengetahui Type Hints: str, int, float, bool, list, Optional.....	67
3.4.1 str.....	67
3.4.2 int.....	67
3.4.3 float.....	67
3.4.4 bool.....	68
3.4.5 list.....	68
3.4.6 Optional.....	68
3.5 Instalasi Pydantic.....	69
Bagian Praktik.....	70
3.6 Praktik 1 — Membuat Model Mahasiswa.....	70
3.7 Menangani Error Instalasi EmailStr.....	72
3.8 Praktik 2 — Menguji Data Mahasiswa yang Salah.....	72
3.9 Konsep Fail Fast.....	73
3.10 Praktik 3 — Membuat Model Berita.....	74
3.11 Praktik 4 — Membuat Model Produk.....	75
3.12 Mengubah Pydantic Model Menjadi JSON Schema.....	76
3.13 Mengapa Ini Penting untuk LLM.....	76
3.14 Parsing Output LLM Menjadi Object Python.....	77
3.15 Praktik 5 — Ekstraksi Data dari Teks Pendaftaran Mahasiswa.....	78
3.16 Praktik 6 — Menangani Output LLM yang Salah.....	79
3.17 Membuat Fungsi Validator agar Kode Lebih Rapi.....	80
3.18 Menolak Field Tambahan dari LLM.....	82
3.19 Membatasi Program Studi dengan Literal.....	83
3.20 Validasi Minat Riset.....	83
3.21 Menghubungkan Pydantic dengan JSON Schema untuk LLM Lokal.....	84
3.22 Menyimpan Output Valid dan Output Gagal.....	85
3.23 Pola Error Handling untuk Pipeline LLM.....	87
3.24 Checklist Validator Output LLM.....	88
3.25 Latihan Mandiri.....	89
Latihan 1 — Model Mahasiswa Lebih Ketat.....	89
Latihan 2 — Model Berita Kampus.....	89

Latihan 3 — Model Produk.....	89
Latihan 4 — Output LLM Salah.....	90
3.26 Ringkasan Bab.....	90
Bab 4 — Structured Output dengan Open Source LLM: Ollama, vLLM, dan llama.cpp	91
4.1 Mengapa Structured Output di Open Source LLM Penting.....	91
4.2 Posisi Ollama, vLLM, dan llama.cpp.....	92
4.3 Prompt Biasa vs JSON Schema vs Grammar.....	93
4.3.1 Prompt Biasa.....	93
4.3.2 JSON Schema.....	93
4.3.3 Grammar.....	93
4.4 Temperatur, Determinisme, dan Risiko Output Rusak.....	94
4.5 Kapan Cukup Prompt, Kapan Perlu Schema, Kapan Perlu Grammar.....	94
Bagian Praktik.....	95
4.6 Praktik 1 — Menyiapkan Model Berita Kampus dengan Pydantic.....	95
4.7 Membuat Model BeritaKampus.....	97
4.8 Membuat JSON Schema dari Pydantic.....	98
4.9 Praktik 2 — Menjalankan Structured Output dengan Ollama.....	98
4.10 Membuat Prompt Ekstraksi Berita Kampus.....	99
4.11 Mengirim Schema dari Python ke Ollama.....	100
4.12 Memvalidasi Output Ollama dengan Pydantic.....	102
4.13 Menyimpan Hasil Valid dan Error.....	103
4.14 Menyatukan Semua Pipeline.....	104
4.15 Kode Lengkap Praktek Ollama.....	105
4.16 Hasil JSON yang Diharapkan.....	108
4.17 Praktik 3 — Membandingkan Prompt Bebas dan Schema-Based Output.....	109
Prompt bebas.....	109
Prompt berbasis schema.....	109
4.18 Praktik 4 — Eksperimen dengan Model Kecil.....	110
4.19 Praktik 5 — Menambahkan Retry Sederhana.....	111
4.20 vLLM untuk Serving Model Skala Lebih Besar.....	112
4.21 llama.cpp dan Grammar GBNF.....	113
4.22 Risiko dan Batasan Structured Output.....	114
4.23 Desain Pipeline Bab 4.....	115
4.24 Checklist Praktik Structured Output.....	116
4.25 Latihan Mandiri.....	117
Latihan 1 — Ekstraksi Berita Kampus.....	117
Latihan 2 — Bandingkan Temperature.....	117
Latihan 3 — Tambahkan Field jenis_acara.....	117
Latihan 4 — Buat Retry dan Log.....	117
Latihan 5 — Bandingkan Prompt Bebas vs Schema.....	117
4.26 Ringkasan Bab.....	118
Bab 5 — Constrained Generation dengan Instructor dan Outlines.....	119
5.1 Apa Itu Constrained Generation.....	119
5.2 Mengapa Teknik Ini Penting untuk Mahasiswa.....	120

5.3 Validation-after-generation vs Constrained-during-generation.....	121
5.3.1 Validation-after-generation.....	121
5.3.2 Constrained-during-generation.....	121
5.4 Instructor: Output Type-Safe Berbasis Pydantic.....	123
5.5 Outlines: JSON Structured Generation.....	123
5.6 Instructor vs Outlines.....	124
5.7 Keterbatasan Constrained Generation.....	125
1. Schema kompleks dapat memperlambat proses.....	125
2. Model kecil bisa tetap salah secara makna.....	125
3. Validasi fakta tetap diperlukan.....	125
4. Logging tetap wajib.....	125
Bagian Praktik.....	126
5.8 Target Praktik: NER Sederhana.....	126
5.9 Persiapan Folder Praktik.....	127
5.10 Membuat Model Pydantic untuk NER.....	128
5.11 Praktik 1 — Prompt Biasa.....	129
5.12 Praktik 2 — Pydantic Validation Setelah Output.....	130
5.13 Praktik 3 — Extractor dengan Instructor.....	131
5.14 Praktik 4 — Generator JSON dengan Outlines.....	133
5.15 Praktik 5 — Membandingkan Tiga Pendekatan.....	134
5.16 Script Evaluasi Error Sederhana.....	135
5.17 Mengukur Error yang Penting.....	137
1. JSON Invalid.....	137
2. Field Hilang.....	137
3. Tipe Salah.....	137
4. Field Tambahan.....	137
5. Salah Kategori.....	138
5.18 Strategi Praktis agar Sistem Lebih Reliable.....	138
1. Mulai dari schema kecil.....	138
2. Uji bertahap.....	138
3. Logging semua error.....	138
4. Gunakan retry dengan batas.....	139
5. Pisahkan validasi struktur dan evaluasi makna.....	139
5.19 Hubungan dengan Database.....	140
5.20 Checklist Praktik Bab 5.....	141
5.21 Latihan Mandiri.....	142
Latihan 1 — NER Berita Kampus.....	142
Latihan 2 — Tambahkan Field event_type.....	142
Latihan 3 — Uji Teks Tanpa Entitas.....	142
Latihan 4 — Hitung Error Rate.....	142
Latihan 5 — Audit Output.....	143
5.22 Ringkasan Bab.....	143
Bab 6 — Menyimpan Structured Output ke Database JSON.....	144
6.1 Mengapa Output LLM Harus Masuk Database.....	144

6.2 Kolom Biasa vs Kolom JSON.....	145
Kolom biasa cocok untuk:.....	145
Kolom JSON cocok untuk:.....	146
6.3 Catatan Penting tentang MySQL dan MariaDB.....	146
6.4 Desain Tabel untuk Output LLM.....	147
6.5 Membuat Database dan Tabel.....	148
6.6 Mengapa source_text Harus Disimpan.....	149
6.7 Insert JSON Hasil LLM.....	150
6.8 Query Field JSON.....	151
6.9 Query Berdasarkan Lokasi.....	152
6.10 Query Berdasarkan Aktor dalam Array.....	153
6.11 Query Berdasarkan Topik.....	153
6.12 Mengubah JSON Menjadi Tabel dengan JSON_TABLE.....	154
6.13 Agregasi Topik.....	155
6.14 Kolom Biasa dari Field JSON: Generated Column.....	156
6.15 Membuat Index pada Generated Column.....	157
6.16 Generated Column untuk Tanggal.....	157
6.17 Desain Tabel yang Lebih Kuat.....	159
6.18 Audit Trail: Apa Saja yang Harus Disimpan.....	160
Bagian Praktik.....	161
6.19 Praktik Pipeline: Teks Berita → JSON Valid → Database.....	161
6.20 Instalasi Library Python.....	161
6.21 Membuat File Konfigurasi Sederhana.....	162
6.22 Script Insert JSON dari Python.....	163
6.23 Catatan Keamanan SQL.....	166
6.24 Query Output dari Python.....	167
6.25 Membuat Laporan Ringkas dari Data AI.....	168
6.26 Menyimpan Error Validasi.....	169
6.27 Tabel Tambahan: validation_logs.....	170
6.28 Tabel Tambahan: schema_versions.....	171
6.29 Query Audit: Melihat Output Berdasarkan Schema dan Model.....	172
6.30 Query Audit: Melihat Error Terbaru.....	173
6.31 Mengapa Tidak Semua Output AI Perlu Dipecah ke Banyak Tabel.....	174
Pendekatan 1 — Simpan JSON sebagai dokumen.....	174
Pendekatan 2 — Pecah ke tabel relasional.....	174
6.32 Pipeline Praktik Lengkap.....	175
6.33 Checklist Desain Database JSON untuk Output LLM.....	176
6.34 Latihan Mandiri.....	177
Latihan 1 — Insert 5 Berita Kampus.....	177
Latihan 2 — Query Berdasarkan Aktor.....	177
Latihan 3 — Query Berdasarkan Lokasi.....	177
Latihan 4 — Laporan Topik.....	178
Latihan 5 — Audit Model.....	178
6.35 Kesalahan Umum.....	179

1. Menyimpan hanya JSON tanpa input asli.....	179
2. Tidak menyimpan schema version.....	179
3. Tidak menyimpan model.....	179
4. Semua data dimasukkan ke JSON.....	179
5. Tidak memakai generated column.....	179
6. Tidak menyimpan error.....	179
7. Menganggap JSON valid berarti data benar.....	179
6.36 Ringkasan Bab.....	180
Bab 7 — Otomatisasi Workflow Structured Output dengan n8n.....	181
7.1 Posisi n8n dalam Pipeline AI.....	181
7.2 Mengapa n8n Penting untuk Mahasiswa.....	182
7.3 Kapan Memakai Python, Kapan Memakai n8n.....	183
7.4 Node Penting dalam Workflow Structured Output.....	184
7.4.1 Webhook Node.....	184
7.4.2 HTTP Request Node.....	184
7.4.3 AI Agent / LLM Node.....	184
7.4.4 Structured Output Parser Node.....	185
7.4.5 Code Node.....	185
7.4.6 Database Node.....	186
7.5 Validasi Schema di Workflow.....	187
Lapisan 1 — Structured Output Parser.....	187
Lapisan 2 — Code Node.....	187
Lapisan 3 — Python/Pydantic Service.....	187
7.6 Error Branch dan Retry.....	188
7.7 Hybrid Architecture: n8n + Python + Database.....	189
Bagian Praktik.....	190
7.8 Contoh Praktik: Sistem Input Form Mahasiswa.....	190
7.9 JSON Schema untuk Data Mahasiswa.....	191
7.10 Workflow 1 — Webhook → LLM → JSON Schema → Database.....	192
Struktur node.....	192
Fungsi setiap node.....	192
7.11 Contoh Payload Webhook.....	193
7.12 Code Node untuk Menyiapkan Payload.....	194
7.13 Workflow 2 — Input Form → Extraction → Validation → Database.....	195
7.14 Python Validator Service Sederhana.....	196
7.15 HTTP Request dari n8n ke Python Validator.....	198
7.16 IF Node: Cabang Valid dan Invalid.....	199
7.17 Desain Tabel Database untuk Workflow.....	200
7.18 Workflow 3 — Berita Online → Structured Summary → Database.....	201
7.19 Logging Workflow untuk Debugging.....	202
7.20 Membuat Error Handling Path.....	203
7.21 Strategi Retry.....	204
7.22 Checklist Workflow n8n untuk Structured Output.....	205
7.23 Kesalahan Umum.....	206

1. Menganggap Parser Sama dengan Validasi Penuh.....	206
2. Tidak Menyimpan Input Asli.....	206
3. Tidak Membuat Jalur Error.....	206
4. Terlalu Banyak Logika di Code Node.....	206
5. Tidak Memberi Versi Schema.....	206
6. Menggunakan Schema Terlalu Kompleks.....	206
7. Tidak Menguji dengan Data Buruk.....	206
7.24 Latihan Mandiri.....	207
Latihan 1 — Form Mahasiswa.....	207
Latihan 2 — Cabang Error.....	207
Latihan 3 — Retry.....	207
Latihan 4 — Workflow Berita.....	207
Latihan 5 — Laporan Workflow.....	208
7.25 Ringkasan Bab.....	208
Bab 8 — Studi Kasus Interaksi Database Berbasis Structured Output.....	209
8.1 Peta Besar Bab.....	209
8.2 Struktur Folder Proyek.....	210
8.3 Tabel Audit Umum.....	211
Studi Kasus 1 — Ekstraksi Data Berita Kampus.....	212
8.4 Tujuan Studi Kasus.....	212
8.5 Schema Berita Kampus.....	212
8.6 Tabel Berita Kampus.....	213
8.7 Model Pydantic Berita.....	214
8.8 Prompt Ekstraksi Berita.....	215
8.9 Query Berdasarkan Organisasi dan Lokasi.....	216
Studi Kasus 2 — Named Entity Recognition Bahasa Indonesia.....	217
8.10 Tujuan Studi Kasus.....	217
8.11 Schema NER.....	217
8.12 Tabel NER Relasional.....	218
8.13 Model Pydantic NER.....	219
8.14 Contoh Output NER.....	220
8.15 Insert NER ke Database.....	221
8.16 Query Top Entity.....	222
Studi Kasus 3 — Form Intelligence.....	223
8.17 Tujuan Studi Kasus.....	223
8.18 Tabel Form Mahasiswa.....	223
8.19 Model Pydantic Form Mahasiswa.....	224
8.20 Insert Form Mahasiswa.....	225
8.21 Query Form Mahasiswa.....	226
Studi Kasus 4 — Invoice / Kwitansi Sederhana.....	227
8.22 Tujuan Studi Kasus.....	227
8.23 Schema Invoice.....	227
8.24 Model Pydantic Invoice.....	229
8.25 Tabel Invoice.....	230

8.26 Contoh Input OCR.....	231
8.27 Query Invoice.....	232
Studi Kasus 5 — AI Agent dengan Tool Calling Berbasis JSON Schema.....	233
8.28 Tujuan Studi Kasus.....	233
8.29 Schema Tool Action.....	233
8.30 Model Pydantic Tool Action.....	234
8.31 Contoh Input dan Output Tool Calling.....	235
8.32 Dispatcher Aman di Python.....	236
8.33 Query Summary Invoice.....	237
8.34 Mengapa Tool Calling Harus Dibatasi Schema.....	238
8.35 Logging Tool Action.....	238
8.36 Checklist Mini-Project Bab 8.....	239
8.37 Strategi Demo Portofolio.....	239
8.38 Kesalahan Umum.....	240
1. Langsung menyimpan output LLM.....	240
2. Tidak menyimpan input asli.....	240
3. Semua field disimpan sebagai JSON.....	240
4. LLM diminta membuat SQL bebas.....	240
5. Tidak memisahkan data valid dan error.....	240
6. Tidak memberi versi schema.....	240
7. Terlalu banyak field sejak awal.....	240
8.39 Latihan Mandiri.....	241
Latihan 1 — Berita Kampus.....	241
Latihan 2 — NER Bahasa Indonesia.....	241
Latihan 3 — Form Intelligence.....	241
Latihan 4 — Invoice Sederhana.....	241
Latihan 5 — Tool Calling Aman.....	242
8.40 Ringkasan Bab.....	242
Bab 9 — Reliability, Error Handling, Audit, dan Evaluasi Kualitas Output.....	243
9.1 Mengapa Reliability Lebih Penting daripada Demo yang Terlihat Pintar.....	243
9.2 Jenis Error dalam Structured Output.....	244
9.2.1 JSON Invalid.....	244
9.2.2 Field Hilang.....	244
9.2.3 Tipe Data Salah.....	245
9.2.4 Hallucinated Field.....	245
9.2.5 Nilai Tidak Masuk Akal.....	246
9.2.6 Duplikasi Data.....	246
9.3 Strategi Retry.....	247
9.4 Praktik 1 — Retry Otomatis di Python.....	248
9.5 Fallback Jika Output Gagal Validasi.....	249
9.6 Schema Versioning.....	250
9.7 Logging Prompt, Model, Parameter, Latency, dan Error.....	252
9.8 Praktik 2 — Logger Error di Python.....	253
9.9 Human-in-the-Loop untuk Data Sensitif.....	255

9.10 Evaluasi Kualitas Output.....	256
9.10.1 Valid JSON Rate.....	256
9.10.2 Schema Compliance Rate.....	256
9.10.3 Extraction Accuracy.....	256
9.10.4 False Positive.....	256
9.10.5 False Negative.....	256
9.11 Tabel Evaluasi Output.....	257
9.12 Praktik 3 — Menghitung Metrik Sederhana dengan Python.....	258
9.13 Praktik 4 — Precision dan Recall untuk NER.....	259
9.14 Dashboard Sederhana Valid vs Gagal.....	260
9.15 Error Handling di Workflow n8n.....	261
9.16 Keamanan Data dan Etika Penggunaan LLM.....	262
9.17 Checklist Audit.....	263
9.18 Latihan Mandiri.....	264
Latihan 1 — Buat Tabel Error Log.....	264
Latihan 2 — Retry Otomatis.....	264
Latihan 3 — Fallback Review Manual.....	264
Latihan 4 — Dashboard Valid vs Gagal.....	264
Latihan 5 — Evaluasi NER.....	264
9.19 Ringkasan Bab.....	265
Bab 10 — Proyek Akhir: Membangun Sistem AI + Database yang Siap Didemokan..	266
10.1 Tujuan Proyek Akhir.....	266
10.2 Pilihan Proyek Akhir.....	267
Pilihan 1 — Sistem Ekstraksi Berita Indonesia ke Database.....	267
Pilihan 2 — Sistem NER Otomatis untuk Dokumen Kampus.....	268
Pilihan 3 — Sistem Input Form Pintar Berbasis LLM.....	268
Pilihan 4 — Sistem Ekstraksi Invoice / Kwitansi Sederhana.....	269
Pilihan 5 — Sistem Chatbot Database dengan Structured Query Intent.....	270
Pilihan 6 — Sistem RAG Sederhana dengan Metadata JSON.....	270
Pilihan 7 — Sistem Klasifikasi Keluhan Mahasiswa ke Database.....	271
Pilihan 8 — Sistem Laporan Otomatis dari Data JSON.....	271
10.3 Komponen Wajib Proyek.....	272
10.4 Arsitektur Minimal Proyek.....	273
10.5 Struktur Folder Proyek Akhir.....	274
10.6 Template Model Pydantic.....	275
10.7 Template Tabel Database.....	276
10.8 Template Pipeline Python End-to-End.....	277
10.9 Evaluasi Kualitas Output.....	279
10.10 Dokumentasi Demo.....	280
10.11 Screenshot yang Wajib Ada.....	281
10.12 Rubrik Penilaian.....	281
Detail Penilaian.....	281
10.13 Checklist Sebelum Demo.....	283
10.14 Pola Presentasi 5 Menit.....	284

10.15 Ringkasan Bab.....	284
Lampiran A — Cheat Sheet JSON Schema.....	285
A.1 Keyword Penting JSON Schema.....	285
A.2 Contoh Schema Mahasiswa.....	286
A.3 Contoh Schema Berita.....	287
A.4 Contoh Schema Invoice.....	288
A.5 Contoh Schema NER.....	289
Lampiran B — Template Pydantic Model.....	290
B.1 Model Mahasiswa.....	290
B.2 Model Berita.....	290
B.3 Model Entitas.....	291
B.4 Model Invoice.....	291
B.5 Model Intent Chatbot.....	292
B.6 Contoh Validasi Error.....	292
Lampiran C — Template Tabel Database JSON.....	293
C.1 Tabel Ilm_outputs.....	293
C.2 Tabel documents.....	293
C.3 Tabel entities.....	294
C.4 Tabel validation_logs.....	294
C.5 Tabel schema_versions.....	294
Lampiran D — Contoh Query SQL untuk JSON.....	295
D.1 Insert JSON.....	295
D.2 Extract Field.....	295
D.3 Filter Nested Field / Array.....	296
D.4 Generated Column.....	296
D.5 Indexing Generated Column.....	296
D.6 Agregasi Entitas.....	296
D.7 Agregasi Topik dari JSON Array.....	297
Lampiran E — Template Workflow n8n.....	298
E.1 Workflow 1 — Webhook → LLM → Structured Output Parser → Database.....	298
E.2 Workflow 2 — Email / Form Input → Extract → Validate → Store.....	299
E.3 Workflow 3 — HTTP API → Python Validator → Database.....	299
E.4 Error Branch dan Retry.....	300
E.5 Template Data Log Workflow.....	300
Lampiran F — 30 Prompt Structured Output untuk Mahasiswa.....	301
F.1 Ekstraksi Biodata.....	301
F.2 Ekstraksi Berita.....	302
F.3 NER.....	303
F.4 Invoice.....	304
F.5 Klasifikasi.....	305
F.6 Form Intelligence.....	306
F.7 Database Intent.....	307
F.8 Validasi JSON.....	308
F.9 Audit Output.....	309

F.10 Debugging Schema.....	310
Ringkasan Lampiran.....	311
Glossary.....	312
Referensi.....	322
Tentang Penulis.....	325

Bab 1 — Dari Teks Bebas ke Data Terstruktur: Mengapa Structured Output Penting

Golden Rule: LLM tidak boleh dipercaya sebelum *output*-nya divalidasi, distrukturkan, disimpan, dan dapat diaudit. Prinsip ini sejalan dengan arah utama dokumen buku yang menempatkan *structured output* sebagai jembatan dari jawaban AI yang bebas menuju data yang dapat dipakai sistem informasi.

1.1 Masalah Utama LLM: Pintar, tetapi Tidak Selalu Stabil

LLM mampu menulis ringkasan, menjawab pertanyaan, membuat klasifikasi, dan mengekstrak informasi dari teks. Masalahnya, jawaban LLM sering hadir dalam bentuk **teks bebas**. Bagi manusia, teks bebas mudah dibaca. Bagi sistem informasi, teks bebas sering menjadi masalah.

Sebuah aplikasi database tidak cukup hanya menerima kalimat seperti:

“Nama mahasiswa adalah Ahmad, NIM-nya 230101, program studinya Informatika, dan ia tertarik pada AI.”

Kalimat itu terlihat jelas bagi manusia. Tetapi bagi mesin, masih perlu ditanya:

- Mana field **nama**?
- Mana field **NIM**?
- Apakah NIM angka atau teks?
- Apakah program studi harus mengikuti daftar resmi?
- Apakah minat riset boleh lebih dari satu?
- Apakah email wajib ada?
- Apakah data sudah valid sebelum masuk database?

Di sinilah masalah LLM menjadi serius. **Jawaban yang terdengar benar belum tentu siap diproses**. LLM dapat menulis format berbeda untuk input yang mirip. Hari ini ia menulis *nama*, besok *Nama Lengkap*, lalu lusa *student_name*. Bagi manusia, variasi ini wajar. Bagi database, variasi ini bisa merusak pipeline.

Masalah yang paling sering muncul adalah:

- **Format berubah-ubah** walaupun prompt sama.
- **Field hilang** karena model menganggap informasi tertentu tidak penting.
- **Nama field tidak konsisten** sehingga aplikasi gagal membaca data.
- **Tipe data salah**, misalnya angka ditulis sebagai teks bebas.
- **Nilai tidak masuk akal**, misalnya tahun lahir lebih besar dari tahun sekarang.
- **Field palsu muncul**, yaitu data yang tidak diminta tetapi dibuat oleh model.
- **Struktur tidak valid**, misalnya kurung JSON tidak lengkap.
- **Output sulit diaudit** karena tidak ada aturan formal yang dipakai.

Masalah ini membuat LLM berisiko jika langsung dihubungkan ke database, workflow, atau aplikasi kampus. **AI yang menarik untuk demo belum tentu aman untuk sistem nyata.**

1.2 Mengapa Database Membutuhkan Data yang Machine-Readable

Database tidak bekerja dengan “kesan”. Database bekerja dengan **struktur**. Aplikasi membutuhkan data yang bisa dibaca, dicek, dicari, difilter, dihitung, dan digabungkan dengan data lain.

Data yang baik untuk sistem informasi harus memiliki beberapa ciri:

- **Konsisten**, nama field tidak berubah-ubah.
- **Eksplisit**, arti setiap field jelas.
- **Valid**, tipe data dan batas nilainya dapat diuji.
- **Lengkap**, field wajib tidak boleh hilang.
- **Terlacak**, asal data dan proses validasinya dapat diaudit.
- **Dapat dipakai ulang**, bukan hanya dibaca sekali oleh manusia.

JSON menjadi format penting karena ringan, mudah dibaca manusia, dan mudah diproses program. JSON sendiri adalah format pertukaran data berbasis teks yang sederhana, memakai objek, array, string, number, boolean, dan `null`; dokumen referensi MySQL & JSON juga menekankan JSON sebagai format yang mudah dibaca dan relatif mudah diparsing oleh program.

Namun JSON saja belum cukup. JSON dapat valid secara sintaks, tetapi tetap buruk secara isi. Contoh:

```
{
  "nama": "Ahmad",
  "nim": 230101,
  "prodi": "Informatika",
  "umur": -5
}
```

Secara bentuk, JSON ini bisa terlihat rapi. Tetapi secara makna, `umur: -5` tidak masuk akal. Karena itu, kita membutuhkan **schema**.

1.3 Dari JSON Biasa ke JSON Schema

JSON biasa menjawab pertanyaan: “**Bagaimana data ditulis?**”

JSON Schema menjawab pertanyaan: “**Data seperti apa yang boleh diterima?**”

JSON Schema adalah bahasa deklaratif untuk mendeskripsikan struktur, tipe data, dan aturan validasi dokumen JSON; dokumentasi resminya menjelaskan JSON Schema sebagai cara untuk menstandarisasi dan mendefinisikan ekspektasi terhadap data JSON.

(json-schema.org)

Dengan JSON Schema, kita dapat menentukan:

- Field apa saja yang boleh ada.
- Field apa saja yang wajib ada.
- Tipe data setiap field.
- Nilai minimum dan maksimum.
- Pola teks tertentu, misalnya format email sederhana.
- Daftar nilai yang diperbolehkan.
- Struktur objek bertingkat.
- Struktur array.
- Apakah field tambahan boleh diterima atau harus ditolak.

Contoh sederhana:

```
{
  "type": "object",
  "properties": {
    "nama": { "type": "string" },
    "nim": { "type": "string" },
    "prodi": { "type": "string" },
    "minat_riset": {
      "type": "array",
      "items": { "type": "string" }
    }
  },
  "required": ["nama", "nim", "prodi"]
}
```

Schema ini memberi kontrak sederhana:

- **nama** harus teks.
- **nim** harus teks.
- **prodi** harus teks.
- **minat_riset** berbentuk array berisi teks.
- **nama**, **nim**, dan **prodi** wajib ada.

Dalam buku ini, JSON Schema tidak diposisikan sebagai teori tambahan. JSON Schema adalah **kontrak data** antara LLM, Python, validator, database, dan workflow. Outline buku juga menegaskan bahwa JSON Schema menjadi “kontrak” antara LLM, program Python, workflow n8n, dan database.

1.4 Perbedaan Teks Bebas, JSON Biasa, JSON Mode, JSON Schema, dan Structured Output

Mahasiswa perlu memahami lima tingkat kedisiplinan output.

1. Teks Bebas

Teks bebas adalah jawaban natural seperti paragraf biasa.

Contoh:

“Mahasiswa bernama Rina, NIM 230045, dari Prodi Sistem Informasi, tertarik pada data analytics dan AI.”

Kelebihan:

- Mudah dibaca manusia.
- Cocok untuk penjelasan.
- Cocok untuk narasi.

Kelemahan:

- Sulit diproses otomatis.
- Field tidak eksplisit.
- Format mudah berubah.
- Risiko ambigu tinggi.

2. JSON Biasa

JSON biasa sudah lebih terstruktur.

```
{
  "nama": "Rina",
  "nim": "230045",
  "prodi": "Sistem Informasi",
  "minat": ["data analytics", "AI"]
}
```

Kelebihan:

- Mudah diproses Python.
- Bisa disimpan di database.
- Lebih rapi daripada teks bebas.

Kelemahan:

- Belum tentu mengikuti aturan yang sama.
- Field bisa berubah.
- Tipe data bisa tidak konsisten.
- Belum tentu semua field wajib ada.

3. JSON Mode

JSON mode biasanya berarti model diarahkan agar menghasilkan JSON valid. Ini membantu, tetapi belum cukup. JSON yang valid belum tentu sesuai kebutuhan aplikasi.

Masalahnya:

- JSON bisa valid, tetapi field salah.
- JSON bisa valid, tetapi nilai tidak masuk akal.
- JSON bisa valid, tetapi tidak sesuai kontrak database.

4. JSON Schema

JSON Schema membuat aturan formal. Ia tidak hanya meminta model menulis JSON, tetapi mendefinisikan **struktur yang diharapkan**.

Kekuatan JSON Schema:

- Membatasi tipe data.
- Menentukan field wajib.
- Mengurangi variasi output.
- Memudahkan validasi otomatis.
- Menjadi dokumentasi teknis yang jelas.

5. Structured Output

Structured output adalah praktik membuat LLM menghasilkan data sesuai struktur tertentu, biasanya memakai schema. Dalam konteks Ollama, dokumentasi resminya menyebut *structured outputs* sebagai cara untuk menerapkan JSON Schema pada respons model agar ekstraksi data lebih reliabel dan konsisten. (docs.ollama.com)

Dengan *structured output*, targetnya bukan sekadar “LLM bisa menjawab”. Targetnya adalah:

- LLM menghasilkan data.
- Data mengikuti schema.
- Data lolos validasi.
- Data bisa masuk database.
- Data bisa dipakai workflow.
- Data bisa diaudit.

Ini adalah pergeseran penting: **dari AI sebagai penulis teks menuju AI sebagai produsen data.**

1.5 Mengapa Structured Output Penting untuk Mahasiswa

Bagi mahasiswa, *structured output* adalah keterampilan strategis. Ini bukan hanya teknik prompt. Ini adalah fondasi untuk membangun aplikasi AI yang lebih serius.

Mahasiswa yang memahami *structured output* dapat membuat:

- Sistem ekstraksi berita.
- Sistem NER untuk dokumen kampus.
- Sistem pembacaan invoice sederhana.
- Sistem input form otomatis.
- Sistem klasifikasi keluhan mahasiswa.
- Sistem chatbot yang dapat membaca dan menulis data.
- Sistem ringkasan survei berbasis database.
- Sistem workflow otomatis berbasis JSON.

Tanpa *structured output*, banyak aplikasi AI berhenti sebagai demo. Dengan *structured output*, aplikasi mulai bisa masuk ke sistem nyata.

Contoh perbedaan:

Tanpa structured output:

“Berita ini membahas seminar AI di kampus ITTS pada 12 Mei 2026 dengan pembicara utama Dr. Budi.”

Sulit langsung dimasukkan ke tabel.

Dengan structured output:

```
{
  "judul": "Seminar AI di Kampus ITTS",
  "tanggal": "2026-05-12",
  "lokasi": "ITTS",
  "aktor": ["Dr. Budi"],
  "topik": ["AI", "seminar kampus"]
}
```

Output kedua dapat:

- Disimpan.
- Dicari.
- Difilter berdasarkan tanggal.
- Digabungkan dengan data acara lain.
- Dipakai untuk dashboard.
- Divalidasi.
- Diaudit.

Nilai praktisnya jelas: mahasiswa tidak hanya belajar memakai LLM, tetapi belajar membuat LLM menjadi bagian dari pipeline data.

1.6 Masalah Data: Ambiguitas, Celah Pengetahuan, dan Blind Spot

Salah satu pelajaran penting dari literatur JSON Schema dan data product adalah bahwa masalah data sering tidak terlihat pada awalnya. Buku *Unifying Business, Data, and Code* menekankan tiga ancaman utama dalam data hygiene: **ambiguity**, **knowledge gaps**, dan **blind spots**.

Dalam konteks LLM, tiga masalah ini sangat nyata.

Ambiguitas

Ambiguitas terjadi ketika satu data dapat ditafsirkan lebih dari satu cara.

Contoh:

```
{
  "tanggal": "05/06/2026"
}
```

Apakah ini 5 Juni atau 6 Mei?

Tanpa aturan, sistem bisa salah membaca. Dalam aplikasi akademik, kesehatan, keuangan, logistik, atau hukum, kesalahan kecil seperti ini bisa berdampak besar.

Celah Pengetahuan

Celah pengetahuan terjadi ketika informasi penting tidak tersedia.

Contoh:

```
{
  "nama": "Dina",
  "prodi": "TI"
}
```

Apakah TI berarti Teknik Informatika atau Teknik Industri? Apakah kampus memakai kode resmi? Apakah format prodi harus mengikuti daftar tertentu?

Blind Spot

Blind spot terjadi ketika tim tidak sadar bahwa data mereka bermasalah.

Contoh:

- Semua mengira `field` status jelas.
- Tetapi ada nilai `aktif`, `Aktif`, `ACTIVE`, `active`, dan `masih kuliah`.
- Sistem menganggap itu lima kategori berbeda.

Structured output membantu mengurangi masalah ini karena schema memaksa tim menjawab pertanyaan dasar: **data apa yang benar-benar kita butuhkan, dalam bentuk apa, dan dengan aturan apa?**

1.7 Pipeline Besar Buku: Dari Input Teks ke Sistem yang Dapat Diaudit

Buku ini memakai alur besar:

input teks → LLM → JSON Schema → validasi → database → workflow otomatis → audit

Alur ini penting karena LLM tidak berdiri sendiri. LLM harus menjadi bagian dari sistem yang lebih besar.

1. Input Teks

Input dapat berupa:

- Berita.
- Biodata mahasiswa.
- Form pendaftaran.
- Email kampus.
- Dokumen invoice.
- Hasil survei.
- Transkrip wawancara.
- Chat pengguna.

Input biasanya tidak rapi. Tugas LLM adalah membantu mengekstrak informasi penting.

2. LLM

LLM membaca input dan menghasilkan kandidat output. Kata “kandidat” penting. Output LLM belum boleh dianggap benar.

3. JSON Schema

Schema mendefinisikan bentuk output yang boleh diterima. Ini menjadi kontrak.

4. Validasi

Validator memeriksa apakah output sesuai schema. Jika gagal, sistem dapat menolak, meminta ulang, atau menandai error.

5. Database

Data valid disimpan. Data tidak valid jangan langsung masuk ke tabel utama. Ini prinsip dasar data hygiene.

6. Workflow Otomatis

Workflow dapat mengirim data ke dashboard, laporan, notifikasi, atau proses lanjutan. Dalam outline buku, n8n diposisikan sebagai salah satu alat workflow untuk menghubungkan webhook, LLM, parser, validator, dan database.

7. Audit

Audit menjawab pertanyaan:

- Input awalnya apa?
- Prompt yang dipakai apa?
- Model yang digunakan apa?
- Schema versi berapa?
- Validasi berhasil atau gagal?
- Error apa yang terjadi?
- Siapa yang memeriksa output sensitif?

Tanpa audit, sistem AI sulit dipercaya. Dengan audit, sistem AI dapat diperbaiki.

1.8 Peran Python dalam Ekosistem Structured Output

Python penting karena menjadi penghubung antara LLM, schema, validator, dan database. Python juga memiliki ekosistem yang kuat untuk pemrosesan JSON, validasi data, otomasi, dan eksperimen AI.

Dalam praktik modern, mahasiswa perlu memahami konsep:

- Membaca teks.
- Mengirim prompt ke LLM lokal.
- Menerima output JSON.
- Memvalidasi output.
- Menyimpan data.
- Membuat log.
- Mengulang proses jika validasi gagal.

Python juga menyediakan *virtual environment* untuk mengisolasi paket per proyek. Dokumentasi resmi Python menjelaskan bahwa modul `venv` membuat lingkungan ringan yang memiliki paket sendiri dan terpisah dari instalasi dasar Python. ([Python documentation](#))

Namun Bab 1 tidak masuk ke perintah teknis. Detail instalasi dan kode akan dibahas di bab berikutnya. Di Bab 1, yang penting adalah memahami posisi Python sebagai **lapisan kendali**.

1.9 Peran Open Source LLM dalam Kemandirian Teknologi

Open source LLM penting karena memberi mahasiswa ruang belajar yang lebih bebas. Mahasiswa dapat bereksperimen secara lokal, memahami alur data, menguji model, dan membangun prototipe tanpa selalu bergantung pada layanan tertutup.

Peran open source LLM dalam buku ini:

- **Belajar lebih transparan**, karena pipeline dapat dilihat dari input sampai output.
- **Eksperimen lebih mandiri**, terutama untuk tugas kampus dan riset.
- **Kontrol data lebih baik**, karena data tidak harus selalu keluar dari lingkungan lokal.
- **Biaya belajar lebih terkendali**, terutama untuk latihan berulang.
- **Cocok untuk portofolio**, karena mahasiswa dapat mendemokan sistem end-to-end.

Ollama menjadi salah satu alat penting karena mendukung *structured output* berbasis JSON Schema pada respons model, sehingga cocok untuk eksperimen lokal dan pembelajaran structured data. (docs.ollama.com)

Tetapi prinsipnya tetap sama: **model lokal bukan berarti otomatis benar**. Output tetap harus divalidasi.

1.10 Use Case Mahasiswa

Bab ini menyajikan cara berpikir untuk berbagai studi kasus.

Ekstraksi Data Berita

Input berupa artikel berita kampus. Output berupa JSON:

- judul
- tanggal
- lokasi
- aktor
- organisasi
- topik
- ringkasan

Manfaat:

- Membuat arsip berita.
- Membuat dashboard isu kampus.
- Membuat pencarian berbasis metadata.

Form Otomatis

Input berupa teks bebas dari calon mahasiswa. Output berupa JSON:

- nama
- email
- nomor kontak
- prodi pilihan
- jalur pendaftaran
- minat

Manfaat:

- Mengurangi input manual.
- Membantu validasi awal.
- Menyiapkan data untuk database pendaftaran.

NER

Input berupa dokumen. Output berupa entitas:

- orang
- organisasi
- lokasi
- tanggal
- topik
- produk
- kegiatan

Manfaat:

- Membuat indeks dokumen.
- Membantu riset teks.
- Membuat dataset NLP sederhana.

Invoice Sederhana

Input berupa teks invoice. Output berupa JSON:

- nomor invoice
- tanggal
- nama vendor
- daftar item
- total
- pajak
- status pembayaran

Manfaat:

- Membuat sistem pencatatan.
- Mengurangi salah input.
- Menyiapkan audit dokumen.

Survei

Input berupa jawaban responden. Output berupa JSON:

- identitas anonim
- kategori jawaban
- sentimen
- isu utama
- rekomendasi tindak lanjut

Manfaat:

- Membantu analisis kualitatif.
- Membuat ringkasan cepat.
- Menghubungkan survei dengan dashboard.

Chatbot Database

Input berupa pertanyaan pengguna. Output berupa intent JSON:

- aksi
- tabel target
- filter
- field yang diminta
- batas hasil

Manfaat:

- Membuat chatbot lebih terkontrol.
- Menghindari query liar.
- Memisahkan bahasa manusia dari operasi database.

1.11 Risiko Jika LLM Langsung Dihubungkan ke Database

LLM yang langsung menulis ke database tanpa validasi adalah desain yang berbahaya. Risikonya:

- **Data rusak masuk ke tabel utama.**
- **Field wajib hilang.**
- **Duplikasi meningkat.**
- **Nilai palsu ikut tersimpan.**
- **Aplikasi downstream gagal.**
- **Laporan menjadi salah.**
- **Audit sulit dilakukan.**
- **Kesalahan kecil menyebar ke sistem lain.**

Dalam dokumen referensi MySQL & JSON, validasi JSON Schema bahkan dapat dipakai sebelum data diterima oleh database melalui mekanisme validasi dan constraint. Ini memperkuat ide bahwa data JSON sebaiknya diuji sebelum dianggap layak disimpan.

Prinsip pentingnya:

- **Jangan simpan output mentah sebagai kebenaran.**
- **Simpan output mentah sebagai bukti proses.**
- **Simpan output valid sebagai data operasional.**
- **Simpan error sebagai bahan evaluasi.**

Dengan cara ini, sistem AI tidak hanya cepat, tetapi juga bertanggung jawab.

1.12 Cara Berpikir Baru: Dari Prompt Engineering ke Data Engineering

Banyak orang memulai AI dengan prompt engineering. Itu wajar. Tetapi untuk aplikasi sistem informasi, prompt engineering saja tidak cukup.

Prompt bertanya:

“Bagaimana cara membuat model menjawab?”

Structured output bertanya:

“Bagaimana cara membuat jawaban model bisa diproses, divalidasi, disimpan, dan diaudit?”

Perbedaannya strategis.

Prompt engineering fokus pada kualitas jawaban.
Data engineering fokus pada kualitas aliran data.

Dalam buku ini, mahasiswa diarahkan naik kelas:

- Dari prompt bebas ke schema.
- Dari jawaban teks ke JSON.
- Dari JSON ke validasi.
- Dari validasi ke database.
- Dari database ke workflow.
- Dari workflow ke audit.
- Dari demo ke sistem.

Ini adalah pola pikir *AI engineering*.

1.13 Prinsip Desain Awal Structured Output

Sebelum masuk ke bab teknis, mahasiswa perlu memegang beberapa prinsip.

1. Mulai dari Output yang Dibutuhkan

Jangan mulai dari model. Mulailah dari data apa yang dibutuhkan sistem.

Pertanyaan awal:

- Data apa yang ingin disimpan?
- Field apa yang wajib?
- Field apa yang opsional?
- Tipe datanya apa?
- Nilai apa yang dianggap tidak valid?
- Output akan dipakai oleh siapa?
- Output akan masuk ke tabel apa?

2. Schema Harus Sederhana

Schema yang terlalu rumit akan sulit dipahami dan sulit diuji. Untuk mahasiswa, schema awal sebaiknya kecil, jelas, dan bertahap.

Prinsipnya:

- Sedikit field, tetapi penting.
- Nama field jelas.
- Tipe data eksplisit.
- Field wajib tidak terlalu banyak.
- Hindari struktur bertingkat yang belum perlu.

3. Validasi Wajib Ada

Validasi bukan aksesori. Validasi adalah pagar.

Tanpa validasi, sistem hanya berharap model benar.

Dengan validasi, sistem memiliki mekanisme menolak data buruk.

4. Error Harus Dicatat

Error bukan kegagalan total. Error adalah sumber belajar.

Yang perlu dicatat:

- Input.
- Output mentah.
- Schema.
- Pesan error.
- Waktu proses.
- Model yang digunakan.
- Status validasi.

5. Human-in-the-Loop untuk Data Sensitif

Untuk data penting, manusia tetap perlu memeriksa hasil tertentu.

Contoh:

- Data akademik.
- Data keuangan.
- Data identitas.
- Data legal.
- Data keputusan administratif.

AI membantu mempercepat. Manusia menjaga tanggung jawab.

1.14 Peta Praktik yang Akan Dibangun di Bab Berikutnya

Walaupun Bab 1 bersifat teori, mahasiswa perlu melihat arah praktik buku.

Alur praktik buku:

- Bab 2: membuat JSON dan JSON Schema sederhana.
- Bab 3: memakai Python dan Pydantic untuk validasi.
- Bab 4: memakai open source LLM untuk menghasilkan JSON lebih stabil.
- Bab 5: memakai teknik *constrained generation*.
- Bab 6: menyimpan output valid ke database JSON.
- Bab 7: mengotomasi workflow.
- Bab 8: membangun studi kasus.
- Bab 9: menguji reliability, error handling, dan audit.
- Bab 10: membuat proyek akhir.

Outline buku juga menegaskan bahwa reliability menjadi tema utama, karena *structured output* bukan sekadar membuat LLM menulis JSON, tetapi membuat output AI cukup stabil untuk sistem nyata.

1.15 Ringkasan Bab

Bab ini menegaskan bahwa LLM tidak cukup hanya menghasilkan jawaban yang terlihat pintar. Untuk sistem informasi, jawaban harus menjadi data yang **terstruktur, valid, konsisten, tersimpan, dan dapat diaudit**.

Inti Bab 1:

- Teks bebas cocok untuk manusia, tetapi lemah untuk mesin.
- JSON membuat output lebih rapi, tetapi belum tentu valid secara makna.
- JSON Schema memberi kontrak struktur dan aturan validasi.
- *Structured output* membuat LLM lebih siap masuk pipeline database.
- Python menjadi penghubung antara LLM, validator, dan database.
- Open source LLM memberi ruang belajar mandiri dan strategis.
- Validasi dan audit adalah fondasi sistem AI yang serius.

Output akhir yang diharapkan: mahasiswa memahami bahwa AI yang berguna untuk sistem informasi bukan hanya AI yang bisa menjawab, tetapi **AI yang mampu menghasilkan data terstruktur yang dapat dipakai ulang, diuji, disimpan, dan dipertanggungjawabkan**.

Bab 2 — Fondasi JSON dan JSON Schema untuk Aplikasi LLM

Golden Rule: Schema yang baik bukan schema yang paling panjang, tetapi schema yang paling jelas, ketat, mudah diuji, dan siap dipakai oleh LLM, Python, workflow, serta database.

Bab ini menempatkan Bab 2 sebagai fondasi JSON, JSON Schema, constraint, nested object, validasi Python, dan desain schema sederhana untuk data mahasiswa, berita, serta transaksi.

2.1 Mengapa JSON Menjadi Bahasa Antara Manusia, LLM, dan Database

JSON adalah format pertukaran data yang **ringan, berbasis teks, dan independen dari bahasa pemrograman**. Standar RFC 8259 mendefinisikan JSON sebagai format pertukaran data yang ringan, berbasis teks, tidak bergantung bahasa, dan digunakan untuk merepresentasikan data terstruktur. ([IETF Datatracker](#))

Bagi mahasiswa, JSON penting karena ia menjadi “bahasa tengah” antara:

- **teks manusia**, seperti biodata, berita, invoice, survei;
- **LLM**, yang mengekstrak informasi dari teks;
- **Python**, yang memeriksa dan mengolah data;
- **database**, yang menyimpan hasil;
- **workflow**, yang menjalankan proses otomatis.

Dalam konteks LLM, JSON membuat output lebih mudah diproses daripada paragraf bebas. Tetapi JSON saja belum cukup. JSON harus diberi aturan. Di situlah JSON Schema menjadi penting.

2.2 Review JSON: Object, Array, String, Number, Boolean, Null

JSON memiliki bentuk dasar yang sederhana. Dokumen referensi *MySQL & JSON* menjelaskan bahwa JSON memakai token struktural seperti `{}`, `[]`, `:`, dan `,`, serta nilai seperti string, number, `false`, `null`, dan `true`; objek dibuka dengan `{}` dan array dibuka dengan `[]`.

2.2.1 Object

object adalah kumpulan pasangan **key-value**.

Contoh:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika"
}
```

Penjelasan:

- `{}` berarti data berbentuk object.
- `"nama"`, `"nim"`, dan `"prodi"` adalah **key**.
- `"Siti Aulia"`, `"2301001"`, dan `"Informatika"` adalah **value**.
- Key dalam JSON harus berupa string.

JSON Schema menjelaskan object sebagai tipe pemetaan yang menghubungkan key ke value; dalam JSON, key selalu string. ([JSON Schema](#))

2.2.2 Array

array adalah daftar nilai.

Contoh:

```
{
  "nama": "Siti Aulia",
  "minat_riset": ["AI", "NLP", "Database"]
}
```

Penjelasan:

- `minat_riset` berisi banyak nilai.
- Nilainya ditulis dalam `[]`.
- Setiap item dipisahkan dengan koma.

Dalam JSON Schema, array dapat divalidasi dengan pendekatan list, yaitu semua item mengikuti schema yang sama, atau tuple, yaitu posisi item memiliki makna tertentu. ([JSON Schema](#))

2.2.3 String

string adalah teks.

Contoh:

```
{
  "email": "siti@example.ac.id"
}
```

String selalu memakai tanda kutip ganda.

2.2.4 Number

number adalah angka.

Contoh:

```
{
  "ipk": 3.75,
  "tahun_masuk": 2023
}
```

Catatan penting: untuk NIM, nomor telepon, kode pos, atau nomor dokumen, sering lebih aman memakai **string**, bukan number. Alasannya, kode identitas bukan angka untuk dihitung. Kode identitas adalah teks yang kebetulan berisi digit.

2.2.5 Boolean

boolean hanya memiliki dua nilai:

```
{
  "aktif": true,
  "lulus": false
}
```

Gunakan boolean untuk status yang benar-benar dua pilihan.

2.2.6 Null

null berarti nilai kosong atau belum tersedia.

```
{
  "email": null
}
```

Gunakan *null* dengan hati-hati. Jangan memakai *null* untuk menutupi data yang seharusnya wajib ada.

2.3 Masalah JSON Biasa: Valid secara Bentuk, Belum Tentu Benar secara Isi

JSON berikut valid secara bentuk:

```
{
  "nama": "Budi",
  "nim": 12345,
  "email": "budi bukan email",
  "tahun_masuk": 1800,
  "minat_riset": "AI, database"
}
```

Tetapi secara kualitas data, banyak masalah:

- `nim` sebaiknya string, bukan number.
- `email` bukan format email yang masuk akal.
- `tahun_masuk` tidak realistis.
- `minat_riset` seharusnya array, bukan satu string panjang.
- Tidak ada aturan apakah field tambahan boleh masuk.

Inilah alasan JSON Schema dibutuhkan. JSON Schema tidak hanya bertanya, “**Apakah ini JSON?**” tetapi bertanya, “**Apakah JSON ini sesuai kontrak data yang kita butuhkan?**”

2.4 Apa Itu JSON Schema

JSON Schema adalah bahasa deklaratif untuk mendeskripsikan struktur, tipe data, dan aturan validasi dokumen JSON. Situs resmi JSON Schema menekankan bahwa JSON Schema membantu membuat aturan validasi yang presisi, menciptakan pemahaman bersama, dan meningkatkan interoperabilitas antar sistem. ([JSON Schema](#))

Dalam buku ini, JSON Schema diposisikan sebagai **kontrak** antara:

- LLM sebagai penghasil output;
- Python sebagai validator;
- database sebagai tempat penyimpanan;
- n8n sebagai workflow;
- manusia sebagai auditor.

Konsep ini sejalan dengan outline buku yang menyatakan bahwa JSON Schema menjadi kontrak antara LLM, program Python, workflow, dan database.

2.5 Struktur Dasar JSON Schema

Schema paling sederhana biasanya berisi:

- **type**
- **properties**
- **required**
- **items**

Contoh schema data mahasiswa:

```
{
  "type": "object",
  "properties": {
    "nama": {
      "type": "string"
    },
    "nim": {
      "type": "string"
    },
    "prodi": {
      "type": "string"
    },
    "email": {
      "type": "string"
    },
    "minat_riset": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  },
  "required": ["nama", "nim", "prodi", "email", "minat_riset"]
}
```

Penjelasan bagian per bagian

- **type**: menentukan bentuk utama data.
- **object**: berarti data harus berbentuk object JSON.
- **properties**: daftar field yang diizinkan dan aturan tiap field.
- **nama, nim, prodi, email, minat_riset**: nama field.
- **type: string**: nilai harus teks.
- **type: array**: nilai harus daftar.
- **items**: aturan untuk setiap item di dalam array.
- **required**: field yang wajib ada.

JSON Schema mendefinisikan **properties** sebagai object yang berisi nama property dan schema untuk memvalidasi property tersebut. ([JSON Schema](#))

2.6 Constraint Penting dalam JSON Schema

Constraint adalah aturan tambahan agar data lebih disiplin.

2.6.1 enum

`enum` membatasi nilai hanya dari daftar tertentu.

Contoh:

```
{
  "type": "object",
  "properties": {
    "prodi": {
      "type": "string",
      "enum": ["Informatika", "Sistem Informasi",
        "Teknik Elektro"]
    }
  },
  "required": ["prodi"]
}
```

Manfaat:

- Menghindari variasi liar.
- Mencegah `informatika`, `IF`, `Teknik Informatika`, dan `Informatics` bercampur tanpa aturan.
- Cocok untuk kategori resmi.

Dalam desain data product berbasis JSON Schema, `enum` dapat dipakai untuk membuat daftar nilai baku seperti milestone atau status.

2.6.2 minimum dan maximum

Digunakan untuk angka.

Contoh:

```
{
  "type": "object",
  "properties": {
    "tahun_masuk": {
      "type": "integer",
      "minimum": 2020,
      "maximum": 2026
    }
  },
  "required": ["tahun_masuk"]
}
```

Manfaat:

- Mencegah tahun tidak realistis.
- Cocok untuk umur, tahun, nilai, jumlah, harga, dan skor.

2.6.3 minLength dan maxLength

Digunakan untuk panjang string.

Contoh:

```
{
  "type": "object",
  "properties": {
    "nim": {
      "type": "string",
      "minLength": 6,
      "maxLength": 20
    }
  },
  "required": ["nim"]
}
```

Manfaat:

- Mencegah nilai terlalu pendek.
- Mencegah teks terlalu panjang.
- Cocok untuk NIM, kode barang, nama singkat, dan kategori.

2.6.4 pattern

`pattern` memakai *regular expression* untuk memeriksa pola teks.

Contoh sederhana untuk email kampus:

```
{
  "type": "object",
  "properties": {
    "email": {
      "type": "string",
      "pattern": "^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]+$"
    }
  },
  "required": ["email"]
}
```

Manfaat:

- Mencegah email tanpa @.
- Mencegah teks bebas masuk sebagai email.
- Cocok untuk kode, NIM, nomor dokumen, dan format tanggal sederhana.

Catatan kritis: **pattern email sederhana tidak menjamin email benar-benar aktif.**

Ia hanya memeriksa bentuk dasar. Validasi lanjutan tetap diperlukan jika sistem membutuhkan kepastian lebih tinggi.

2.7 Nested Object dan Array

Data nyata jarang datar. Sering ada object di dalam object, atau array berisi object.

Contoh data mahasiswa dengan alamat:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "alamat": {
    "kota": "Tangerang Selatan",
    "provinsi": "Banten"
  },
  "minat_riset": ["AI", "NLP", "Database"]
}
```

Schema:

```
{
  "type": "object",
  "properties": {
    "nama": { "type": "string" },
    "nim": { "type": "string" },
    "prodi": { "type": "string" },
    "alamat": {
      "type": "object",
      "properties": {
        "kota": { "type": "string" },
        "provinsi": { "type": "string" }
      },
      "required": ["kota", "provinsi"]
    },
    "minat_riset": {
      "type": "array",
      "items": { "type": "string" }
    }
  },
  "required": ["nama", "nim", "prodi", "alamat", "minat_riset"]
}
```

Nested object berguna, tetapi jangan terlalu cepat membuat schema terlalu dalam. Untuk LLM, struktur terlalu kompleks dapat menaikkan risiko output gagal.

2.8 Schema Sederhana vs Schema Terlalu Kompleks

Schema sederhana:

- field sedikit;
- nama field jelas;
- tipe data eksplisit;
- constraint cukup;
- mudah diuji.

Schema terlalu kompleks:

- field terlalu banyak;
- nested object terlalu dalam;
- terlalu banyak nilai opsional;
- constraint sulit dipahami;
- sulit dijelaskan ke LLM;
- sulit dipakai mahasiswa pemula.

Prinsip praktis: mulai dari schema kecil, lalu naik bertahap.

Contoh schema terlalu rumit untuk tahap awal:

```
{
  "type": "object",
  "properties": {
    "mahasiswa": {
      "type": "object",
      "properties": {
        "identitas": {
          "type": "object",
          "properties": {
            "data_pribadi": {
              "type": "object",
              "properties": {
                "nama_lengkap": { "type": "string" }
              }
            }
          }
        }
      }
    }
  }
}
```

Lebih baik untuk pemula:

```
{
  "type": "object",
  "properties": {
    "nama": { "type": "string" },
    "nim": { "type": "string" },
    "prodi": { "type": "string" }
  }
}
```

```
    },  
    "required": ["nama", "nim", "prodi"]  
}
```

Buku *Unifying Business, Data, and Code* menekankan bahwa data yang baik perlu membawa struktur, makna, dan konteks; tetapi proses pendefinisian konsep juga harus membantu mengurangi ambiguitas, knowledge gaps, dan blind spots.

2.9 Kesalahan Umum Saat Membuat Schema untuk LLM

1. Field Terlalu Banyak

LLM lebih mudah gagal jika diminta mengisi terlalu banyak field sekaligus.

Contoh buruk:

- nama
- nama_depan
- nama_belakang
- nama_panggilan
- nim
- npm
- nomor_induk
- kode_mahasiswa
- program
- jurusan
- fakultas
- peminatan
- topik_skripsi
- topik_riset
- minat_riset
- bidang_ai
- catatan
- ringkasan
- kesimpulan

Schema seperti ini membingungkan.

2. Nama Field Ambigu

Contoh ambigu:

```
{
  "status": "aktif"
}
```

Status apa?

- Status mahasiswa?
- Status pembayaran?
- Status kelulusan?
- Status validasi?
- Status workflow?

Lebih baik:

```
{
  "status_mahasiswa": "aktif"
}
```

3. Tipe Data Tidak Konsisten

Contoh masalah:

```
{
  "tahun_masuk": "2023"
}
```

Di data lain:

```
{
  "tahun_masuk": 2023
}
```

Sistem akan bingung jika satu field kadang string, kadang integer.

4. Semua Field Dibuat Opsional

Jika semua field opsional, schema menjadi terlalu longgar. LLM bisa mengembalikan data kosong tetapi tetap terlihat “valid”.

5. Tidak Menolak Field Tambahan

LLM kadang membuat field baru yang tidak diminta.

Contoh:

```
{
  "nama": "Siti",
  "nim": "2301001",
  "ramalan_karir": "akan sukses sebagai AI engineer"
}
```

Field `ramalan_karir` tidak relevan. Untuk schema yang ketat, tambahkan:

```
"additionalProperties": false
```

Dalam konteks validasi, pertanyaan pentingnya adalah: **apakah semua hal yang tidak kita inginkan sudah dilarang?** Literatur JSON Schema menekankan filosofi constraint: setiap keyword menambah batasan terhadap data yang dianggap valid.

2.10 Prinsip Desain Schema untuk LLM

Schema untuk LLM harus berbeda dari schema database besar. LLM membutuhkan instruksi yang jelas, pendek, dan tidak ambigu.

Gunakan prinsip berikut.

1. Singkat

Gunakan field yang benar-benar dibutuhkan.

Buruk:

```
"nama_mahasiswa_lengkap_sesuai_dokumen_resmi"
```

Lebih baik:

```
"nama"
```

2. Eksplisit

Nama field harus jelas.

Buruk:

```
"info": "AI"
```

Lebih baik:

```
"minat_riset": ["AI"]
```

3. Ketat

Gunakan constraint.

Contoh:

```
"prodi": {  
  "type": "string",  
  "enum": ["Informatika", "Sistem Informasi"]  
}
```

4. Mudah Diuji

Schema harus bisa diuji dengan data valid dan data salah.

Minimal siapkan:

- 1 contoh valid.
- 1 contoh field hilang.
- 1 contoh tipe salah.
- 1 contoh nilai di luar aturan.
- 1 contoh field tambahan.

5. Cocok untuk Database

Field harus mudah disimpan.

Baik:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika"
}
```

Kurang baik:

```
{
  "kalimat": "Nama Siti Aulia dengan NIM 2301001 berasal
dari Informatika"
}
```


Bagian Praktik

2.11 Praktik 1 — Membuat JSON Data Mahasiswa

Input teks:

Siti Aulia adalah mahasiswa Informatika dengan NIM 2301001.
Emailnya siti.aulia@example.ac.id. Ia tertarik pada AI, NLP,
dan database.

Output JSON yang diharapkan:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "minat_riset": ["AI", "NLP", "Database"]
}
```

Schema:

```
{
  "type": "object",
  "properties": {
    "nama": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "nim": {
      "type": "string",
      "minLength": 6,
      "maxLength": 20
    },
    "prodi": {
      "type": "string",
      "enum": ["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    },
    "email": {
      "type": "string",
      "pattern": "^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]+$"
    },
    "minat_riset": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,

```

```
        "maxItems": 5
      },
    },
    "required": ["nama", "nim", "prodi", "email",
"minat_riset"],
    "additionalProperties": false
  }
}
```

Penjelasan section-by-section

- **type: object**: seluruh data harus berbentuk object.
- **properties**: daftar field yang diizinkan.
- **nama**: harus string, minimal 2 karakter, maksimal 100 karakter.
- **nim**: dibuat string karena NIM adalah identitas, bukan angka hitung.
- **prodi**: memakai **enum** agar nilai tidak liar.
- **email**: memakai **pattern** untuk memeriksa bentuk dasar email.
- **minat_riset**: harus array.
- **items**: setiap isi array harus string.
- **minItems**: minimal ada satu minat.
- **maxItems**: maksimal lima minat agar output tidak melebar.
- **required**: semua field wajib ada.
- **additionalProperties: false**: field tambahan ditolak.

2.12 Praktik 2 — Membuat Schema Data Berita

Contoh input:

Pada 12 Mei 2026, ITTS mengadakan seminar AI lokal di Tangerang Selatan. Acara ini menghadirkan Dr. Budi sebagai pembicara utama dan membahas penggunaan open source LLM untuk pendidikan.

Output JSON:

```
{
  "judul": "Seminar AI Lokal di ITTS",
  "tanggal": "2026-05-12",
  "lokasi": "Tangerang Selatan",
  "aktor": ["ITTS", "Dr. Budi"],
  "topik": ["AI lokal", "open source LLM", "pendidikan"],
  "ringkasan": "ITTS mengadakan seminar tentang AI lokal dan open source LLM untuk pendidikan."
}
```

Schema:

```
{
  "type": "object",
  "properties": {
    "judul": {
      "type": "string",
      "minLength": 5,
      "maxLength": 150
    },
    "tanggal": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
    },
    "lokasi": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "aktor": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1
    },
    "topik": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  }
}
```

```
        "minItems": 1,
        "maxItems": 8
    },
    "ringkasan": {
        "type": "string",
        "minLength": 20,
        "maxLength": 300
    }
},
"required": ["judul", "tanggal", "lokasi", "aktor",
"topik", "ringkasan"],
"additionalProperties": false
}
```

Schema berita ini cocok untuk:

- ekstraksi berita kampus;
- arsip kegiatan;
- dashboard isu;
- sistem pencarian berbasis metadata;
- bahan awal NER.

2.13 Praktik 3 — Membuat Schema Data Transaksi Sederhana

Contoh output JSON:

```
{
  "nomor_transaksi": "TRX-2026-0001",
  "tanggal": "2026-05-12",
  "pembeli": "Siti Aulia",
  "items": [
    {
      "nama_produk": "Buku AI Lokal",
      "jumlah": 2,
      "harga_satuan": 75000
    }
  ],
  "total": 150000,
  "status": "paid"
}
```

Schema:

```
{
  "type": "object",
  "properties": {
    "nomor_transaksi": {
      "type": "string",
      "pattern": "^TRX-\\d{4}-\\d{4}$"
    },
    "tanggal": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
    },
    "pembeli": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "items": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "properties": {
          "nama_produk": {
            "type": "string"
          },
          "jumlah": {
            "type": "integer",
            "minimum": 1
          }
        }
      }
    }
  }
}
```

```

        "harga_satuan": {
            "type": "number",
            "minimum": 0
        },
        "required": ["nama_produk", "jumlah",
"harga_satuan"],
        "additionalProperties": false
    },
    "total": {
        "type": "number",
        "minimum": 0
    },
    "status": {
        "type": "string",
        "enum": ["draft", "paid", "cancelled"]
    },
    "required": ["nomor_transaksi", "tanggal", "pembeli",
"items", "total", "status"],
    "additionalProperties": false
}

```

Catatan kritis

Schema ini bisa memeriksa:

- apakah `items` ada;
- apakah setiap item punya `nama_produk`, `jumlah`, `harga_satuan`;
- apakah `jumlah` minimal 1;
- apakah `harga_satuan` tidak negatif;
- apakah `status` hanya salah satu dari `draft`, `paid`, `cancelled`.

Tetapi schema ini belum memeriksa apakah:

$\text{total} = \text{jumlah} \times \text{harga_satuan}$

Validasi seperti itu biasanya dilakukan oleh kode Python atau aturan database. **JSON Schema kuat untuk struktur, tetapi tidak semua logika bisnis cocok dipaksakan ke schema.**

2.14 Praktik 4 — Validasi JSON Menggunakan Python

Python dapat memakai pustaka `jsonschema` untuk memvalidasi data terhadap schema. Dokumentasi `jsonschema` menyatakan bahwa cara paling sederhana untuk memvalidasi instance terhadap schema adalah memakai fungsi `validate(instance, schema)`. ([jsonschema](#))

Instalasi pustaka

```
pip install jsonschema
```

Penjelasan:

- `pip`: alat instalasi paket Python.
- `install`: perintah untuk memasang paket.
- `jsonschema`: pustaka Python untuk validasi JSON Schema.

Script validasi sederhana

```
from jsonschema import validate, ValidationError

schema_mahasiswa = {
    "type": "object",
    "properties": {
        "nama": {
            "type": "string",
            "minLength": 2,
            "maxLength": 100
        },
        "nim": {
            "type": "string",
            "minLength": 6,
            "maxLength": 20
        },
        "prodi": {
            "type": "string",
            "enum": ["Informatika", "Sistem Informasi",
"Teknik Elektro"]
        },
        "email": {
            "type": "string",
            "pattern": "^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$"
        },
        "minat_riset": {
            "type": "array",
            "items": {
                "type": "string"
            },
            "minItems": 1,

```

```

        "maxItems": 5
    },
    "required": ["nama", "nim", "prodi", "email",
"minat_riset"],
    "additionalProperties": False
}

data_mahasiswa = {
    "nama": "Siti Aulia",
    "nim": "2301001",
    "prodi": "Informatika",
    "email": "siti.aulia@example.ac.id",
    "minat_riset": ["AI", "NLP", "Database"]
}

try:
    validate(instance=data_mahasiswa,
schema=schema_mahasiswa)
    print("VALID: data mahasiswa sesuai schema.")
except ValidationError as error:
    print("INVALID:", error.message)

```

Penjelasan section-by-section

- `from jsonschema import validate, ValidationError`
Mengambil fungsi `validate` dan class error `ValidationError`.
- `schema_mahasiswa = {...}`
Menyimpan aturan JSON Schema dalam variabel Python.
- `data_mahasiswa = {...}`
Menyimpan data JSON yang akan diuji.
- `try:`
Menjalankan kode yang berpotensi error.
- `validate(instance=data_mahasiswa, schema=schema_mahasiswa)`
Memeriksa apakah data sesuai schema.
- `print("VALID...")`
Ditampilkan jika data lolos validasi.
- `except ValidationError as error:`
Menangkap error jika data tidak sesuai schema.
- `error.message`
Menampilkan pesan kesalahan yang mudah dibaca.

2.15 Praktik 5 — Menguji Data yang Salah

Ubah data menjadi salah:

```
data_mahasiswa_salah = {
    "nama": "S",
    "nim": "123",
    "prodi": "AI Engineering",
    "email": "bukan-email",
    "minat_riset": []
}

try:
    validate(instance=data_mahasiswa_salah,
             schema=schema_mahasiswa)
    print("VALID: data mahasiswa sesuai schema.")
except ValidationError as error:
    print("INVALID:", error.message)
```

Kemungkinan error:

- nama terlalu pendek;
- NIM terlalu pendek;
- prodi tidak ada dalam enum;
- email tidak cocok pattern;
- minat_riset kosong.

Catatan penting: fungsi `validate()` biasanya berhenti pada satu error pertama. Untuk audit yang lebih lengkap, Bab 9 dapat memakai validator yang mengumpulkan banyak error sekaligus.

2.16 Praktik 6 — Menyimpan Schema dan Data dalam File

Struktur folder sederhana:

```
structured-output-lab/
├── schemas/
│   └── mahasiswa.schema.json
├── data/
│   └── mahasiswa.json
└── validate_mahasiswa.py
```

File schemas/mahasiswa.schema.json

```
{
  "type": "object",
  "properties": {
    "nama": { "type": "string", "minLength": 2 },
    "nim": { "type": "string", "minLength": 6 },
    "prodi": {
      "type": "string",
      "enum": ["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    },
    "email": {
      "type": "string",
      "pattern": "^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]+$"
    },
    "minat_riset": {
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1
    }
  },
  "required": ["nama", "nim", "prodi", "email",
"minat_riset"],
  "additionalProperties": false
}
```

File data/mahasiswa.json

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "minat_riset": ["AI", "NLP", "Database"]
}
```

File validate_mahasiswa.py

```
import json
from pathlib import Path
```

```

from jsonschema import validate, ValidationError

schema_path = Path("schemas/mahasiswa.schema.json")
data_path = Path("data/mahasiswa.json")

schema = json.loads(schema_path.read_text(encoding="utf-8"))
data = json.loads(data_path.read_text(encoding="utf-8"))

try:
    validate(instance=data, schema=schema)
    print("VALID: data sesuai schema.")
except ValidationError as error:
    print("INVALID:", error.message)

```

Penjelasan section-by-section

- `import json`
Memakai modul bawaan Python untuk membaca JSON.
- `from pathlib import Path`
Memakai cara modern untuk membaca path file.
- `from jsonschema import validate, ValidationError`
Memakai pustaka validasi schema.
- `schema_path = Path(...)`
Menentukan lokasi file schema.
- `data_path = Path(...)`
Menentukan lokasi file data.
- `read_text(encoding="utf-8")`
Membaca file sebagai teks dengan encoding UTF-8.
- `json.loads(...)`
Mengubah teks JSON menjadi object Python.
- `validate(...)`
Memvalidasi data terhadap schema.

2.17 Hubungan JSON Schema dengan Database JSON

Database modern dapat menyimpan JSON, mengekstrak nilai dari JSON, dan memvalidasi struktur tertentu. Dalam referensi *MySQL & JSON*, JSON Schema validation dapat digunakan untuk memastikan key wajib ada, tipe data benar, dan nilai sesuai batas sebelum dokumen diterima oleh database.

Namun untuk mahasiswa, prinsip awalnya sederhana:

- Validasi pertama dilakukan di Python.
- Data valid boleh masuk database.
- Data tidak valid masuk log error.
- Schema disimpan dan diberi versi.
- Output mentah LLM jangan langsung dianggap benar.

Database JSON memang fleksibel, tetapi fleksibilitas tanpa schema dapat berubah menjadi kekacauan. Dokumen database JSON juga memperlihatkan bahwa JSON dapat berisi object, array, string, null, boolean, dan number; ketika nilai JSON diekstrak ke SQL, developer perlu memetakan tipe data dengan tepat.

2.18 Catatan Keamanan: JSON Bukan Otomatis Aman

JSON hanyalah format data. Ia bukan jaminan keamanan.

Contoh:

```
{
  "role": "admin",
  "akses": true
}
```

Data ini valid secara JSON, tetapi belum tentu sah secara sistem.

Dokumen tentang JSON Web Token menjelaskan bahwa JWT membungkus JSON dalam format standar dan signed JWT dapat memverifikasi integritas isi, tetapi signed JWT tidak menyembunyikan data dari orang yang memegang token.

Pelajaran untuk mahasiswa:

- Jangan memasukkan rahasia ke JSON sembarangan.
- Jangan percaya field `role` hanya karena ditulis oleh LLM.
- Jangan jadikan output LLM sebagai keputusan akses.
- Validasi struktur berbeda dengan validasi otorisasi.
- Keamanan tetap butuh desain khusus.

2.19 Menghubungkan Bab 2 ke Bab 3

Bab 2 membangun pondasi:

- JSON sebagai format data.
- JSON Schema sebagai kontrak.
- Constraint sebagai pagar kualitas.
- Python sebagai alat validasi awal.

Bab 3 akan melanjutkan ke Pydantic. Outline buku menyebut Pydantic sebagai lapisan validasi antara LLM dan database karena mampu memakai type hints Python untuk validasi, serialisasi, dan pembuatan JSON Schema.

Dengan kata lain:

- Bab 2: menulis schema secara manual.
- Bab 3: membuat model data dengan Python.
- Bab 4: meminta LLM lokal menghasilkan output mengikuti schema.
- Bab 6: menyimpan hasil valid ke database.
- Bab 9: membuat logging, error handling, dan audit.

2.20 Checklist Desain JSON Schema untuk Mahasiswa

Gunakan checklist ini sebelum memakai schema untuk LLM.

Pertanyaan	Jawaban
Apakah field utama sudah jelas?	Ya / Tidak
Apakah nama field singkat dan tidak ambigu?	Ya / Tidak
Apakah setiap field punya tipe data?	Ya / Tidak
Apakah field wajib sudah dimasukkan ke required ?	Ya / Tidak
Apakah kategori memakai enum ?	Ya / Tidak
Apakah angka punya minimum atau maximum jika perlu?	Ya / Tidak
Apakah teks penting punya minLength , maxLength , atau pattern ?	Ya / Tidak
Apakah array punya items ?	Ya / Tidak
Apakah field tambahan perlu ditolak?	Ya / Tidak
Apakah schema sudah diuji dengan data benar dan data salah?	Ya / Tidak

2.21 Latihan Mandiri

Latihan 1 — Biodata Mahasiswa

Buat JSON Schema untuk data:

- nama
- nim
- prodi
- email
- semester
- minat_riset

Aturan:

- nama string, minimal 2 karakter;
- nim string, minimal 6 karakter;
- prodi enum;
- email memakai pattern sederhana;
- semester integer, minimum 1, maximum 14;
- minat_riset array string, minimal 1 item.

Latihan 2 — Berita Kampus

Buat JSON Schema untuk:

- judul
- tanggal
- lokasi
- organisasi
- tokoh
- topik
- ringkasan

Aturan:

- tanggal memakai format `YYYY-MM-DD`;
- topik array;
- ringkasan maksimal 300 karakter;
- field tambahan ditolak.

Latihan 3 — Transaksi Sederhana

Buat JSON Schema untuk:

- nomor_transaksi
- tanggal
- pembeli
- items
- total
- status

Aturan:

- status enum;
- items array minimal 1;
- setiap item berisi nama_produk, jumlah, harga_satuan;
- jumlah minimum 1;
- harga_satuan minimum 0.

2.22 Ringkasan Bab

Bab ini membangun fondasi penting untuk aplikasi LLM berbasis data.

Inti Bab 2:

- **JSON adalah format pertukaran data yang ringan dan mudah diproses.**
- **JSON terdiri dari object, array, string, number, boolean, dan null.**
- **JSON biasa belum cukup untuk sistem informasi karena belum punya kontrak validasi.**
- **JSON Schema mendefinisikan struktur, tipe data, field wajib, dan constraint.**
- **Constraint seperti `enum`, `minimum`, `maximum`, `minLength`, `maxLength`, dan `pattern` membuat output lebih disiplin.**
- **Schema untuk LLM harus singkat, eksplisit, ketat, dan mudah diuji.**
- **Python dapat digunakan untuk memvalidasi JSON sebelum data masuk database.**
- **Data valid boleh diproses; data tidak valid harus ditolak atau dicatat sebagai error.**

Output bab ini: **mahasiswa mampu membuat JSON Schema sederhana yang siap dipakai untuk validasi output LLM**, khususnya untuk data mahasiswa, berita, dan transaksi sederhana.

Bab 3 — Python, Pydantic, dan Validasi Output LLM

Golden Rule: Lebih baik output LLM gagal saat validasi daripada data rusak berhasil masuk database.

Bab ini menempatkan Pydantic sebagai lapisan validasi antara LLM dan database karena mampu memakai *type hints* Python, melakukan validasi, serialisasi, dan menghasilkan JSON Schema dari model Python.

3.1 Mengapa Python Menjadi Alat Utama Eksperimen Structured Output

Python menjadi pilihan kuat untuk eksperimen *structured output* karena sederhana, mudah dibaca, dan ekosistemnya luas. Untuk mahasiswa, Python ideal karena dapat dipakai untuk:

- membaca teks input;
- memanggil LLM lokal;
- menerima output JSON;
- memvalidasi struktur data;
- menangani error;
- menyimpan hasil ke file;
- mengirim data valid ke database;
- membuat log audit.

Dalam pipeline buku ini, Python berada di tengah:

teks → LLM → output JSON → Python validator → database → workflow

Perannya bukan hanya sebagai bahasa pemrograman. Python menjadi **penjaga gerbang data**. LLM boleh menghasilkan jawaban, tetapi Python yang menentukan apakah output itu layak diproses lebih lanjut.

Dokumentasi resmi Python menjelaskan bahwa *type hints* dapat memakai tipe sederhana seperti `str`, `float`, dan tipe yang lebih kompleks melalui modul `typing`. Ini penting karena Pydantic memakai anotasi tipe tersebut untuk memahami struktur data yang diharapkan. ([Python documentation](#))

3.2 Dari JSON Schema ke Pydantic

Pada Bab 2, schema ditulis langsung dalam JSON. Cara itu baik untuk memahami fondasi. Tetapi dalam proyek nyata, mahasiswa sering membutuhkan pendekatan yang lebih nyaman di Python.

Pydantic memberi cara menulis schema sebagai class Python.

Contoh JSON Schema sederhana:

```
{
  "type": "object",
  "properties": {
    "nama": { "type": "string" },
    "nim": { "type": "string" },
    "tahun_masuk": { "type": "integer" }
  },
  "required": ["nama", "nim", "tahun_masuk"]
}
```

Dalam Pydantic, ide yang sama dapat ditulis seperti ini:

```
from pydantic import BaseModel

class Mahasiswa(BaseModel):
    nama: str
    nim: str
    tahun_masuk: int
```

Maknanya sama: data mahasiswa harus memiliki `nama`, `nim`, dan `tahun_masuk` dengan tipe yang benar.

Pydantic menjelaskan model sebagai class yang mewarisi `BaseModel` dan mendefinisikan field sebagai atribut yang diberi anotasi tipe; Pydantic juga menyediakan `model_validate()`, `model_validate_json()`, dan `model_dump()` untuk validasi serta serialisasi. (pydantic.dev)

3.3 Pydantic sebagai Model Data

Model data adalah definisi tentang bentuk data yang dianggap sah.

Dalam konteks LLM:

- LLM menghasilkan output mentah.
- Pydantic membaca output itu.
- Pydantic memeriksa tipe data.
- Pydantic menolak data yang tidak sesuai.
- Pydantic mengubah data valid menjadi object Python.
- Object Python dapat disimpan, dikirim, atau dipakai workflow.

Contoh model:

```
from pydantic import BaseModel

class Mahasiswa(BaseModel):
    nama: str
    nim: str
    prodi: str
    email: str
    tahun_masuk: int
    minat_riset: list[str]
```

Model ini menjadi **kontrak data**.

Artinya:

- `nama` harus string.
- `nim` harus string.
- `prodi` harus string.
- `email` harus string.
- `tahun_masuk` harus integer.
- `minat_riset` harus list berisi string.

Buku *Unifying Business, Data, and Code* menekankan pentingnya menyatukan struktur, makna, dan konteks agar data tidak menjadi sumber ambiguitas, *knowledge gaps*, dan *blind spots*. Pydantic membantu menerjemahkan prinsip itu ke dalam kode yang bisa diuji.

3.4 Mengenal Type Hints: str, int, float, bool, list, Optional

Type hints adalah anotasi tipe data dalam Python. Ia memberi tahu pembaca kode dan alat validasi tentang bentuk data yang diharapkan.

3.4.1 str

`str` dipakai untuk teks.
`nama: str`

Cocok untuk:

- nama;
- NIM;
- email;
- kode transaksi;
- program studi;
- kategori.

Catatan penting: NIM lebih aman sebagai `str`, bukan `int`, karena NIM adalah identitas, bukan angka untuk dihitung.

3.4.2 int

`int` dipakai untuk bilangan bulat.
`tahun_masuk: int`
`semester: int`
`jumlah: int`

Cocok untuk:

- tahun masuk;
- semester;
- jumlah barang;
- skor bulat.

3.4.3 float

`float` dipakai untuk angka pecahan.
`ipk: float`
`harga: float`

Cocok untuk:

- IPK;
- nilai rata-rata;
- harga;
- persentase.

3.4.4 bool

`bool` dipakai untuk nilai benar atau salah.

```
aktif: bool
```

Cocok untuk:

- status aktif;
- sudah diverifikasi;
- data valid;
- perlu tindak lanjut.

3.4.5 list

`list` dipakai untuk daftar.

```
minat_riset: list[str]
```

Artinya: `minat_riset` harus berupa list, dan setiap item di dalamnya harus string.

3.4.6 Optional

`Optional` dipakai ketika field boleh bernilai `None`.

```
from typing import Optional  
catatan: Optional[str] = None
```

Artinya:

- catatan boleh berisi string;
- catatan juga boleh kosong;
- default-nya adalah `None`.

Gunakan `Optional` dengan disiplin. Jangan membuat semua field menjadi opsional. Jika semua data opsional, validasi menjadi terlalu lemah.

3.5 Instalasi Pydantic

Gunakan lingkungan Python yang bersih untuk proyek praktik.

```
python -m venv .venv
source .venv/bin/activate
pip install pydantic
```

Penjelasan baris per baris

- `python`
Menjalankan interpreter Python.
- `-m venv`
Memanggil modul `venv` untuk membuat *virtual environment*.
- `.venv`
Nama folder lingkungan proyek.
- `source .venv/bin/activate`
Mengaktifkan lingkungan Python lokal proyek.
- `pip install pydantic`
Memasang pustaka Pydantic.

Prinsip penting: gunakan lingkungan terpisah agar paket proyek tidak bercampur dengan proyek lain.

Bagian Praktik

3.6 Praktik 1 — Membuat Model Mahasiswa

Buat file:

```
bab03_pydantic/  
└─ model_mahasiswa.py
```

Isi file:

```
from pydantic import BaseModel, EmailStr, Field  
from typing import Optional  
  
class Mahasiswa(BaseModel):  
    nama: str = Field(min_length=2, max_length=100)  
    nim: str = Field(min_length=6, max_length=20)  
    prodi: str  
    email: EmailStr  
    tahun_masuk: int = Field(ge=2020, le=2026)  
    minat_riset: list[str] = Field(min_length=1,  
max_length=5)  
    catatan: Optional[str] = None  
  
data = {  
    "nama": "Siti Aulia",  
    "nim": "2301001",  
    "prodi": "Informatika",  
    "email": "siti.aulia@example.ac.id",  
    "tahun_masuk": 2023,  
    "minat_riset": ["AI", "NLP", "Database"]  
}  
  
mahasiswa = Mahasiswa.model_validate(data)  
  
print(mahasiswa)  
print(mahasiswa.nama)  
print(mahasiswa.model_dump())
```

Penjelasan section-by-section

- `from pydantic import BaseModel, EmailStr, Field`
Mengambil class utama Pydantic. `BaseModel` dipakai untuk membuat model. `EmailStr` dipakai untuk validasi email. `Field` dipakai untuk memberi constraint.
- `from typing import Optional`
Mengambil `Optional` agar field boleh bernilai `None`.
- `class Mahasiswa(BaseModel):`
Membuat model data bernama `Mahasiswa`.
- `nama: str = Field(min_length=2, max_length=100)`
`nama` harus string, minimal 2 karakter, maksimal 100 karakter.
- `nim: str = Field(min_length=6, max_length=20)`
`nim` harus string dan panjangnya dibatasi.
- `email: EmailStr`
Email harus mengikuti bentuk email yang valid menurut validator Pydantic.
- `tahun_masuk: int = Field(ge=2020, le=2026)`
`ge` berarti *greater than or equal*, minimal 2020.
`le` berarti *less than or equal*, maksimal 2026.
- `minat_riset: list[str]`
Berisi daftar string.
- `catatan: Optional[str] = None`
Field catatan boleh kosong.
- `Mahasiswa.model_validate(data)`
Memvalidasi dictionary Python terhadap model.
- `mahasiswa.model_dump()`
Mengubah object Pydantic kembali menjadi dictionary.

Pydantic menggunakan tipe bawaan dan tipe pustaka standar seperti `int`, `str`, dan `date` untuk menentukan validasi serta serialisasi; constraint dapat ditambahkan untuk membuat aturan lebih ketat. (pydantic.dev)

3.7 Menangani Error Instalasi EmailStr

EmailStr membutuhkan paket tambahan. Jika muncul error, pasang:

```
pip install "pydantic[email]"
```

Penjelasan:

- `pydantic[email]`
Memasang Pydantic bersama dukungan validasi email.

Alternatif sederhana untuk pembelajaran awal: gunakan `str` untuk email, lalu validasi format email dibahas bertahap.

3.8 Praktik 2 — Menguji Data Mahasiswa yang Salah

Tambahkan kode berikut:

```
from pydantic import ValidationError

data_salah = {
    "nama": "S",
    "nim": "123",
    "prodi": "Informatika",
    "email": "bukan-email",
    "tahun_masuk": 2010,
    "minat_riset": []
}

try:
    mahasiswa = Mahasiswa.model_validate(data_salah)
    print("VALID:", mahasiswa)
except ValidationError as error:
    print("INVALID")
    print(error)
```

Apa yang diuji?

- `nama` terlalu pendek.
- `nim` terlalu pendek.
- `email` tidak valid.
- `tahun_masuk` di bawah batas.
- `minat_riset` kosong.

Pydantic akan menghasilkan `ValidationError`. Ini perilaku yang diinginkan.

Fail fast: sistem lebih baik berhenti sekarang daripada menyimpan data salah.

3.9 Konsep Fail Fast

Fail fast berarti sistem sengaja dibuat cepat gagal ketika menemukan data yang rusak.

Dalam sistem AI, ini sangat penting karena LLM bisa:

- salah menebak field;
- menulis tipe data keliru;
- mengarang nilai;
- menghilangkan field wajib;
- menambah field yang tidak diminta;
- menulis JSON tidak lengkap.

Tanpa *fail fast*, data rusak bisa masuk database. Setelah itu, kerusakan menyebar ke laporan, dashboard, analisis, dan keputusan.

Prinsipnya:

- **Gagal di validator lebih murah daripada gagal di database.**
- **Gagal di database lebih murah daripada gagal di laporan pimpinan.**
- **Gagal di laporan lebih mahal karena sudah mempengaruhi keputusan.**

3.10 Praktik 3 — Membuat Model Berita

Buat file:

model_berita.py

Isi:

```
from pydantic import BaseModel, Field, ValidationError

class Berita(BaseModel):
    judul: str = Field(min_length=5, max_length=150)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    lokasi: str = Field(min_length=2, max_length=100)
    aktor: list[str] = Field(min_length=1)
    topik: list[str] = Field(min_length=1, max_length=8)
    ringkasan: str = Field(min_length=20, max_length=300)

data_berita = {
    "judul": "Seminar AI Lokal di ITTS",
    "tanggal": "2026-05-12",
    "lokasi": "Tangerang Selatan",
    "aktor": ["ITTS", "Dr. Budi"],
    "topik": ["AI lokal", "open source LLM", "pendidikan"],
    "ringkasan": "ITTS mengadakan seminar tentang AI lokal dan open source LLM untuk pendidikan."
}

try:
    berita = Berita.model_validate(data_berita)
    print("VALID")
    print(berita.model_dump())
except ValidationError as error:
    print("INVALID")
    print(error)
```

Penjelasan penting

- **tanggal** memakai **pattern** agar mengikuti format **YYYY-MM-DD**.
- **aktor** harus list dan minimal satu item.
- **topik** dibatasi agar LLM tidak membuat daftar terlalu panjang.
- **ringkasan** dibatasi panjangnya.

Model seperti ini cocok untuk pipeline:

artikel berita → LLM → JSON berita → Pydantic → database berita

3.11 Praktik 4 — Membuat Model Produk

Buat file:

model_produk.py

Isi:

```
from pydantic import BaseModel, Field, ValidationError
from typing import Literal, Optional

class Produk(BaseModel):
    kode_produk: str = Field(pattern=r"^\PRD-\d{4}$")
    nama_produk: str = Field(min_length=2, max_length=120)
    kategori: Literal["buku", "alat", "software", "lainnya"]
    harga: float = Field(ge=0)
    stok: int = Field(ge=0)
    aktif: bool = True
    catatan: Optional[str] = None

data_produk = {
    "kode_produk": "PRD-0001",
    "nama_produk": "Buku AI Lokal",
    "kategori": "buku",
    "harga": 75000,
    "stok": 20,
    "aktif": True
}

try:
    produk = Produk.model_validate(data_produk)
    print("VALID")
    print(produk.model_dump())
except ValidationError as error:
    print("INVALID")
    print(error)
```

Penjelasan section-by-section

- `Literal["buku", "alat", "software", "lainnya"]`
Membatasi nilai kategori. Ini mirip `enum` pada JSON Schema.
- `harga: float = Field(ge=0)`
Harga tidak boleh negatif.
- `stok: int = Field(ge=0)`
Stok tidak boleh negatif.
- `aktif: bool = True`
Jika field `aktif` tidak ada, nilainya otomatis `True`.
- `catatan: Optional[str] = None`
Catatan boleh kosong.

3.12 Mengubah Pydantic Model Menjadi JSON Schema

Salah satu kekuatan Pydantic adalah dapat menghasilkan JSON Schema dari model.

Contoh:

```
import json
from model_mahasiswa import Mahasiswa
schema = Mahasiswa.model_json_schema()
print(json.dumps(schema, indent=2, ensure_ascii=False))
```

Penjelasan

- `Mahasiswa.model_json_schema()`
Menghasilkan JSON Schema dari model Pydantic.
- `json.dumps(...)`
Mengubah object Python menjadi teks JSON.
- `indent=2`
Membuat output rapi dengan spasi dua.
- `ensure_ascii=False`
Membuat karakter non-ASCII tetap tampil normal.

Dokumentasi Pydantic menjelaskan bahwa `BaseModel.model_json_schema()` mengembalikan dictionary yang dapat dipakai sebagai JSON Schema dari sebuah model. (pydantic.dev)

3.13 Mengapa Ini Penting untuk LLM

Schema dari Pydantic dapat dipakai untuk:

- dokumentasi struktur output;
- validasi hasil LLM;
- parameter *structured output* pada LLM lokal;
- kontrak antara Python dan workflow;
- rujukan desain tabel database;
- audit versi schema.

Alurnya:

```
Pydantic model
  ↓
JSON Schema
  ↓
LLM structured output
  ↓
JSON output
  ↓
Pydantic validation
  ↓
database
```

Dengan cara ini, mahasiswa tidak perlu menulis aturan dua kali. Model Python menjadi pusat kebenaran struktur data.

3.14 Parsing Output LLM Menjadi Object Python

Dalam praktik, LLM sering menghasilkan output dalam bentuk string JSON.

Contoh output LLM:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP", "Database"]
}
```

Di Python, output itu bisa disimpan sebagai string:

```
output_llm = """
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP", "Database"]
}
"""
```

Lalu divalidasi:

```
from pydantic import ValidationError
from model_mahasiswa import Mahasiswa

try:
    mahasiswa = Mahasiswa.model_validate_json(output_llm)
    print("VALID")
    print(mahasiswa.model_dump())
except ValidationError as error:
    print("INVALID")
    print(error)
```

Pydantic menyediakan `model_validate_json()` untuk memvalidasi data JSON langsung terhadap model; dokumentasi performa Pydantic juga menyarankan memakai `model_validate_json()` dibanding memanggil `json.loads()` lalu `model_validate()` untuk banyak kasus validasi JSON. (pydantic.dev)

3.15 Praktik 5 — Ekstraksi Data dari Teks Pendaftaran Mahasiswa

Contoh input manusia:

Nama saya Siti Aulia. Saya mendaftar ke Program Studi Informatika.
NIM sementara saya 2301001. Email saya siti.aulia@example.ac.id.
Saya masuk tahun 2023 dan tertarik pada AI, NLP, serta Database.

Target output LLM:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP", "Database"]
}
```

Validator Pydantic:

```
from pydantic import BaseModel, EmailStr, Field, ValidationError

class Mahasiswa(BaseModel):
    nama: str = Field(min_length=2, max_length=100)
    nim: str = Field(min_length=6, max_length=20)
    prodi: str
    email: EmailStr
    tahun_masuk: int = Field(ge=2020, le=2026)
    minat_riset: list[str] = Field(min_length=1, max_length=5)

output_llm = """
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP", "Database"]
}
"""

try:
    mahasiswa = Mahasiswa.model_validate_json(output_llm)
    print("VALID: output LLM lolos validasi.")
    print(mahasiswa.model_dump())
except ValidationError as error:
    print("INVALID: output LLM ditolak.")
    print(error)
```

Makna praktik ini

Mahasiswa belajar bahwa output LLM tidak langsung dipakai. Output harus melewati pagar:

LLM → JSON string → Pydantic → object valid

Jika gagal, data tidak boleh masuk database.

3.16 Praktik 6 — Menangani Output LLM yang Salah

Contoh output LLM bermasalah:

```
output_llm_salah = """
{
  "nama": "S",
  "nim": "123",
  "prodi": "Informatika",
  "email": "siti-aulia",
  "tahun_masuk": 2015,
  "minat_riset": []
}
"""
```

Validasi:

```
try:
    mahasiswa =
Mahasiswa.model_validate_json(output_llm_salah)
    print("VALID:", mahasiswa.model_dump())
except ValidationError as error:
    print("INVALID: output LLM ditolak.")
    print(error.errors())
```

Penjelasan

- `error.errors()`
Menghasilkan daftar error dalam bentuk terstruktur.

Contoh informasi error yang biasanya berguna:

- lokasi field yang salah;
- jenis error;
- pesan error;
- input yang gagal.

Ini sangat penting untuk audit. Dalam Bab 9, error seperti ini dapat disimpan ke file log atau tabel audit.

3.17 Membuat Fungsi Validator agar Kode Lebih Rapi

Buat file:

validator_mahasiswa.py

Isi:

```
from pydantic import BaseModel, EmailStr, Field,
ValidationError

class Mahasiswa(BaseModel):
    nama: str = Field(min_length=2, max_length=100)
    nim: str = Field(min_length=6, max_length=20)
    prodi: str
    email: EmailStr
    tahun_masuk: int = Field(ge=2020, le=2026)
    minat_riset: list[str] = Field(min_length=1,
max_length=5)

def validasi_output_llm(output_json: str) -> tuple[bool,
dict]:
    try:
        mahasiswa =
Mahasiswa.model_validate_json(output_json)
        return True, mahasiswa.model_dump()
    except ValidationError as error:
        return False, {"errors": error.errors()}

output_llm = """
{
    "nama": "Siti Aulia",
    "nim": "2301001",
    "prodi": "Informatika",
    "email": "siti.aulia@example.ac.id",
    "tahun_masuk": 2023,
    "minat_riset": ["AI", "NLP", "Database"]
}
"""
status, hasil = validasi_output_llm(output_llm)

print(status)
print(hasil)
```


Penjelasan section-by-section

- `def validasi_output_llm(output_json: str) -> tuple[bool, dict]:`
Membuat fungsi yang menerima string JSON dan mengembalikan dua nilai: status validasi dan hasil.
- `try:`
Mencoba validasi.
- `Mahasiswa.model_validate_json(output_json)`
Memvalidasi output LLM langsung dari string JSON.
- `return True, mahasiswa.model_dump()`
Jika valid, kembalikan status `True` dan data bersih.
- `except ValidationError as error:`
Jika tidak valid, tangkap error.
- `return False, {"errors": error.errors()}`
Jika gagal, kembalikan status `False` dan daftar error.

Fungsi ini adalah pola dasar untuk pipeline yang lebih besar.

3.18 Menolak Field Tambahan dari LLM

LLM kadang menambahkan field yang tidak diminta.

Contoh:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP"],
  "ramalan_karir": "akan menjadi pakar AI terkenal"
}
```

Field `ramalan_karir` tidak dibutuhkan. Untuk menolak field tambahan, gunakan konfigurasi:

```
from pydantic import BaseModel, ConfigDict, EmailStr, Field

class Mahasiswa(BaseModel):
    model_config = ConfigDict(extra="forbid")

    nama: str = Field(min_length=2, max_length=100)
    nim: str = Field(min_length=6, max_length=20)
    prodi: str
    email: EmailStr
    tahun_masuk: int = Field(ge=2020, le=2026)
    minat_riset: list[str] = Field(min_length=1,
max_length=5)
```

Penjelasan

- `ConfigDict(extra="forbid")`
Menolak field tambahan yang tidak didefinisikan dalam model.

Prinsip: LLM tidak boleh bebas menambah struktur data tanpa izin.

3.19 Membatasi Program Studi dengan Literal

Agar prodi tidak liar, gunakan Literal.

```
from typing import Literal
from pydantic import BaseModel, EmailStr, Field

class Mahasiswa(BaseModel):
    nama: str = Field(min_length=2, max_length=100)
    nim: str = Field(min_length=6, max_length=20)
    prodi: Literal["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    email: EmailStr
    tahun_masuk: int = Field(ge=2020, le=2026)
    minat_riset: list[str] = Field(min_length=1,
max_length=5)
```

Mengapa ini penting?

Tanpa pembatasan, LLM bisa menulis:

- informatika
- Informatics
- IF
- Teknik Informatika
- Ilmu Komputer
- Computer Science

Dengan `Literal`, hanya nilai yang diizinkan yang diterima.

3.20 Validasi Minat Riset

minat_riset sebaiknya berupa list, bukan satu string panjang.

Benar:

```
"minat_riset": ["AI", "NLP", "Database"]
```

Kurang baik:

```
"minat_riset": "AI, NLP, Database"
```

Model:

```
minat_riset: list[str] = Field(min_length=1, max_length=5)
```

Maknanya:

- harus list;
- item harus string;
- minimal satu item;
- maksimal lima item.

Ini membuat data lebih mudah dicari dan dianalisis.

3.21 Menghubungkan Pydantic dengan JSON Schema untuk LLM Lokal

Banyak alat LLM lokal mendukung schema atau format terstruktur. Dalam outline buku, Bab 4 diarahkan pada *structured output* dengan Ollama, vLLM, dan llama.cpp, termasuk penggunaan JSON Schema atau grammar untuk membatasi output.

Pydantic dapat menjadi sumber schema:

```
import json
from model_mahasiswa import Mahasiswa

schema = Mahasiswa.model_json_schema()

with open("mahasiswa.schema.json", "w", encoding="utf-8") as file:
    json.dump(schema, file, indent=2, ensure_ascii=False)
```

Penjelasan

- `model_json_schema()`
Menghasilkan schema.
- `open(..., "w")`
Membuka file untuk ditulis.
- `encoding="utf-8"`
Memastikan teks aman untuk bahasa Indonesia.
- `json.dump(...)`
Menyimpan schema ke file JSON.

Schema ini nanti dapat dipakai sebagai rujukan untuk meminta LLM menghasilkan output yang lebih disiplin.

3.22 Menyimpan Output Valid dan Output Gagal

Dalam sistem yang baik, hasil valid dan gagal dipisahkan.

Struktur folder:

```
bab03_pydantic/  
├── output_valid/  
├── output_error/  
├── schemas/  
└── validator_mahasiswa.py
```

Contoh fungsi simpan:

```
import json  
from pathlib import Path  
from datetime import datetime  
  
def simpan_json(folder: str, nama_file: str, data: dict) ->  
None:  
    Path(folder).mkdir(parents=True, exist_ok=True)  
  
    path = Path(folder) / nama_file  
  
    with path.open("w", encoding="utf-8") as file:  
        json.dump(data, file, indent=2, ensure_ascii=False)  
  
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")  
  
    status, hasil = validasi_output_llm(output_llm)  
  
    if status:  
        simpan_json("output_valid",  
f"mahasiswa_{timestamp}.json", hasil)  
    else:  
        simpan_json("output_error", f"error_{timestamp}.json",  
hasil)
```

Penjelasan

- `Path(folder).mkdir(...)`
Membuat folder jika belum ada.
- `parents=True`
Membuat folder induk jika diperlukan.
- `exist_ok=True`
Tidak error jika folder sudah ada.
- `datetime.now()`
Mengambil waktu saat ini.
- `strftime(...)`
Mengubah waktu menjadi teks untuk nama file.
- `if status:`
Jika valid, simpan ke folder valid.
- `else:`
Jika gagal, simpan ke folder error.

Ini adalah fondasi audit sederhana.

3.23 Pola Error Handling untuk Pipeline LLM

Pipeline LLM harus siap menghadapi error.

Jenis error umum:

- output bukan JSON;
- JSON tidak lengkap;
- field wajib hilang;
- tipe data salah;
- nilai di luar batas;
- list kosong;
- field tambahan;
- format tanggal salah;
- email tidak valid;
- nilai kategori tidak sesuai daftar.

Pola penanganan:

```
Output LLM masuk
    ↓
Validasi Pydantic
    ↓
Jika valid → lanjut simpan
    ↓
Jika gagal → simpan error
    ↓
Opsional → minta LLM memperbaiki output
    ↓
Validasi ulang
```

Jangan langsung memperbaiki diam-diam tanpa catatan. Jika sistem memperbaiki data, proses koreksi harus dicatat.

3.24 Checklist Validator Output LLM

Gunakan checklist ini sebelum output LLM masuk database.

Pertanyaan	Ya/Tidak
Apakah output berbentuk JSON valid?	
Apakah semua field wajib tersedia?	
Apakah tipe data sesuai model?	
Apakah email valid?	
Apakah tahun masuk masuk akal?	
Apakah program studi sesuai daftar resmi?	
Apakah minat riset berbentuk list?	
Apakah field tambahan ditolak?	
Apakah error disimpan jika validasi gagal?	
Apakah schema/model diberi versi?	

3.25 Latihan Mandiri

Latihan 1 — Model Mahasiswa Lebih Ketat

Tambahkan field:

- semester: int
- aktif: bool
- jalur_masuk: Literal["reguler", "beasiswa", "transfer"]

Aturan:

- semester minimal 1;
- semester maksimal 14;
- aktif default True.

Latihan 2 — Model Berita Kampus

Buat model `BeritaKampus` dengan field:

- judul;
- tanggal;
- lokasi;
- organisasi;
- tokoh;
- topik;
- ringkasan.

Gunakan:

- `Field(min_length=...)`
- `Field(max_length=...)`
- `list[str]`
- `pattern` untuk tanggal.

Latihan 3 — Model Produk

Buat model `Produk` dengan field:

- kode_produk;
- nama_produk;
- kategori;
- harga;
- stok;
- aktif.

Gunakan:

- `Literal` untuk kategori;
- `Field(ge=0)` untuk harga dan stok;
- `pattern` untuk kode produk.

Latihan 4 — Output LLM Salah

Buat tiga contoh output LLM yang salah:

- field hilang;
- tipe data salah;
- nilai enum tidak sesuai.

Validasi dengan Pydantic dan simpan error-nya.

3.26 Ringkasan Bab

Bab ini memperkuat posisi Python dan Pydantic sebagai fondasi validasi output LLM.

Inti Bab 3:

- Python menjadi alat utama eksperimen karena sederhana, fleksibel, dan kuat untuk pipeline data.
- Pydantic membuat model data memakai *type hints* Python.
- Model Pydantic dapat memvalidasi output LLM sebelum masuk database.
- `str`, `int`, `float`, `bool`, `list`, dan `Optional` adalah fondasi tipe data yang harus dikuasai mahasiswa.
- `Field`, `Literal`, dan `ConfigDict(extra="forbid")` membuat validasi lebih ketat.
- `model_validate_json()` dapat membaca string JSON dari LLM dan mengubahnya menjadi object Python yang valid.
- `ValidationError` harus ditangani, dicatat, dan dipakai sebagai bahan audit.
- Konsep *fail fast* mencegah data rusak menyebar ke database, dashboard, dan workflow.

Output bab ini: mahasiswa mampu membangun validator Python agar output LLM tidak langsung dipercaya sebelum lolos pemeriksaan struktur.

Bab 4 — Structured Output dengan Open Source LLM: Ollama, vLLM, dan llama.cpp

Golden Rule: Prompt yang baik membantu LLM menjawab; schema yang baik memaksa LLM menghasilkan data yang bisa divalidasi.

Bab ini adalah jantung *open source* buku. Bab 4 diarahkan untuk membahas Ollama, vLLM, dan llama.cpp sebagai alat *structured output* berbasis JSON Schema, Pydantic model, dan grammar GBNF.

4.1 Mengapa Structured Output di Open Source LLM Penting

LLM lokal makin penting karena mahasiswa dapat belajar, bereksperimen, dan membangun prototipe tanpa selalu bergantung pada layanan tertutup. Tetapi ada masalah besar: **LLM tetap menghasilkan teks probabilistik**. Jawabannya bisa berubah, field bisa hilang, dan format JSON bisa rusak.

Karena itu, *structured output* dibutuhkan untuk membuat LLM lebih disiplin.

Tujuannya:

- menghasilkan JSON yang stabil;
- mengikuti JSON Schema;
- mengurangi field liar;
- memudahkan validasi Python;
- menyiapkan data untuk database;
- membuat pipeline lebih mudah diaudit.

Ollama secara resmi mendukung *structured outputs* dengan memberikan JSON Schema pada parameter `format`; dokumentasinya juga menyarankan agar schema ikut ditulis di prompt agar respons model lebih terarah. (docs.ollama.com)

4.2 Posisi Ollama, vLLM, dan llama.cpp

Tiga alat ini punya peran berbeda.

Alat	Peran Utama	Cocok Untuk
Ollama	Eksperimen LLM lokal yang mudah	Laptop, kelas, demo mahasiswa
vLLM	<i>Serving</i> model skala lebih besar	Server lokal, lab, API internal
llama.cpp	Menjalankan model GGUF dan grammar	Perangkat terbatas, eksperimen <i>constrained decoding</i>

vLLM mendukung *structured outputs* seperti `choice`, `regex`, `json`, `grammar`, dan `structural_tag`; untuk JSON, output diarahkan mengikuti JSON Schema. (docs.vllm.ai)

llama.cpp memakai GBNF, yaitu format grammar untuk membatasi output model, misalnya memaksa model menghasilkan JSON valid atau pola tertentu. ([GitHub](https://github.com/jmorganca/llama))

4.3 Prompt Biasa vs JSON Schema vs Grammar

4.3.1 Prompt Biasa

Prompt biasa hanya memberi instruksi.

Contoh:

Ekstrak berita kampus berikut menjadi JSON dengan field judul, tanggal, lokasi, aktor, topik, ringkasan.

Masalahnya:

- model bisa menambah penjelasan;
- model bisa menulis JSON tidak valid;
- field bisa berubah nama;
- tanggal bisa tidak konsisten;
- array bisa berubah menjadi string.

Prompt biasa cocok untuk eksplorasi awal, tetapi tidak cukup kuat untuk pipeline database.

4.3.2 JSON Schema

JSON Schema memberi kontrak struktur.

Contoh field wajib:

- `judul`
- `tanggal`
- `lokasi`
- `aktor`
- `topik`
- `ringkasan`

Dengan schema, model diarahkan menghasilkan JSON sesuai struktur. Setelah itu, Python memvalidasi ulang.

Prinsip penting: schema membantu, tetapi validasi tetap wajib.

4.3.3 Grammar

Grammar lebih ketat. Ia membatasi bentuk token yang boleh dihasilkan model.

Grammar cocok jika:

- output harus sangat kaku;
- format kecil dan jelas;
- ingin membatasi model pada pola tertentu;
- JSON Schema belum cukup;
- output sering rusak walaupun sudah diberi schema.

llama.cpp mendukung GBNF untuk membatasi output model melalui grammar.

([GitHub](#))

4.4 Temperatur, Determinisme, dan Risiko Output Rusak

LLM bekerja dengan probabilitas. Parameter seperti `temperature` memengaruhi variasi jawaban.

Secara praktis:

- `temperature` rendah → output lebih stabil;
- `temperature` tinggi → output lebih kreatif, tetapi lebih berisiko rusak;
- untuk ekstraksi data, gunakan nilai rendah;
- untuk penulisan kreatif, nilai lebih tinggi bisa dipakai.

Untuk *structured output*, gunakan prinsip:

- **temperature rendah;**
- **schema eksplisit;**
- **validasi Pydantic;**
- **catat error;**
- **ulang proses jika gagal.**

Jangan mengejar kreativitas untuk data terstruktur. **Database membutuhkan konsistensi, bukan gaya bahasa.**

4.5 Kapan Cukup Prompt, Kapan Perlu Schema, Kapan Perlu Grammar

Gunakan panduan berikut.

Kondisi	Teknik
Eksperimen cepat	Prompt biasa
Output perlu JSON tetapi belum kritikal	Prompt + contoh JSON
Output masuk database	JSON Schema + validasi Python
Output sering rusak	JSON Schema + retry + validasi
Format sangat ketat	Grammar / <i>constrained decoding</i>
Banyak user dan beban tinggi	vLLM
Eksperimen laptop	Ollama
Model GGUF dan perangkat terbatas	llama.cpp

Aturan tajam: semakin dekat output LLM ke database, semakin ketat pagar validasinya.

Bagian Praktik

4.6 Praktik 1 — Menyiapkan Model Berita Kampus dengan Pydantic

Kita akan membuat ekstraksi entitas dari berita kampus.

Target output wajib:

- judul
- tanggal
- lokasi
- aktor
- topik
- ringkasan

Buat folder:

```
bab04_structured_output/  
├─ extract_berita_ollama.py  
├─ output_valid/  
└─ output_error/
```

Buat file `extract_berita_ollama.py`.

```
import json  
from pathlib import Path  
from datetime import datetime  
from typing import Literal  
  
import requests  
from pydantic import BaseModel, ConfigDict, Field,  
ValidationError
```

Penjelasan section-by-section

- `import json`
Dipakai untuk membaca dan menulis data JSON.
- `from pathlib import Path`
Dipakai untuk membuat folder dan path file dengan rapi.
- `from datetime import datetime`
Dipakai untuk membuat nama file berbasis waktu.
- `from typing import Literal`
Dipakai untuk membatasi nilai tertentu.
- `import requests`
Dipakai untuk mengirim HTTP request ke Ollama lokal.
- `BaseModel`
Fondasi model Pydantic.
- `ConfigDict`
Dipakai untuk menolak field tambahan.
- `Field`
Dipakai untuk membuat constraint.
- `ValidationError`
Dipakai untuk menangkap error validasi.

Pydantic dapat menghasilkan JSON Schema otomatis dari model Python melalui `model_json_schema()`. ([Pydantic](#))

4.7 Membuat Model BeritaKampus

Tambahkan kode berikut.

```
class BeritaKampus(BaseModel):
    model_config = ConfigDict(extra="forbid")

    judul: str = Field(min_length=5, max_length=150)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    lokasi: str = Field(min_length=2, max_length=100)
    aktor: list[str] = Field(min_length=1, max_length=10)
    topik: list[str] = Field(min_length=1, max_length=8)
    ringkasan: str = Field(min_length=20, max_length=300)
```

Penjelasan

- `class BeritaKampus(BaseModel):`
Membuat model data untuk berita kampus.
- `model_config = ConfigDict(extra="forbid")`
Menolak field tambahan dari LLM.
- `judul: str`
Judul harus berupa teks.
- `Field(min_length=5, max_length=150)`
Judul minimal 5 karakter dan maksimal 150 karakter.
- `tanggal: str`
Tanggal disimpan sebagai teks.
- `pattern=r"^\d{4}-\d{2}-\d{2}$"`
Tanggal harus mengikuti format YYYY-MM-DD.
- `aktor: list[str]`
Aktor harus berupa daftar teks.
- `topik: list[str]`
Topik juga berupa daftar teks.
- `ringkasan: str`
Ringkasan harus berupa teks pendek.

Catatan kritis: tanggal masih divalidasi sebagai pola teks. Pemeriksaan apakah tanggal benar-benar ada, misalnya 2026-02-30, dapat ditambahkan di bab lanjut.

4.8 Membuat JSON Schema dari Pydantic

Tambahkan kode:

```
schema_berita = BeritaKampus.model_json_schema()
```

Penjelasan:

- `BeritaKampus` adalah model Pydantic.
- `model_json_schema()` mengubah model menjadi JSON Schema.
- Schema ini nanti dikirim ke Ollama melalui parameter `format`.

Ini penting karena satu model Python dapat dipakai untuk dua fungsi:

- **mengarahkan output LLM;**
- **memvalidasi output LLM.**

4.9 Praktik 2 — Menjalankan Structured Output dengan Ollama

Pastikan Ollama sudah berjalan di komputer lokal. Pilih model kecil yang tersedia di mesin. Untuk eksperimen mahasiswa, gunakan model kecil lebih dulu agar proses ringan.

Contoh variabel:

```
OLLAMA_URL = "http://localhost:11434/api/chat"  
MODEL_NAME = "llama3.2:3b"
```

Penjelasan

- `OLLAMA_URL`
Alamat API lokal Ollama.
- `localhost`
Berarti layanan berjalan di komputer sendiri.
- `11434`
Port standar Ollama.
- `/api/chat`
Endpoint chat Ollama.
- `MODEL_NAME`
Nama model yang akan dipakai.

Dokumentasi API Ollama menyebut *structured outputs* didukung dengan memberikan JSON Schema pada parameter `format`; model kemudian diminta menghasilkan respons yang cocok dengan schema tersebut. ([GitHub](#))

4.10 Membuat Prompt Ekstraksi Berita Kampus

Tambahkan input berita:

```
teks_berita = """
Pada 12 Mei 2026, Institut Teknologi Tangerang Selatan
mengadakan seminar AI lokal di Tangerang Selatan. Acara ini
menghadirkan Dr. Budi Santoso sebagai pembicara utama.
Seminar membahas penggunaan open source LLM, JSON Schema, dan
Python untuk membangun aplikasi kampus berbasis data.
"""
```

Buat prompt:

```
prompt = f"""
Ekstrak informasi penting dari berita kampus berikut.
```

Aturan:

- Jawab hanya dalam JSON.
- Ikuti schema yang diberikan.
- Jangan menambah field di luar schema.
- Gunakan tanggal format YYYY-MM-DD.
- Jika aktor lebih dari satu, tulis sebagai array.
- Jika topik lebih dari satu, tulis sebagai array.

```
Teks berita:
{teks_berita}
"""
```

Penjelasan

- `f"..."`
Membuat teks panjang dengan kemampuan menyisipkan variabel.
- `{teks_berita}`
Menyisipkan isi berita ke dalam prompt.
- "Jawab hanya dalam JSON"
Mencegah model menambahkan narasi.
- "Ikuti schema"
Menguatkan batas struktur.
- "Jangan menambah field"
Mengurangi risiko field liar.

Ollama menyarankan schema juga dimasukkan ke prompt untuk membantu model tetap mengikuti respons yang diinginkan. (docs.ollama.com)

4.11 Mengirim Schema dari Python ke Ollama

Tambahkan fungsi berikut.

```
def panggil_ollama(prompt: str, schema: dict) -> str:
    payload = {
        "model": MODEL_NAME,
        "messages": [
            {
                "role": "user",
                "content": prompt
            }
        ],
        "format": schema,
        "stream": False,
        "options": {
            "temperature": 0
        }
    }

    response = requests.post(OLLAMA_URL, json=payload,
                             timeout=120)
    response.raise_for_status()

    hasil = response.json()
    return hasil["message"]["content"]
```

Penjelasan section-by-section

- `def panggil_ollama(prompt: str, schema: dict) -> str:`
Membuat fungsi bernama `panggil_ollama`.
Fungsi menerima `prompt` berbentuk string dan `schema` berbentuk dictionary.
Fungsi mengembalikan string.
- `payload = {...}`
Paket data yang dikirim ke Ollama.
- `"model": MODEL_NAME`
Menentukan model yang digunakan.
- `"messages": [...]`
Isi percakapan yang dikirim ke model.
- `"role": "user"`
Menandakan pesan berasal dari pengguna.
- `"content": prompt`
Isi instruksi yang dibaca model.
- `"format": schema`
Bagian penting untuk *structured output*. Schema dikirim ke Ollama.
- `"stream": False`
Respons dikembalikan sekaligus, bukan bertahap.
- `"temperature": 0`
Membuat output lebih stabil.

- `requests.post(...)`
Mengirim request ke API Ollama.
- `timeout=120`
Batas waktu menunggu respons, dalam detik.
- `response.raise_for_status()`
Jika terjadi error HTTP, Python akan menghentikan proses.
- `response.json()`
Mengubah respons menjadi dictionary Python.
- `return hasil["message"]["content"]`
Mengambil isi jawaban model.

4.12 Memvalidasi Output Ollama dengan Pydantic

Tambahkan fungsi validasi:

```
def validasi_output(output_llm: str) -> tuple[bool, dict]:
    try:
        data = BeritaKampus.model_validate_json(output_llm)
        return True, data.model_dump()
    except ValidationError as error:
        return False, {"errors": error.errors(),
            "raw_output": output_llm}
```

Penjelasan

- `output_llm: str`
Output dari Ollama masih berupa string.
- `BeritaKampus.model_validate_json(output_llm)`
Memvalidasi string JSON langsung dengan Pydantic.
- `data.model_dump()`
Mengubah object valid menjadi dictionary.
- `except ValidationError as error`
Menangkap error jika output tidak sesuai model.
- `raw_output`
Menyimpan output mentah untuk audit.

Prinsip: structured output tetap harus divalidasi. Jangan percaya model hanya karena sudah diberi schema.

4.13 Menyimpan Hasil Valid dan Error

Tambahkan fungsi simpan:

```
def simpan_json(folder: str, prefix: str, data: dict) ->
None:
    Path(folder).mkdir(parents=True, exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    path = Path(folder) / f"{prefix}_{timestamp}.json"

    with path.open("w", encoding="utf-8") as file:
        json.dump(data, file, indent=2, ensure_ascii=False)
```

Penjelasan

- `folder`
Nama folder tujuan.
- `prefix`
Awalan nama file.
- `Path(folder).mkdir(...)`
Membuat folder jika belum ada.
- `parents=True`
Membuat folder induk jika dibutuhkan.
- `exist_ok=True`
Tidak error jika folder sudah ada.
- `datetime.now().strftime(...)`
Membuat timestamp.
- `json.dump(...)`
Menulis dictionary menjadi file JSON.
- `ensure_ascii=False`
Agar teks Indonesia tetap terbaca normal.

4.14 Menyatukan Semua Pipeline

Tambahkan kode utama:

```
if __name__ == "__main__":
    output_llm = panggil_ollama(prompt, schema_berita)

    print("=== OUTPUT MENTAH LLM ===")
    print(output_llm)

    status, hasil = validasi_output(output_llm)

    if status:
        print("\nVALID: output LLM sesuai schema.")
        print(json.dumps(hasil, indent=2,
ensure_ascii=False))
        simpan_json("output_valid", "berita", hasil)
    else:
        print("\nINVALID: output LLM ditolak.")
        print(json.dumps(hasil, indent=2,
ensure_ascii=False))
        simpan_json("output_error", "error", hasil)
```

Penjelasan

- `if __name__ == "__main__":`
Memastikan kode utama berjalan hanya saat file dijalankan langsung.
- `output_llm = panggil_ollama(...)`
Mengirim prompt dan schema ke Ollama.
- `print(output_llm)`
Menampilkan output mentah untuk inspeksi.
- `status, hasil = validasi_output(output_llm)`
Memvalidasi output dengan Pydantic.
- `if status:`
Jika valid, simpan ke `output_valid`.
- `else:`
Jika gagal, simpan ke `output_error`.

4.15 Kode Lengkap Praktek Ollama

```
import json
from pathlib import Path
from datetime import datetime

import requests
from pydantic import BaseModel, ConfigDict, Field,
ValidationError

OLLAMA_URL = "http://localhost:11434/api/chat"
MODEL_NAME = "llama3.2:3b"

class BeritaKampus(BaseModel):
    model_config = ConfigDict(extra="forbid")

    judul: str = Field(min_length=5, max_length=150)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    lokasi: str = Field(min_length=2, max_length=100)
    aktor: list[str] = Field(min_length=1, max_length=10)
    topik: list[str] = Field(min_length=1, max_length=8)
    ringkasan: str = Field(min_length=20, max_length=300)

schema_berita = BeritaKampus.model_json_schema()

teks_berita = """
Pada 12 Mei 2026, Institut Teknologi Tangerang Selatan
mengadakan seminar AI lokal
di Tangerang Selatan. Acara ini menghadirkan Dr. Budi Santoso
sebagai pembicara utama.
Seminar membahas penggunaan open source LLM, JSON Schema, dan
Python untuk
membangun aplikasi kampus berbasis data.
"""

prompt = f"""
Ekstrak informasi penting dari berita kampus berikut.

Aturan:
- Jawab hanya dalam JSON.
- Ikuti schema yang diberikan.
- Jangan menambah field di luar schema.
- Gunakan tanggal format YYYY-MM-DD.
- Jika aktor lebih dari satu, tulis sebagai array.
- Jika topik lebih dari satu, tulis sebagai array.
```

```

Teks berita:
{teks_berita}
"""

def panggil_ollama(prompt: str, schema: dict) -> str:
    payload = {
        "model": MODEL_NAME,
        "messages": [
            {
                "role": "user",
                "content": prompt
            }
        ],
        "format": schema,
        "stream": False,
        "options": {
            "temperature": 0
        }
    }

    response = requests.post(OLLAMA_URL, json=payload,
timeout=120)
    response.raise_for_status()

    hasil = response.json()
    return hasil["message"]["content"]

def validasi_output(output_llm: str) -> tuple[bool, dict]:
    try:
        data = BeritaKampus.model_validate_json(output_llm)
        return True, data.model_dump()
    except ValidationError as error:
        return False, {"errors": error.errors()},
"raw_output": output_llm}

def simpan_json(folder: str, prefix: str, data: dict) ->
None:
    Path(folder).mkdir(parents=True, exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    path = Path(folder) / f"{prefix}_{timestamp}.json"

    with path.open("w", encoding="utf-8") as file:
        json.dump(data, file, indent=2, ensure_ascii=False)

```

```

if __name__ == "__main__":
    output_llm = panggil_ollama(prompt, schema_berita)

    print("=== OUTPUT MENTAH LLM ===")
    print(output_llm)

    status, hasil = validasi_output(output_llm)

    if status:
        print("\nVALID: output LLM sesuai schema.")
        print(json.dumps(hasil, indent=2,
ensure_ascii=False))
        simpan_json("output_valid", "berita", hasil)
    else:
        print("\nINVALID: output LLM ditolak.")
        print(json.dumps(hasil, indent=2,
ensure_ascii=False))
        simpan_json("output_error", "error", hasil)

```

4.16 Hasil JSON yang Diharapkan

Output ideal:

```
{
  "judul": "Seminar AI Lokal di Institut Teknologi Tangerang Selatan",
  "tanggal": "2026-05-12",
  "lokasi": "Tangerang Selatan",
  "aktor": [
    "Institut Teknologi Tangerang Selatan",
    "Dr. Budi Santoso"
  ],
  "topik": [
    "AI lokal",
    "open source LLM",
    "JSON Schema",
    "Python",
    "aplikasi kampus berbasis data"
  ],
  "ringkasan": "Institut Teknologi Tangerang Selatan mengadakan seminar AI lokal tentang open source LLM, JSON Schema, dan Python untuk aplikasi kampus berbasis data."
}
```

Yang perlu diperiksa:

- Apakah semua field ada?
- Apakah **tanggal** formatnya benar?
- Apakah **aktor** berbentuk array?
- Apakah **topik** berbentuk array?
- Apakah tidak ada field tambahan?
- Apakah ringkasan tidak terlalu panjang?
- Apakah hasil lolos Pydantic?

4.17 Praktik 3 — Membandingkan Prompt Bebas dan Schema-Based Output

Prompt bebas

Ringkas berita ini dan ambil informasinya dalam JSON.

Risiko:

- model menulis pembuka;
- JSON bercampur paragraf;
- field berubah;
- tanggal tidak standar;
- array berubah menjadi kalimat.

Prompt berbasis schema

Ikuti schema. Jawab hanya JSON. Jangan menambah field.

Ditambah parameter:

```
"format": schema_berita
```

Keunggulan:

- struktur lebih stabil;
- field lebih konsisten;
- parsing lebih mudah;
- validasi lebih jelas;
- siap masuk pipeline database.

Ollama menyatakan *structured outputs* dapat memberi reliabilitas dan konsistensi lebih baik dibanding JSON mode biasa. (ollama.com)

4.18 Praktik 4 — Eksperimen dengan Model Kecil

Mahasiswa sebaiknya mulai dari model kecil.

Tujuan eksperimen:

- memahami alur schema;
- melihat kualitas ekstraksi;
- menguji validasi;
- melihat error;
- membandingkan output beberapa kali.

Tabel uji sederhana:

Percobaan	Teknik	Temperature	Valid?	Catatan
1	Prompt bebas	0.7		
2	Prompt bebas	0		
3	Schema	0.7		
4	Schema	0		
5	Schema + validasi	0		

Kesimpulan yang dicari:

- Apakah schema mengurangi error?
- Apakah temperature rendah lebih stabil?
- Apakah model kecil cukup untuk tugas ekstraksi?
- Field apa yang paling sering salah?
- Apakah perlu retry?

4.19 Praktik 5 — Menambahkan Retry Sederhana

Jika output gagal, sistem dapat mencoba ulang.

```
def ekstraksi_dengan_retry(prompt: str, schema: dict,
    max_retry: int = 3) -> tuple[bool, dict]:
    for percobaan in range(1, max_retry + 1):
        output_llm = panggil_ollama(prompt, schema)
        status, hasil = validasi_output(output_llm)

        if status:
            hasil["percobaan"] = percobaan
            return True, hasil

    return False, {
        "errors": "Output tetap gagal setelah retry.",
        "max_retry": max_retry
    }
```

Penjelasan

- `max_retry: int = 3`
Sistem mencoba maksimal tiga kali.
- `for percobaan in range(...)`
Mengulang proses ekstraksi.
- `if status:`
Jika valid, langsung berhenti.
- `hasil["percobaan"] = percobaan`
Mencatat percobaan ke berapa yang berhasil.
- `return False`
Jika semua gagal, sistem mengembalikan error.

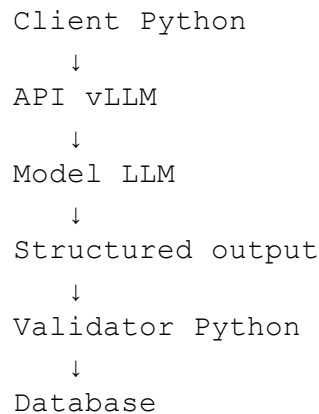
Catatan kritis: retry membantu, tetapi bukan solusi utama. Schema tetap harus diperbaiki jika error sering terjadi.

4.20 vLLM untuk Serving Model Skala Lebih Besar

Ollama cocok untuk laptop dan praktikum. vLLM cocok saat model perlu disajikan sebagai layanan untuk banyak request.

vLLM mendukung *structured outputs* melalui beberapa pilihan, termasuk JSON Schema, regex, pilihan tetap, dan grammar. (docs.vllm.ai)

Pola pemakaiannya:



Kapan memakai vLLM?

- banyak pengguna;
- beban request lebih tinggi;
- model lebih besar;
- butuh API server;
- praktikum sudah naik ke level lab;
- proyek akhir perlu demo multi-user.

Kapan belum perlu vLLM?

- baru belajar JSON Schema;
- baru membuat validator;
- hanya satu laptop;
- model kecil sudah cukup;
- belum perlu layanan API.

4.21 llama.cpp dan Grammar GBNF

llama.cpp penting karena ringan dan banyak dipakai untuk model GGUF. Salah satu kekuatannya adalah dukungan grammar GBNF.

GBNF dapat dipakai untuk membatasi bentuk output. Contoh sederhana grammar JSON sangat ketat bisa menjadi rumit, tetapi ide dasarnya adalah:

```
root ::= object
object ::= "{" pair "}"
pair ::= "\"status\"" ":" value
value ::= "\"valid\"" | "\"invalid\""
```

Artinya model hanya boleh menghasilkan object dengan field `status` yang bernilai `valid` atau `invalid`.

Kapan grammar berguna?

- output kecil dan sangat formal;
- ingin membatasi nilai secara keras;
- JSON Schema belum cukup stabil;
- eksperimen membutuhkan format ekstrem;
- ingin memahami *constrained decoding* lebih dalam.

Kapan grammar tidak ideal?

- schema sangat besar;
- field sangat banyak;
- struktur berubah-ubah;
- mahasiswa masih pemula;
- debugging grammar menghabiskan waktu.

Prinsip praktis: gunakan JSON Schema dulu. Pakai grammar jika butuh kontrol lebih ketat.

4.22 Risiko dan Batasan Structured Output

Structured output bukan sihir. Ia meningkatkan stabilitas, tetapi tidak menjamin kebenaran semantik.

Contoh:

```
{
  "judul": "Seminar AI Lokal",
  "tanggal": "2026-05-12",
  "lokasi": "Tangerang Selatan",
  "aktor": ["Dr. Budi Santoso"],
  "topik": ["AI"],
  "ringkasan": "Seminar membahas AI lokal."
}
```

JSON ini bisa valid. Tetapi tetap perlu ditanya:

- Apakah tanggal benar dari sumber?
- Apakah nama aktor benar?
- Apakah ringkasan tidak mengarang?
- Apakah topik terlalu sempit?
- Apakah lokasi sudah lengkap?

Schema memeriksa struktur. **Schema tidak otomatis memeriksa fakta.**

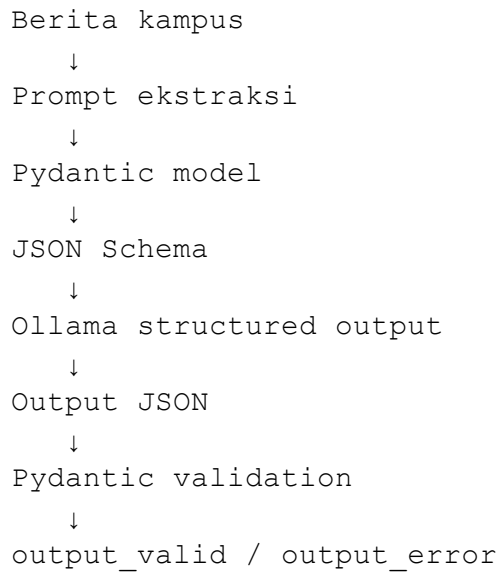
Karena itu pipeline yang baik tetap memerlukan:

- validasi struktur;
- validasi format;
- validasi nilai;
- penyimpanan input asli;
- audit output mentah;
- pemeriksaan manusia untuk data sensitif.

Dokumen utama buku menekankan bahwa LLM tidak boleh dipercaya sebelum output divalidasi, distrukturkan, disimpan, dan dapat diaudit.

4.23 Desain Pipeline Bab 4

Pipeline yang dibangun dalam bab ini:



Pipeline ini sederhana, tetapi strategis. Inilah dasar sistem AI yang siap masuk database.

Di bab berikutnya, teknik ini dapat diperkuat dengan *constrained generation* memakai library khusus. Dalam outline buku, Bab 5 diarahkan pada Instructor dan Outlines untuk membuat output lebih terkendali.

4.24 Checklist Praktik Structured Output

Gunakan checklist ini sebelum output masuk database.

Pertanyaan	Ya/Tidak
Apakah model lokal sudah berjalan?	
Apakah schema dibuat dari Pydantic?	
Apakah schema dikirim ke LLM?	
Apakah prompt melarang field tambahan?	
Apakah temperatur rendah?	
Apakah output mentah disimpan saat error?	
Apakah output divalidasi ulang?	
Apakah field tambahan ditolak?	
Apakah hasil valid disimpan terpisah?	
Apakah hasil error dicatat?	

4.25 Latihan Mandiri

Latihan 1 — Ekstraksi Berita Kampus

Gunakan teks berita berbeda. Wajib hasilkan JSON:

- `judul`
- `tanggal`
- `lokasi`
- `aktor`
- `topik`
- `ringkasan`

Uji apakah output lolos Pydantic.

Latihan 2 — Bandingkan Temperature

Jalankan ekstraksi dengan:

- `temperature: 0`
- `temperature: 0.3`
- `temperature: 0.7`

Catat:

- jumlah output valid;
- field yang hilang;
- format yang rusak;
- kualitas ringkasan.

Latihan 3 — Tambahkan Field `jenis_acara`

Tambahkan field:

```
jenis_acara: Literal["seminar", "workshop", "kuliah umum", "lomba", "lainnya"]
```

Uji apakah model mampu memilih kategori dengan benar.

Latihan 4 — Buat Retry dan Log

Tambahkan retry maksimal tiga kali. Simpan semua error ke folder `output_error`.

Latihan 5 — Bandingkan Prompt Bebas vs Schema

Buat tabel hasil:

Teks	Prompt Bebas Valid?	Schema Valid?	Catatan
------	---------------------	---------------	---------

4.26 Ringkasan Bab

Bab ini menunjukkan cara membuat LLM lokal menghasilkan JSON yang lebih stabil dan siap masuk pipeline data.

Inti Bab 4:

- **Open source LLM penting untuk kemandirian belajar dan eksperimen mahasiswa.**
- **Ollama cocok untuk eksperimen lokal berbasis laptop.**
- **vLLM cocok untuk serving model skala lebih besar.**
- **llama.cpp cocok untuk model GGUF dan grammar GBNF.**
- **Prompt biasa tidak cukup untuk data yang akan masuk database.**
- **JSON Schema membuat output lebih terkendali.**
- **Grammar memberi kontrol lebih ketat, tetapi lebih kompleks.**
- **Temperature rendah lebih cocok untuk ekstraksi data.**
- **Output LLM tetap wajib divalidasi ulang dengan Pydantic.**
- **Data valid dan error harus disimpan terpisah agar bisa diaudit.**

Output bab ini: **mahasiswa memahami cara membuat LLM lokal menghasilkan JSON yang lebih stabil, divalidasi, disimpan, dan siap masuk pipeline database.**

Bab 5 — Constrained Generation dengan Instructor dan Outlines

Golden Rule: Structured output bukan hanya meminta LLM menulis JSON, tetapi membatasi ruang jawaban agar output lebih stabil, valid, dan siap diproses sistem.

Bab ini menempatkan Bab 5 sebagai praktik Instructor dan Outlines, setelah mahasiswa memahami JSON Schema, Pydantic, dan *structured output* dengan LLM lokal. Outline buku juga menegaskan bahwa *reliability* adalah tema utama: output AI harus cukup stabil untuk sistem nyata, bukan sekadar terlihat rapi saat demo.

5.1 Apa Itu Constrained Generation

Constrained generation adalah teknik untuk membatasi output LLM agar mengikuti bentuk tertentu sejak proses generasi berlangsung.

Jika prompt biasa hanya berkata:

“Tolong jawab dalam JSON.”

Maka *constrained generation* berusaha membuat model menghasilkan output yang memang dibatasi oleh struktur:

- field tertentu;
- tipe data tertentu;
- daftar nilai tertentu;
- pola tertentu;
- objek JSON tertentu;
- model Pydantic tertentu.

Perbedaannya tajam:

- **Prompt biasa:** berharap LLM patuh.
- **Validasi setelah generasi:** membiarkan LLM menjawab, lalu output diperiksa.
- **Constrained generation:** membatasi ruang jawaban sejak model menghasilkan token.

Outlines menjelaskan bahwa banyak pendekatan mencoba memperbaiki output setelah generasi dengan parsing, regex, atau kode rapuh, sedangkan Outlines dirancang untuk menjamin struktur output selama generasi berlangsung. ([GitHub](#))

5.2 Mengapa Teknik Ini Penting untuk Mahasiswa

Mahasiswa sering memulai dari prompt seperti:

Ekstrak nama orang, organisasi, lokasi, tanggal, dan topik dari teks berikut.
Jawab dalam JSON.

Prompt ini bisa berhasil. Tetapi tidak selalu stabil.

Masalah umum:

- JSON tidak valid.
- Field hilang.
- Nama field berubah.
- Array berubah menjadi string.
- Nilai tanggal tidak konsisten.
- Model menambahkan penjelasan.
- Model membuat field yang tidak diminta.
- Output sulit dimasukkan ke database.

Untuk proyek akademik sederhana, error seperti ini mungkin hanya mengganggu. Untuk pipeline data, error seperti ini bisa merusak sistem.

Karena itu Bab 5 memperkenalkan dua lapisan penting:

- **Instructor** untuk output *type-safe* berbasis Pydantic.
- **Outlines** untuk *JSON structured generation* yang lebih ketat.

Instructor mendeskripsikan dirinya sebagai pustaka Python untuk ekstraksi data terstruktur dari LLM, memakai Pydantic sebagai schema, serta menangani validasi, retry, dan error handling. (python.useinstructor.com)

5.3 Validation-after-generation vs Constrained-during-generation

Ada dua pola besar.

5.3.1 Validation-after-generation

Alur:

```
Prompt
↓
LLM menghasilkan output
↓
Python parsing
↓
Pydantic validation
↓
Jika gagal → retry / error log
```

Kelebihan:

- mudah dipahami;
- cocok untuk pemula;
- fleksibel;
- cepat dibuat;
- cocok untuk eksperimen awal.

Kelemahan:

- model tetap bisa menghasilkan JSON rusak;
- butuh parsing tambahan;
- retry bisa menambah waktu;
- error masih sering muncul;
- kualitas sangat bergantung pada prompt.

5.3.2 Constrained-during-generation

Alur:

```
Schema / Pydantic model
↓
LLM dibatasi saat menghasilkan output
↓
Output lebih sesuai struktur
↓
Validasi akhir tetap dilakukan
↓
Data siap masuk pipeline
```

Kelebihan:

- format lebih stabil;
- error parsing lebih sedikit;
- lebih cocok untuk sistem otomatis;
- output lebih mudah diproses kode;
- mengurangi kebutuhan pembersihan manual.

Kelemahan:

- setup lebih kompleks;
- schema besar bisa memperlambat;
- tidak semua model sama kuat;
- debugging bisa lebih sulit;
- validasi fakta tetap belum otomatis.

Prinsip: *Constrained generation* mengurangi error format, tetapi tidak menggantikan validasi, logging, dan audit.

5.4 Instructor: Output Type-Safe Berbasis Pydantic

Instructor bekerja dekat dengan Pydantic. Mahasiswa mendefinisikan model data, lalu meminta LLM mengisi model itu dari teks.

Contoh model:

```
from pydantic import BaseModel, Field

class EntitasNER(BaseModel):
    person: list[str] = Field(default_factory=list)
    organization: list[str] = Field(default_factory=list)
    location: list[str] = Field(default_factory=list)
    date: list[str] = Field(default_factory=list)
    topic: list[str] = Field(default_factory=list)
```

Model ini menjadi kontrak:

- `person` harus list string.
- `organization` harus list string.
- `location` harus list string.
- `date` harus list string.
- `topic` harus list string.

Instructor memakai Pydantic model untuk mendefinisikan output schema dan memberikan respons yang sudah tervalidasi sebagai object Python. Instructor juga mendukung model lokal seperti Ollama dan llama-cpp-python. (python.useinstructor.com)

5.5 Outlines: JSON Structured Generation

Outlines fokus pada struktur output saat generasi berlangsung. Dokumentasi Outlines menyatakan bahwa Outlines dapat membuat model open source mengembalikan objek JSON mengikuti struktur yang ditentukan pengguna, sehingga output dapat diproses oleh kode *downstream*. ([Dottxt AI](#))

Artinya, Outlines cocok untuk kasus seperti:

- ekstraksi entitas;
- klasifikasi intent;
- ekstraksi metadata dokumen;
- data form otomatis;
- ringkasan terstruktur;
- output yang harus masuk database.

Outlines juga mendukung penggunaan Pydantic model sebagai struktur output. Dokumentasi Outlines memperlihatkan pola: definisikan class Pydantic, panggil model dengan output type tersebut, lalu sistem mengembalikan object valid. ([Dottxt AI](#))

5.6 Instructor vs Outlines

Aspek	Instructor	Outlines
Fokus utama	Validasi, retry, output typed	Struktur output saat generasi
Basis kuat	Pydantic	Pydantic / JSON / grammar
Cocok untuk	Pipeline ekstraksi cepat	Output lebih terkendali
Pola berpikir	Model → LLM → validasi/retry	Model/schema → generasi dibatasi
Kelebihan	Praktis, dekat dengan Pydantic	Format lebih stabil sejak generasi
Risiko	Tetap bergantung pada respons model	Schema kompleks bisa lebih berat
Untuk mahasiswa	Mudah mulai	Bagus untuk naik level

Keduanya bukan musuh. Keduanya bisa dipelajari sebagai dua teknik untuk tujuan yang sama: **membuat output LLM lebih reliable.**

5.7 Keterbatasan Constrained Generation

Teknik ini kuat, tetapi punya batas.

1. Schema kompleks dapat memperlambat proses

Semakin banyak constraint, semakin kecil ruang jawaban. Ini bagus untuk kontrol, tetapi bisa membuat generasi lebih lambat.

2. Model kecil bisa tetap salah secara makna

Output bisa valid secara struktur, tetapi salah isi.

Contoh:

```
{
  "person": ["ITTS"],
  "organization": ["Dr. Budi"]
}
```

Strukturnya valid, tetapi klasifikasi entitasnya salah.

3. Validasi fakta tetap diperlukan

Schema tidak otomatis tahu apakah tanggal, nama, lokasi, atau organisasi benar.

4. Logging tetap wajib

Jika output gagal atau aneh, sistem harus menyimpan:

- input teks;
- prompt;
- schema;
- model;
- parameter;
- output mentah;
- error validasi;
- waktu proses.

Dokumen utama buku menegaskan bahwa LLM tidak boleh dipercaya sebelum output divalidasi, distrukturkan, disimpan, dan dapat diaudit.

Bagian Praktik

5.8 Target Praktik: NER Sederhana

Kita akan membuat *Named Entity Recognition* sederhana.

Input:

Pada 12 Mei 2026, Institut Teknologi Tangerang Selatan mengadakan seminar AI lokal di Tangerang Selatan. Acara ini menghadirkan Dr. Budi Santoso dan membahas JSON Schema, Python, serta open source LLM.

Output wajib:

```
{
  "person": ["Dr. Budi Santoso"],
  "organization": ["Institut Teknologi Tangerang Selatan"],
  "location": ["Tangerang Selatan"],
  "date": ["2026-05-12"],
  "topic": ["AI lokal", "JSON Schema", "Python", "open source LLM"]
}
```

Target praktik:

- membuat model Pydantic;
- membuat extractor dengan Instructor;
- membuat generator JSON dengan Outlines;
- membandingkan prompt biasa, Pydantic validation, dan *constrained generation*;
- mengukur error sederhana.

5.9 Persiapan Folder Praktik

Buat struktur folder:

```
bab05_constrained_generation/  
├── data/  
│   └── berita_kampus.txt  
├── output/  
│   ├── prompt_biasa/  
│   ├── pydantic_validation/  
│   ├── instructor/  
│   └── outlines/  
├── ner_model.py  
├── ner_prompt_biasa.py  
├── ner_instructor.py  
├── ner_outlines.py  
└── evaluasi_error.py
```

Penjelasan:

- `data/` menyimpan input teks.
- `output/` menyimpan hasil eksperimen.
- `ner_model.py` menyimpan model Pydantic.
- `ner_prompt_biasa.py` menjalankan prompt biasa.
- `ner_instructor.py` menjalankan Instructor.
- `ner_outlines.py` menjalankan Outlines.
- `evaluasi_error.py` membandingkan hasil.

5.10 Membuat Model Pydantic untuk NER

File: ner_model.py

```
from pydantic import BaseModel, ConfigDict, Field

class NEROutput(BaseModel):
    model_config = ConfigDict(extra="forbid")

    person: list[str] = Field(default_factory=list, max_length=10)
    organization: list[str] = Field(default_factory=list,
max_length=10)
    location: list[str] = Field(default_factory=list, max_length=10)
    date: list[str] = Field(default_factory=list, max_length=10)
    topic: list[str] = Field(default_factory=list, max_length=10)
```

Penjelasan section-by-section

- `BaseModel`
Fondasi model Pydantic.
- `ConfigDict(extra="forbid")`
Menolak field tambahan dari LLM.
- `person: list[str]`
Daftar nama orang.
- `organization: list[str]`
Daftar organisasi.
- `location: list[str]`
Daftar lokasi.
- `date: list[str]`
Daftar tanggal.
- `topic: list[str]`
Daftar topik.
- `default_factory=list`
Jika tidak ada entitas, field menjadi list kosong, bukan `None`.
- `max_length=10`
Membatasi jumlah item agar output tidak melebar.

Pydantic dapat menghasilkan JSON Schema dari `BaseModel` melalui `model_json_schema()`, sehingga satu model dapat dipakai untuk validasi dan acuan struktur output. ([Pydantic](#))

5.11 Praktik 1 — Prompt Biasa

File: `ner_prompt_biasa.py`

```
import json
from pathlib import Path
import requests

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL_NAME = "llama3.2:3b"

teks = Path("data/berita_kampus.txt").read_text(encoding="utf-8")

prompt = f"""
Ekstrak entitas dari teks berikut.

Kembalikan JSON dengan field:
person, organization, location, date, topic.

Teks:
{teks}
"""

payload = {
    "model": MODEL_NAME,
    "prompt": prompt,
    "stream": False,
    "options": {
        "temperature": 0
    }
}

response = requests.post(OLLAMA_URL, json=payload, timeout=120)
response.raise_for_status()
hasil = response.json()["response"]

Path("output/prompt_biasa").mkdir(parents=True, exist_ok=True)
Path("output/prompt_biasa/hasil.json").write_text(hasil,
encoding="utf-8")
print(hasil)
```

Penjelasan

- Prompt biasa hanya meminta JSON.
- Tidak ada schema formal.
- Output bisa valid, bisa tidak.
- Hasil disimpan apa adanya.

Ollama API mendukung parameter `format`, termasuk string `"json"` atau JSON Schema object; pada praktik prompt biasa ini, parameter schema belum digunakan agar mahasiswa melihat perbedaannya. ([Ollama Docs](#))

5.12 Praktik 2 — Pydantic Validation Setelah Output

Tambahkan validasi terhadap hasil prompt biasa.

File: `validasi_prompt_biasa.py`

```
import json
from pathlib import Path
from pydantic import ValidationError
from ner_model import NEROutput

raw_output =
Path("output/prompt_biasa/hasil.json").read_text(encoding="utf-8")

try:
    data = NEROutput.model_validate_json(raw_output)
    hasil = data.model_dump()
    status = {
        "valid": True,
        "data": hasil
    }
except ValidationError as error:
    status = {
        "valid": False,
        "errors": error.errors(),
        "raw_output": raw_output
    }

Path("output/pydantic_validation").mkdir(parents=True, exist_ok=True)
Path("output/pydantic_validation/status.json").write_text(
    json.dumps(status, indent=2, ensure_ascii=False),
    encoding="utf-8"
)
print(json.dumps(status, indent=2, ensure_ascii=False))
```

Makna praktik ini

Mahasiswa melihat pola:

```
LLM bebas menjawab
↓
Pydantic memeriksa
↓
Jika valid → data dipakai
↓
Jika gagal → data ditolak
```

Ini lebih aman daripada langsung menyimpan output LLM.

5.13 Praktik 3 — Extractor dengan Instructor

Instalasi:

```
pip install instructor pydantic requests
```

Penjelasan:

- `pip` memasang pustaka Python.
- `instructor` membantu menghasilkan output terstruktur berbasis Pydantic.
- `pydantic` menjadi model data dan validator.
- `requests` dipakai untuk komunikasi HTTP jika diperlukan.

Instructor mendukung integrasi dengan Ollama untuk menghasilkan respons *type-safe* menggunakan JSON Schema mode dari model lokal. (python.useinstructor.com)

Contoh `konsep_ner_instructor.py`:

```
import json
from pathlib import Path
from pydantic import ValidationError

from ner_model import NEROutput
```

Untuk menjaga praktik tetap mudah dipahami, inti desainnya adalah:

```
teks =
Path("data/berita_kampus.txt").read_text(encoding="utf-8")
```

```
prompt = f"""
Ekstrak Named Entity Recognition dari teks berikut.
```

Aturan:

- person berisi nama orang.
- organization berisi nama organisasi.
- location berisi lokasi.
- date berisi tanggal dalam format YYYY-MM-DD jika tersedia.
- topic berisi topik utama.
- Jangan menambah field lain.

Teks:

```
{teks}
"""
```

Pola pemakaian Instructor secara konseptual:

```
# Pseudocode agar mudah dipahami mahasiswa.
# Detail client dapat menyesuaikan provider lokal yang
dipakai.
```

```
client = buat_client_instructor_lokal()
```

```

hasil: NEROutput = client.chat.completions.create(
    model="llama3.2:3b",
    response_model=NEROutput,
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ],
)

```

Simpan hasil:

```

Path("output/instructor").mkdir(parents=True, exist_ok=True)

Path("output/instructor/hasil.json").write_text(
    json.dumps(hasil.model_dump(), indent=2,
ensure_ascii=False),
    encoding="utf-8"
)

```

Penjelasan penting

- `response_model=NEROutput`
Ini bagian utama Instructor. Output diarahkan menjadi model Pydantic.
- `hasil: NEROutput`
Hasil bukan sekadar string JSON, tetapi object Python yang sudah mengikuti model.
- `hasil.model_dump()`
Mengubah object menjadi dictionary untuk disimpan.
- Jika output tidak valid, Instructor dapat membantu validasi dan retry sesuai konfigurasi.

Instructor dibangun di atas Pydantic untuk mendapatkan JSON yang lebih reliabel, *type-safe*, dan lebih mudah dipakai dalam kode. ([GitHub](#))

5.14 Praktik 4 — Generator JSON dengan Outlines

Instalasi:

```
pip install outlines pydantic
```

Outlines dapat membuat LLM menghasilkan struktur output tertentu secara langsung, dan dokumentasi resminya menekankan bahwa output terstruktur dijamin saat generasi, bukan hanya diperbaiki setelahnya. ([Dottxt AI](#))

Pola konsep `ner_outlines.py`:

```
import json
from pathlib import Path
from ner_model import NEROutput
```

Prompt:

```
teks =
Path("data/berita_kampus.txt").read_text(encoding="utf-8")

prompt = f"""
Ekstrak Named Entity Recognition dari teks berikut.

Kategori:
- person
- organization
- location
- date
- topic

Teks:
{teks}
"""
```

Pola Outlines secara konseptual:

```
# Pseudocode agar portable untuk berbagai backend model.
# Backend dapat menyesuaikan model lokal yang tersedia.

model = buat_model_outlines_lokal("llama3.2:3b")

hasil = model(
    prompt,
    NEROutput
)
```

Simpan hasil:

```
Path("output/outlines").mkdir(parents=True, exist_ok=True)

Path("output/outlines/hasil.json").write_text(
    json.dumps(hasil.model_dump(), indent=2,
ensure_ascii=False),
    encoding="utf-8"
)
```

Penjelasan

- `NEROutput` dipakai sebagai struktur output.
- Outlines membatasi generasi agar sesuai struktur.
- Hasil lebih siap dipakai kode downstream.
- Validasi akhir tetap disarankan sebelum masuk database.

5.15 Praktik 5 — Membandingkan Tiga Pendekatan

Tiga pendekatan yang dibandingkan:

1. **Prompt biasa**
2. **Prompt biasa + Pydantic validation**
3. **Constrained generation**

Parameter uji:

- apakah JSON valid;
- apakah field lengkap;
- apakah tipe data benar;
- apakah field tambahan muncul;
- apakah entitas salah kategori;
- berapa lama prosesnya;
- apakah perlu retry.

Tabel evaluasi:

Pendekatan	JSON Valid	Field Lengkap	Tipe Benar	Field Tambahan	Catatan
Prompt biasa					
Prompt + Pydantic					
Instructor					
Outlines					

5.16 Script Evaluasi Error Sederhana

File: `evaluasi_error.py`

```
import json
from pathlib import Path
from pydantic import ValidationError

from ner_model import NEROutput

def evaluasi_file(path_file: str) -> dict:
    path = Path(path_file)

    if not path.exists():
        return {
            "file": path_file,
            "exists": False,
            "json_valid": False,
            "schema_valid": False,
            "errors": ["File tidak ditemukan."]
        }

    raw = path.read_text(encoding="utf-8")

    try:
        data = NEROutput.model_validate_json(raw)
        return {
            "file": path_file,
            "exists": True,
            "json_valid": True,
            "schema_valid": True,
            "jumlah_person": len(data.person),
            "jumlah_organization": len(data.organization),
            "jumlah_location": len(data.location),
            "jumlah_date": len(data.date),
            "jumlah_topic": len(data.topic),
            "errors": []
        }
    except ValidationError as error:
        return {
            "file": path_file,
            "exists": True,
            "json_valid": False,
            "schema_valid": False,
            "errors": error.errors()
        }
```

```

files = [
    "output/prompt_biasa/hasil.json",
    "output/instructor/hasil.json",
    "output/outlines/hasil.json"
]

laporan = [evaluasi_file(file) for file in files]

Path("output/evaluasi").mkdir(parents=True, exist_ok=True)
Path("output/evaluasi/laporan.json").write_text(
    json.dumps(laporan, indent=2, ensure_ascii=False),
    encoding="utf-8"
)

print(json.dumps(laporan, indent=2, ensure_ascii=False))

```

Penjelasan section-by-section

- `evaluasi_file(path_file: str)`
Fungsi untuk mengevaluasi satu file output.
- `if not path.exists()`
Mengecek apakah file ada.
- `raw = path.read_text(...)`
Membaca output LLM sebagai teks.
- `NEROutput.model_validate_json(raw)`
Memvalidasi output terhadap model Pydantic.
- `len(data.person)`
Menghitung jumlah entitas orang.
- `except ValidationError`
Menangkap error validasi.
- `laporan = [...]`
Mengevaluasi semua file.
- `laporan.json`
Menyimpan hasil evaluasi.

5.17 Mengukur Error yang Penting

Dalam Bab 5, error yang diukur masih sederhana.

1. JSON Invalid

Contoh:

```
{  
  "person": ["Dr. Budi"],  
  "organization": ["ITTS"],
```

Masalah: kurung penutup tidak lengkap.

2. Field Hilang

Contoh:

```
{  
  "person": ["Dr. Budi"],  
  "organization": ["ITTS"]  
}
```

Jika model mewajibkan `location`, `date`, dan `topic`, output ini gagal.

3. Tipe Salah

Contoh:

```
{  
  "person": "Dr. Budi",  
  "organization": ["ITTS"],  
  "location": ["Tangerang Selatan"],  
  "date": ["2026-05-12"],  
  "topic": ["AI"]  
}
```

Masalah: `person` harus list, bukan string.

4. Field Tambahan

Contoh:

```
{  
  "person": ["Dr. Budi"],  
  "organization": ["ITTS"],  
  "location": ["Tangerang Selatan"],  
  "date": ["2026-05-12"],  
  "topic": ["AI"],  
  "kesimpulan": "Acara ini penting."  
}
```

Masalah: kesimpulan tidak ada di schema.

5. Salah Kategori

Contoh:

```
{
  "person": ["ITTS"],
  "organization": ["Dr. Budi"]
}
```

Masalah: struktur valid, tetapi makna salah.

Catatan kritis: constrained generation sangat membantu error 1–4. Error 5 tetap membutuhkan evaluasi kualitas ekstraksi.

5.18 Strategi Praktis agar Sistem Lebih Reliable

Gunakan strategi berikut.

1. Mulai dari schema kecil

Jangan langsung membuat schema besar. Mulai dari 5 field.

Untuk NER:

- person
- organization
- location
- date
- topic

2. Uji bertahap

Uji dengan:

- 1 teks pendek;
- 5 teks sedang;
- 20 teks variasi;
- teks dengan entitas kosong;
- teks dengan tanggal ambigu.

3. Logging semua error

Simpan:

- input;
- prompt;
- model;
- schema;
- output;
- error;
- waktu;
- teknik yang dipakai.

4. Gunakan retry dengan batas

Retry membantu, tetapi jangan tanpa batas.

Contoh:

- maksimal 3 kali;
- jika gagal, simpan error;
- jangan paksa data masuk database.

5. Pisahkan validasi struktur dan evaluasi makna

Validasi struktur menjawab:

Apakah JSON sesuai schema?

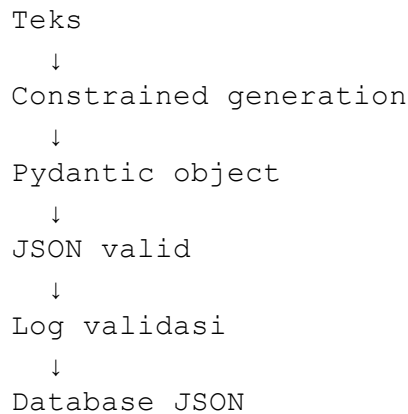
Evaluasi makna menjawab:

Apakah entitas yang diekstrak benar?

Keduanya berbeda.

5.19 Hubungan dengan Database

Output Bab 5 belum langsung masuk database. Tetapi arahnya sudah jelas:



Referensi MySQL & JSON menunjukkan bahwa validasi JSON Schema dapat dipakai sebelum dokumen diterima database, termasuk penggunaan required, minimum, dan constraint check agar dokumen JSON memenuhi aturan sebelum disimpan.

Prinsip untuk mahasiswa:

- jangan simpan output mentah sebagai data final;
- simpan output mentah sebagai audit;
- simpan output valid sebagai data operasional;
- simpan error sebagai bahan evaluasi;
- beri versi pada schema.

5.20 Checklist Praktik Bab 5

Pertanyaan	Ya/Tidak
Apakah model Pydantic sudah dibuat?	
Apakah field tambahan ditolak?	
Apakah prompt biasa sudah diuji?	
Apakah output prompt biasa divalidasi?	
Apakah Instructor sudah dicoba?	
Apakah Outlines sudah dicoba?	
Apakah JSON invalid dihitung?	
Apakah field hilang dicatat?	
Apakah tipe salah dicatat?	
Apakah hasil valid dan error dipisahkan?	
Apakah output siap masuk Bab 6 tentang database JSON?	

5.21 Latihan Mandiri

Latihan 1 — NER Berita Kampus

Gunakan 5 berita kampus pendek. Ekstrak:

- person;
- organization;
- location;
- date;
- topic.

Bandingkan prompt biasa, Pydantic validation, Instructor, dan Outlines.

Latihan 2 — Tambahkan Field event_type

Tambahkan:

```
event_type: Literal["seminar", "workshop",  
"kuliah_umum", "lomba", "lainnya"]
```

Uji apakah model bisa memilih kategori acara dengan benar.

Latihan 3 — Uji Teks Tanpa Entitas

Gunakan teks:

Hari ini cuaca cerah dan mahasiswa sedang belajar pemrograman.

Output yang benar:

- person kosong;
- organization kosong;
- location kosong;
- date kosong;
- topic berisi topik yang relevan jika ada.

Latihan 4 — Hitung Error Rate

Buat tabel:

Teknik	Total Uji	Valid	Invalid	Error Rate
Prompt biasa				
Pydantic validation				
Instructor				
Outlines				

Rumus sederhana:

```
error_rate = jumlah_invalid / total_uji
```

Latihan 5 — Audit Output

Untuk setiap output, simpan:

- `input_text`
- `method`
- `model`
- `schema_version`
- `raw_output`
- `validated_output`
- `error`
- `timestamp`

5.22 Ringkasan Bab

Bab ini memperkenalkan teknik yang lebih ketat dibanding prompt engineering biasa.

Inti Bab 5:

- **Constrained generation** membatasi ruang jawaban LLM agar output lebih stabil.
- **Validation-after-generation** memeriksa output setelah LLM menjawab.
- **Constrained-during-generation** membatasi output saat proses generasi berlangsung.
- **Instructor** memakai Pydantic untuk output type-safe, validasi, retry, dan error handling.
- **Outlines** membuat model menghasilkan JSON sesuai struktur yang ditentukan agar mudah diproses kode downstream.
- **Schema kecil** lebih aman untuk awal pembelajaran.
- **Schema kompleks** bisa memperlambat dan sulit di-debug.
- **Validasi struktur** tidak sama dengan validasi kebenaran fakta.
- **Logging semua error** adalah fondasi reliability.
- **Output yang reliable** adalah output yang valid, dapat ditolak jika salah, dan dapat diaudit.

Output bab ini: mahasiswa memahami bahwa structured output bukan hanya soal meminta LLM menulis JSON, tetapi juga soal membatasi ruang jawaban agar sistem lebih reliable, siap divalidasi, dan siap masuk pipeline database.

Bab 6 — Menyimpan Structured Output ke Database JSON

Golden Rule: Output LLM baru bernilai sebagai data jika dapat disimpan, dicari, divalidasi, dilacak, dan diaudit.

Bab ini menempatkan Bab 6 sebagai titik perubahan: dari output JSON yang hanya tersimpan sebagai file menjadi data yang bisa dikelola dalam database. Dokumen outline juga menekankan bahwa database harus menjadi pusat buku, bukan sekadar tempat menyimpan output; query JSON, *audit trail*, *schema versioning*, dan *generated column* perlu diperkuat.

6.1 Mengapa Output LLM Harus Masuk Database

Pada bab sebelumnya, mahasiswa sudah membuat LLM menghasilkan JSON yang lebih stabil. Namun menyimpan JSON sebagai file saja belum cukup untuk sistem nyata.

File JSON cocok untuk:

- eksperimen kecil;
- debugging;
- menyimpan contoh output;
- dokumentasi latihan.

Tetapi database dibutuhkan ketika data harus:

- dicari berdasarkan aktor;
- difilter berdasarkan lokasi;
- dihitung jumlah topik;
- dikelompokkan berdasarkan model;
- diaudit berdasarkan waktu;
- dibandingkan antar versi schema;
- dipakai oleh dashboard;
- dikirim ke workflow otomatis.

LLM tidak boleh berhenti di output. Output harus masuk ke sistem data.

Pipeline Bab 6:

```
teks berita
↓
LLM lokal
↓
JSON valid
↓
Python validator
↓
database JSON
↓
query, laporan, audit
```

MySQL mendukung tipe data JSON native, melakukan validasi otomatis terhadap dokumen JSON yang disimpan, dan menyimpan dokumen JSON dalam format internal yang dioptimalkan untuk akses elemen JSON. ([MySQL Developer Zone](#))

6.2 Kolom Biasa vs Kolom JSON

Dalam desain tabel, tidak semua data harus masuk kolom JSON. Ada data yang lebih baik disimpan sebagai kolom biasa.

Kolom biasa cocok untuk:

- `id`
- `schema_version`
- `llm_model`
- `validation_status`
- `created_at`
- `source_type`
- `processing_time_ms`

Alasannya:

- sering dipakai untuk filter;
- sering dipakai untuk laporan;
- mudah diindeks;
- tipe datanya jelas;
- tidak berubah-ubah.

Kolom JSON cocok untuk:

- hasil ekstraksi LLM;
- entitas hasil NER;
- metadata yang fleksibel;
- nested object;
- array topik;
- struktur output yang bisa berubah antar schema.

Contoh:

```
{
  "judul": "Seminar AI Lokal di ITTS",
  "tanggal": "2026-05-12",
  "lokasi": "Tangerang Selatan",
  "aktor": ["ITTS", "Dr. Budi Santoso"],
  "topik": ["AI lokal", "JSON Schema", "Python"],
  "ringkasan": "ITTS mengadakan seminar AI lokal tentang JSON Schema dan Python."
}
```

Prinsip desain: data yang sering dipakai untuk kontrol sistem sebaiknya menjadi kolom biasa. Data hasil ekstraksi yang fleksibel boleh masuk kolom JSON.

6.3 Catatan Penting tentang MySQL dan MariaDB

Bab ini memakai gaya SQL yang dekat dengan MySQL. Namun mahasiswa perlu memahami perbedaan penting jika memakai MariaDB.

MySQL memiliki tipe data JSON native dan melakukan validasi otomatis terhadap dokumen JSON di kolom JSON. MariaDB menyediakan tipe **JSON** sebagai alias untuk **LONGTEXT COLLATE utf8mb4_bin**; validasi JSON dapat dilakukan dengan **JSON_VALID()** dan constraint. ([MySQL Developer Zone](#))

Artinya:

- di MySQL, kolom JSON adalah tipe khusus;
- di MariaDB, JSON lebih dekat ke teks panjang dengan validasi;
- fungsi JSON banyak mirip, tetapi detail perilaku bisa berbeda;
- untuk praktek mahasiswa, selalu uji perintah SQL di lingkungan masing-masing.

Agar buku tetap praktis, contoh utama memakai MySQL-style SQL. Jika memakai MariaDB, konsepnya tetap sama: **simpan JSON valid, ekstrak field, buat kolom turunan, beri indeks, dan simpan audit.**

6.4 Desain Tabel untuk Output LLM

Tabel utama yang akan dibuat bernama:

`llm_outputs`

Struktur minimal:

Kolom	Tipe	Fungsi
<code>id</code>	BIGINT	identitas unik
<code>source_text</code>	LONGTEXT	teks asli sebelum diproses LLM
<code>schema_version</code>	VARCHAR	versi schema
<code>llm_model</code>	VARCHAR	nama model LLM
<code>json_output</code>	JSON	output terstruktur dari LLM
<code>validation_status</code>	VARCHAR	status validasi
<code>created_at</code>	TIMESTAMP	waktu data dibuat

Desain ini penting karena output LLM harus dapat dilacak. Dokumen utama buku menegaskan prinsip bahwa LLM tidak boleh dipercaya sebelum output-nya divalidasi, distrukturkan, disimpan, dan dapat diaudit.

6.5 Membuat Database dan Tabel

```
CREATE DATABASE structured_llm;

USE structured_llm;

CREATE TABLE llm_outputs (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    source_text LONGTEXT NOT NULL,
    schema_version VARCHAR(50) NOT NULL,
    llm_model VARCHAR(100) NOT NULL,
    json_output JSON NOT NULL,
    validation_status VARCHAR(30) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Penjelasan bagian per bagian

- `CREATE DATABASE structured_llm;`
Membuat database untuk latihan.
- `USE structured_llm;`
Memilih database yang akan dipakai.
- `CREATE TABLE llm_outputs`
Membuat tabel utama untuk menyimpan output LLM.
- `id BIGINT AUTO_INCREMENT PRIMARY KEY`
Membuat nomor unik otomatis.
- `source_text LONGTEXT NOT NULL`
Menyimpan teks asli. Ini penting untuk audit.
- `schema_version VARCHAR(50) NOT NULL`
Menyimpan versi schema yang dipakai.
- `llm_model VARCHAR(100) NOT NULL`
Menyimpan nama model.
- `json_output JSON NOT NULL`
Menyimpan output LLM yang sudah valid.
- `validation_status VARCHAR(30) NOT NULL`
Menyimpan status seperti `valid` atau `invalid`.
- `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`
Menyimpan waktu insert otomatis.

6.6 Mengapa `source_text` Harus Disimpan

Banyak pemula hanya menyimpan output JSON. Ini berbahaya.

Tanpa `source_text`, kita tidak tahu:

- input awalnya apa;
- apakah LLM mengarang;
- apakah ada entitas yang hilang;
- apakah tanggal berasal dari teks;
- apakah ringkasan terlalu bebas;
- apakah model salah ekstraksi.

Dengan `source_text`, sistem bisa diaudit.

Contoh pertanyaan audit:

- “Kenapa lokasi menjadi Tangerang Selatan?”
- “Dari mana aktor Dr. Budi muncul?”
- “Apakah topik JSON Schema memang ada di teks?”
- “Model apa yang menghasilkan output ini?”
- “Schema versi berapa yang dipakai?”

Audit bukan pekerjaan tambahan. Audit adalah syarat agar output AI layak dipercaya.

6.7 Insert JSON Hasil LLM

Contoh data:

```
INSERT INTO llm_outputs (
    source_text,
    schema_version,
    llm_model,
    json_output,
    validation_status
) VALUES (
    'Pada 12 Mei 2026, Institut Teknologi Tangerang Selatan
    mengadakan seminar AI lokal di Tangerang Selatan. Acara ini
    menghadirkan Dr. Budi Santoso dan membahas JSON Schema,
    Python, serta open source LLM.',
    'berita_v1',
    'llama3.2:3b',
    '{
        "judul": "Seminar AI Lokal di Institut Teknologi
        Tangerang Selatan",
        "tanggal": "2026-05-12",
        "lokasi": "Tangerang Selatan",
        "aktor": [
            "Institut Teknologi Tangerang Selatan",
            "Dr. Budi Santoso"
        ],
        "topik": [
            "AI lokal",
            "JSON Schema",
            "Python",
            "open source LLM"
        ],
        "ringkasan": "Institut Teknologi Tangerang Selatan
        mengadakan seminar AI lokal tentang JSON Schema, Python, dan
        open source LLM."
    }',
    'valid'
);
```

Jika JSON tidak valid secara sintaks, MySQL akan menolak insert ke kolom JSON. Ini berbeda dengan menyimpan JSON sebagai teks biasa. Buku *MySQL & JSON* juga menekankan bahwa penyimpanan JSON sebagai string tidak memberi validasi struktur, sedangkan tipe JSON dirancang untuk menyimpan dokumen JSON valid.

6.8 Query Field JSON

Untuk mengambil field tertentu dari JSON, gunakan `JSON_EXTRACT()` atau operator panah.

Contoh mengambil judul:

```
SELECT
    id,
    JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.judul')) AS judul
FROM llm_outputs;
```

Cara lebih ringkas:

```
SELECT
    id,
    json_output->>'$.judul' AS judul
FROM llm_outputs;
```

Penjelasan

- `JSON_EXTRACT(json_output, '$.judul')`
Mengambil nilai judul dari JSON.
- `JSON_UNQUOTE(...)`
Menghapus tanda kutip JSON dari hasil.
- `->>`
Cara ringkas untuk mengambil nilai JSON sebagai teks.

MySQL menyediakan fungsi pencarian dan ekstraksi JSON seperti `JSON_CONTAINS()`, `JSON_EXTRACT()`, dan operator `MEMBER OF()` untuk memeriksa nilai dalam dokumen JSON. ([MySQL Developer Zone](#))

6.9 Query Berdasarkan Lokasi

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    json_output->>'$.lokasi' AS lokasi
FROM llm_outputs
WHERE json_output->>'$.lokasi' = 'Tangerang Selatan';
```

Query ini menjawab:

Tampilkan semua hasil ekstraksi AI yang lokasinya Tangerang Selatan.

Ini penting untuk:

- arsip berita kampus;
- dashboard kegiatan;
- pencarian lokasi;
- analisis sebaran acara.

6.10 Query Berdasarkan Aktor dalam Array

Karena `aktor` berbentuk array, query-nya berbeda.

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    json_output->'$.aktor' AS aktor
FROM llm_outputs
WHERE JSON_CONTAINS(
    json_output->'$.aktor',
    JSON_QUOTE('Dr. Budi Santoso')
);
```

Penjelasan

- `json_output->'$.aktor'`
Mengambil array aktor sebagai JSON.
- `JSON_QUOTE('Dr. Budi Santoso')`
Mengubah string SQL menjadi string JSON.
- `JSON_CONTAINS(...)`
Memeriksa apakah array aktor mengandung nilai tertentu.

6.11 Query Berdasarkan Topik

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    json_output->'$.topik' AS topik
FROM llm_outputs
WHERE JSON_CONTAINS(
    json_output->'$.topik',
    JSON_QUOTE('JSON Schema')
);
```

Query ini dapat dipakai untuk mencari semua berita yang membahas topik tertentu.

Contoh manfaat:

- mencari berita AI;
- mencari kegiatan Python;
- mencari acara tentang database;
- membangun arsip riset kampus.

6.12 Mengubah JSON Menjadi Tabel dengan JSON_TABLE

Untuk laporan, kadang kita perlu mengubah array JSON menjadi baris tabel. Misalnya setiap topik menjadi satu baris.

```
SELECT
    l.id,
    l.json_output->>'$.judul' AS judul,
    t.topik
FROM llm_outputs AS l,
JSON_TABLE(
    l.json_output,
    '$.topik[*]'
    COLUMNS (
        topik VARCHAR(100) PATH '$'
    )
) AS t;
```

Hasil yang diharapkan

id	judul	topik
1	Seminar AI Lokal...	AI lokal
1	Seminar AI Lokal...	JSON Schema
1	Seminar AI Lokal...	Python
1	Seminar AI Lokal...	open source LLM

`JSON_TABLE()` mengekstrak data JSON dan mengubahnya menjadi bentuk tabel dengan kolom yang didefinisikan pada klausa `COLUMNS`. ([MySQL Developer Zone](#))

Buku *MySQL & JSON* juga menjelaskan bahwa `JSON_TABLE()` dapat memetakan data JSON ke tabel relasional sementara agar dapat diproses dengan SQL biasa.

6.13 Agregasi Topik

Setelah topik menjadi baris, kita dapat menghitung frekuensi topik.

```
SELECT
    t.topik,
    COUNT(*) AS jumlah
FROM llm_outputs AS l,
JSON_TABLE(
    l.json_output,
    '$.topik[*]'
    COLUMNS (
        topik VARCHAR(100) PATH '$'
    )
) AS t
GROUP BY t.topik
ORDER BY jumlah DESC;
```

Query ini menjawab:

Topik apa yang paling sering muncul dari hasil ekstraksi AI?

Manfaat:

- melihat tren acara kampus;
- melihat isu yang sering muncul;
- membuat ringkasan berita;
- membuat dashboard topik.

6.14 Kolom Biasa dari Field JSON: Generated Column

Masalah query JSON adalah performa. Jika tabel makin besar, mengekstrak JSON terus-menerus bisa lambat. Solusinya: buat generated *column* dari field JSON yang sering dipakai.

Contoh: lokasi sering dipakai untuk filter.

```
ALTER TABLE llm_outputs
ADD COLUMN lokasi VARCHAR(100)
GENERATED ALWAYS AS (
    JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.lokasi'))
) STORED;
```

Penjelasan

- `ALTER TABLE`
Mengubah struktur tabel.
- `ADD COLUMN lokasi VARCHAR(100)`
Menambahkan kolom baru bernama `lokasi`.
- `GENERATED ALWAYS AS (...)`
Nilai kolom dibuat otomatis dari ekspresi.
- `JSON_EXTRACT(json_output, '$.lokasi')`
Mengambil field lokasi dari JSON.
- `JSON_UNQUOTE(...)`
Mengubah hasil JSON menjadi teks biasa.
- `STORED`
Nilai disimpan secara fisik agar bisa diindeks.

Dokumentasi MySQL menjelaskan bahwa kolom JSON dapat diindeks secara tidak langsung melalui generated column atau functional index pada ekspresi JSON tertentu. ([MySQL Developer Zone](#))

6.15 Membuat Index pada Generated Column

```
CREATE INDEX idx_lokasi
ON llm_outputs (lokasi);
```

Setelah index dibuat, query lokasi menjadi lebih cepat.

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    lokasi
FROM llm_outputs
WHERE lokasi = 'Tangerang Selatan';
```

Prinsip: jangan mengindeks semua field JSON. Pilih field yang sering dipakai untuk filter dan laporan.

Field yang layak dipertimbangkan:

- lokasi
- tanggal
- schema_version
- llm_model
- validation_status
- created_at

6.16 Generated Column untuk Tanggal

Tambahkan kolom tanggal:

```
ALTER TABLE llm_outputs
ADD COLUMN tanggal DATE
GENERATED ALWAYS AS (
    CAST(JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.tanggal')) AS DATE)
) STORED;
```

Buat index:

```
CREATE INDEX idx_tanggal
ON llm_outputs (tanggal);
```

Query berdasarkan rentang tanggal:

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    tanggal,
    lokasi
FROM llm_outputs
WHERE tanggal BETWEEN '2026-05-01' AND '2026-05-31'
ORDER BY tanggal;
```

Mengapa tanggal perlu generated column?

Karena tanggal sering dipakai untuk:

- laporan bulanan;
- filter dashboard;
- urutan berita;
- audit proses;
- analisis tren.

6.17 Desain Tabel yang Lebih Kuat

Tabel minimal cukup untuk latihan. Namun untuk sistem yang lebih rapi, tambahkan kolom audit.

```
CREATE TABLE llm_outputs (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    source_text LONGTEXT NOT NULL,  
    source_hash CHAR(64),  
    schema_version VARCHAR(50) NOT NULL,  
    llm_model VARCHAR(100) NOT NULL,  
    prompt_version VARCHAR(50),  
    json_output JSON NOT NULL,  
    validation_status VARCHAR(30) NOT NULL,  
    validation_error JSON,  
    processing_time_ms INT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    lokasi VARCHAR(100)  
    GENERATED ALWAYS AS (  
        JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.lokasi'))  
    ) STORED,  
  
    tanggal DATE  
    GENERATED ALWAYS AS (  
        CAST(JSON_UNQUOTE(JSON_EXTRACT(json_output,  
'$.tanggal')) AS DATE)  
    ) STORED,  
  
    INDEX idx_lokasi (lokasi),  
    INDEX idx_tanggal (tanggal),  
    INDEX idx_schema_model (schema_version, llm_model),  
    INDEX idx_status_created (validation_status, created_at)  
);
```

Fungsi kolom tambahan

- `source_hash`
Untuk mendeteksi input duplikat.
- `prompt_version`
Untuk melacak versi prompt.
- `validation_error`
Untuk menyimpan error validasi jika ada.
- `processing_time_ms`
Untuk mengukur performa pipeline.
- `idx_schema_model`
Untuk membandingkan hasil antar model dan schema.
- `idx_status_created`
Untuk audit hasil valid dan gagal.

6.18 Audit Trail: Apa Saja yang Harus Disimpan

Audit trail adalah jejak proses.

Minimal simpan:

- teks input;
- output JSON;
- schema version;
- prompt version;
- model LLM;
- status validasi;
- error validasi;
- waktu proses;
- waktu penyimpanan;
- hash input;
- sumber data.

Mengapa penting?

- **Transparansi:** data dapat ditelusuri.
- **Debugging:** error bisa diperbaiki.
- **Evaluasi model:** model bisa dibandingkan.
- **Evaluasi schema:** schema bisa disempurnakan.
- **Kepercayaan:** output AI tidak menjadi kotak hitam.

Dalam desain data modern, struktur, makna, dan konteks data harus dibawa bersama data itu sendiri agar tidak muncul ambiguitas dan *blind spot*. Buku *Unifying Business, Data, and Code* menekankan bahwa data product yang baik mengemas struktur, meaning, dan konteks, bukan hanya nilai mentah.

Bagian Praktik

6.19 Praktik Pipeline: Teks Berita → JSON Valid → Database

Kita akan membuat pipeline sederhana:

```
teks berita
↓
JSON hasil LLM yang sudah valid
↓
Python
↓
insert ke database
↓
query berdasarkan aktor dan lokasi
```

Kita anggap output LLM sudah divalidasi oleh Pydantic dari Bab 3–5.

6.20 Instalasi Library Python

```
pip install pymysql
```

Penjelasan

- `pip`
Alat untuk memasang paket Python.
- `install`
Perintah untuk memasang.
- `pymysql`
Library Python untuk koneksi ke database MySQL/MariaDB.

6.21 Membuat File Konfigurasi Sederhana

Buat file:

`db_config.py`

Isi:

```
DB_CONFIG = {  
    "host": "localhost",  
    "user": "root",  
    "password": "password_anda",  
    "database": "structured_llm",  
    "charset": "utf8mb4"  
}
```

Penjelasan

- `host`
Lokasi server database.
- `user`
Nama user database.
- `password`
Password database.
- `database`
Nama database yang dipakai.
- `charset`
Gunakan `utf8mb4` agar aman untuk teks Indonesia dan karakter beragam.

Buku *MySQL & JSON* juga menekankan bahwa JSON memakai karakter UTF8MB4.

6.22 Script Insert JSON dari Python

Buat file:

insert_llm_output.py

Isi:

```
import json
import time
import hashlib
import pymysql
from db_config import DB_CONFIG

source_text = """
Pada 12 Mei 2026, Institut Teknologi Tangerang Selatan
mengadakan seminar AI lokal
di Tangerang Selatan. Acara ini menghadirkan Dr. Budi Santoso
dan membahas
JSON Schema, Python, serta open source LLM.
"""

json_output = {
    "judul": "Seminar AI Lokal di Institut Teknologi
Tangerang Selatan",
    "tanggal": "2026-05-12",
    "lokasi": "Tangerang Selatan",
    "aktor": [
        "Institut Teknologi Tangerang Selatan",
        "Dr. Budi Santoso"
    ],
    "topik": [
        "AI lokal",
        "JSON Schema",
        "Python",
        "open source LLM"
    ],
    "ringkasan": "Institut Teknologi Tangerang Selatan
mengadakan seminar AI lokal tentang JSON Schema, Python, dan
open source LLM."
}

schema_version = "berita_v1"
llm_model = "llama3.2:3b"
prompt_version = "prompt_berita_v1"
validation_status = "valid"
processing_time_ms = 850

source_hash = hashlib.sha256(
    source_text.encode("utf-8")
```

```

).hexdigest()

connection = pymysql.connect(**DB_CONFIG)

try:
    with connection.cursor() as cursor:
        sql = """
        INSERT INTO llm_outputs (
            source_text,
            source_hash,
            schema_version,
            llm_model,
            prompt_version,
            json_output,
            validation_status,
            processing_time_ms
        ) VALUES (%s, %s, %s, %s, %s, CAST(%s AS JSON), %s,
%s)
        """

        cursor.execute(
            sql,
            (
                source_text,
                source_hash,
                schema_version,
                llm_model,
                prompt_version,
                json.dumps(json_output, ensure_ascii=False),
                validation_status,
                processing_time_ms
            )
        )

        connection.commit()
        print("Data berhasil disimpan.")

finally:
    connection.close()

```

Penjelasan section-by-section

- `import json`
Mengubah dictionary Python menjadi string JSON.
- `import hashlib`
Membuat hash dari teks sumber.
- `import pymysql`
Koneksi Python ke database.
- `source_text`
Teks asli yang diproses LLM.
- `json_output`
Output LLM yang sudah valid.
- `schema_version`
Versi schema.
- `llm_model`
Nama model.
- `prompt_version`
Versi prompt.
- `source_hash`
Sidik jari digital dari input.
- `pymysql.connect(**DB_CONFIG)`
Membuka koneksi database.
- `cursor.execute(sql, (...))`
Menjalankan insert dengan parameter aman.
- `json.dumps(json_output, ensure_ascii=False)`
Mengubah dictionary menjadi JSON string.
- `CAST(%s AS JSON)`
Memastikan nilai masuk sebagai JSON.
- `connection.commit()`
Menyimpan perubahan.
- `connection.close()`
Menutup koneksi.

6.23 Catatan Keamanan SQL

Gunakan parameter `%s`, jangan menyusun SQL dengan gabungan string.

Buruk:

```
sql = "INSERT INTO llm_outputs (source_text) VALUES ('" +  
source_text + "')
```

Lebih aman:

```
cursor.execute(sql, (source_text,))
```

Mengapa?

- teks input bisa berisi kutip;
- output LLM bisa berisi karakter tak terduga;
- query gabungan string mudah rusak;
- parameter membuat data diperlakukan sebagai nilai, bukan bagian dari perintah SQL.

6.24 Query Output dari Python

Buat file:

query_llm_output.py

Isi:

```
import pymysql
from db_config import DB_CONFIG

connection = pymysql.connect(**DB_CONFIG)

try:
    with connection.cursor(pymysql.cursors.DictCursor) as cursor:
        sql = """
        SELECT
            id,
            json_output->>'$.judul' AS judul,
            json_output->>'$.lokasi' AS lokasi,
            json_output->>'$.tanggal' AS tanggal,
            llm_model,
            schema_version,
            created_at
        FROM llm_outputs
        WHERE json_output->>'$.lokasi' = %s
        ORDER BY created_at DESC
        """

        cursor.execute(sql, ("Tangerang Selatan",))
        rows = cursor.fetchall()

        for row in rows:
            print(row)

finally:
    connection.close()
```

Penjelasan

- `DictCursor`
Hasil query dikembalikan sebagai dictionary.
- `json_output->>'$.judul'`
Mengambil judul dari JSON.
- `WHERE json_output->>'$.lokasi' = %s`
Filter lokasi.
- `ORDER BY created_at DESC`
Data terbaru tampil dulu.

6.25 Membuat Laporan Ringkas dari Data AI

Contoh laporan jumlah berita per lokasi:

```
SELECT
    lokasi,
    COUNT(*) AS jumlah_berita
FROM llm_outputs
WHERE validation_status = 'valid'
GROUP BY lokasi
ORDER BY jumlah_berita DESC;
```

Contoh laporan per model:

```
SELECT
    llm_model,
    schema_version,
    COUNT(*) AS jumlah_output,
    AVG(processing_time_ms) AS rata_waktu_ms
FROM llm_outputs
GROUP BY llm_model, schema_version
ORDER BY jumlah_output DESC;
```

Contoh laporan topik:

```
SELECT
    t.topik,
    COUNT(*) AS jumlah
FROM llm_outputs AS l,
JSON_TABLE(
    l.json_output,
    '$.topik[*]'
    COLUMNS (
        topik VARCHAR(100) PATH '$'
    )
) AS t
WHERE l.validation_status = 'valid'
GROUP BY t.topik
ORDER BY jumlah DESC;
```

Laporan seperti ini membuktikan bahwa output LLM sudah berubah menjadi **aset data**, bukan sekadar teks hasil AI.

6.26 Menyimpan Error Validasi

Jangan buang output gagal. Simpan untuk audit.

Contoh insert error:

```
INSERT INTO llm_outputs (  
    source_text,  
    schema_version,  
    llm_model,  
    json_output,  
    validation_status,  
    validation_error  
) VALUES (  
    'Teks berita contoh yang gagal diproses.',  
    'berita_v1',  
    'llama3.2:3b',  
    '{}',  
    'invalid',  
    '{  
        "error_type": "ValidationError",  
        "field": "tanggal",  
        "message": "tanggal tidak sesuai format YYYY-MM-DD"  
    }'  
) ;
```

Mengapa error disimpan?

- untuk mengetahui pola kegagalan;
- untuk memperbaiki prompt;
- untuk memperbaiki schema;
- untuk membandingkan model;
- untuk membangun retry workflow;
- untuk bahan evaluasi Bab 9.

Error bukan sampah. Error adalah data pembelajaran sistem.

6.27 Tabel Tambahan: validation_logs

Untuk sistem yang lebih rapi, error validasi dapat dipisah.

```
CREATE TABLE validation_logs (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    llm_output_id BIGINT,  
    validation_status VARCHAR(30) NOT NULL,  
    error_detail JSON,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (llm_output_id) REFERENCES llm_outputs(id)  
);
```

Insert log:

```
INSERT INTO validation_logs (  
    llm_output_id,  
    validation_status,  
    error_detail  
) VALUES (  
    1,  
    'invalid',  
    '{  
        "field": "tanggal",  
        "message": "format tanggal tidak valid",  
        "expected": "YYYY-MM-DD"  
    }'  
);
```

Manfaat:

- satu output bisa punya banyak error;
- log tidak mencampuri tabel utama;
- audit lebih rapi;
- evaluasi error lebih mudah.

6.28 Tabel Tambahan: schema_versions

Schema perlu diberi versi.

```
CREATE TABLE schema_versions (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    schema_name VARCHAR(100) NOT NULL,  
    schema_version VARCHAR(50) NOT NULL,  
    schema_body JSON NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE KEY uniq_schema_version (schema_name,  
    schema_version)  
);
```

Insert schema:

```
INSERT INTO schema_versions (  
    schema_name,  
    schema_version,  
    schema_body  
) VALUES (  
    'berita',  
    'berita_v1',  
    '{  
        "type": "object",  
        "properties": {  
            "judul": {"type": "string"},  
            "tanggal": {"type": "string"},  
            "lokasi": {"type": "string"},  
            "aktor": {  
                "type": "array",  
                "items": {"type": "string"}  
            },  
            "topik": {  
                "type": "array",  
                "items": {"type": "string"}  
            },  
            "ringkasan": {"type": "string"}  
        },  
        "required": ["judul", "tanggal", "lokasi", "aktor",  
            "topik", "ringkasan"]  
    }'  
);
```

Mengapa schema version penting?

- schema berubah dari waktu ke waktu;
- output lama harus tetap bisa dibaca;
- audit butuh tahu aturan saat data dibuat;
- model bisa dibandingkan antara versi schema;
- migrasi data lebih aman.

6.29 Query Audit: Melihat Output Berdasarkan Schema dan Model

```
SELECT
    schema_version,
    llm_model,
    validation_status,
    COUNT(*) AS jumlah
FROM llm_outputs
GROUP BY schema_version, llm_model, validation_status
ORDER BY schema_version, llm_model;
```

Query ini menjawab:

Model mana yang paling banyak menghasilkan output valid pada schema tertentu?

Contoh laporan:

schema_version	llm_model	validation_status	jumlah
berita_v1	llama3.2:3b	valid	42
berita_v1	llama3.2:3b	invalid	8
berita_v1	mistral-small	valid	45
berita_v1	mistral-small	invalid	5

Ini penting untuk evaluasi model.

6.30 Query Audit: Melihat Error Terbaru

```
SELECT
    id,
    llm_model,
    schema_version,
    validation_error,
    created_at
FROM llm_outputs
WHERE validation_status = 'invalid'
ORDER BY created_at DESC
LIMIT 20;
```

Tujuan:

- melihat error terbaru;
- memperbaiki prompt;
- memperbaiki schema;
- mendeteksi pola kegagalan.

6.31 Mengapa Tidak Semua Output AI Perlu Dipecah ke Banyak Tabel

Ada dua pendekatan desain.

Pendekatan 1 — Simpan JSON sebagai dokumen

Cocok untuk:

- output fleksibel;
- schema sering berubah;
- riset dan eksperimen;
- audit;
- data semi-terstruktur.

Pendekatan 2 — Pecah ke tabel relasional

Cocok untuk:

- data stabil;
- transaksi penting;
- laporan rutin;
- kebutuhan indeks berat;
- integrasi aplikasi besar.

Untuk buku ini, pendekatan yang dipakai adalah hybrid:

- metadata penting di kolom biasa;
- output LLM disimpan sebagai JSON;
- field penting dibuat *generated column*;
- laporan menggunakan query JSON dan `JSON_TABLE`.

Prinsip: jangan terlalu cepat memecah semua JSON. Tetapi jangan juga menyimpan semua hal dalam JSON tanpa rencana query.

6.32 Pipeline Praktik Lengkap

Struktur folder:

```
bab06_database_json/  
├── db_config.py  
├── create_tables.sql  
├── insert_llm_output.py  
├── query_llm_output.py  
├── report_topics.sql  
└── data/  
    └── berita_sample.json
```

Alur kerja:

1. Buat database.
2. Buat tabel.
3. Ambil output JSON valid dari Bab 4 atau Bab 5.
4. Insert menggunakan Python.
5. Query lokasi.
6. Query aktor.
7. Query topik.
8. Buat generated column.
9. Buat index.
10. Buat laporan ringkas.

6.33 Checklist Desain Database JSON untuk Output LLM

Pertanyaan	Ya/Tidak
Apakah teks sumber disimpan?	
Apakah output JSON disimpan di kolom JSON?	
Apakah schema version disimpan?	
Apakah model LLM disimpan?	
Apakah prompt version disimpan?	
Apakah status validasi disimpan?	
Apakah error validasi disimpan?	
Apakah waktu proses disimpan?	
Apakah field penting dibuat generated column?	
Apakah field yang sering di filter diberi index?	
Apakah query audit tersedia?	
Apakah data invalid tidak dicampur dengan data valid?	

6.34 Latihan Mandiri

Latihan 1 — Insert 5 Berita Kampus

Buat lima data berita dalam bentuk JSON valid.

Setiap data wajib memiliki:

- `judul`
- `tanggal`
- `lokasi`
- `aktor`
- `topik`
- `ringkasan`

Insert ke tabel `llm_outputs`.

Latihan 2 — Query Berdasarkan Aktor

Cari semua berita yang memuat aktor tertentu.

Contoh:

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    json_output->'$.aktor' AS aktor
FROM llm_outputs
WHERE JSON_CONTAINS(
    json_output->'$.aktor',
    JSON_QUOTE('Dr. Budi Santoso')
);
```

Latihan 3 — Query Berdasarkan Lokasi

Cari semua berita dari lokasi tertentu.

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    lokasi,
    tanggal
FROM llm_outputs
WHERE lokasi = 'Tangerang Selatan'
ORDER BY tanggal;
```

Latihan 4 — Laporan Topik

Buat laporan topik paling sering muncul.

Gunakan:

- `JSON_TABLE`
- `GROUP BY`
- `COUNT`
- `ORDER BY`

Latihan 5 — Audit Model

Buat laporan:

- model;
- schema version;
- jumlah output valid;
- jumlah output invalid;
- rata-rata waktu proses.

6.35 Kesalahan Umum

1. Menyimpan hanya JSON tanpa input asli

Akibatnya output sulit diverifikasi.

2. Tidak menyimpan schema version

Akibatnya output lama sulit dipahami saat schema berubah.

3. Tidak menyimpan model

Akibatnya evaluasi model tidak bisa dilakukan.

4. Semua data dimasukkan ke JSON

Akibatnya query dan indexing menjadi sulit.

5. Tidak memakai generated column

Akibatnya query field penting menjadi lambat saat data membesar.

6. Tidak menyimpan error

Akibatnya sistem tidak bisa belajar dari kegagalan.

7. Menganggap JSON valid berarti data benar

JSON valid hanya berarti struktur benar. Fakta tetap perlu dicek.

6.36 Ringkasan Bab

Bab ini mengubah *structured output* dari sekadar file JSON menjadi data yang benar-benar bisa dikelola.

Inti Bab 6:

- Output LLM harus disimpan bersama input, model, schema, waktu, dan status validasi.
- Kolom biasa dipakai untuk metadata penting dan field yang sering difilter.
- Kolom JSON dipakai untuk output LLM yang fleksibel dan semi-terstruktur.
- MySQL mendukung tipe JSON native dan validasi otomatis dokumen JSON.
- MariaDB memakai pendekatan JSON sebagai teks panjang dengan validasi JSON.
- `JSON_EXTRACT`, operator `->>`, `JSON_CONTAINS`, dan `JSON_TABLE` membantu membaca isi JSON.
- Generated column membuat field JSON penting menjadi kolom yang bisa diindeks.
- Index penting untuk query yang sering dilakukan.
- Audit trail wajib ada agar output AI dapat ditelusuri dan dipertanggungjawabkan.
- Error validasi harus disimpan sebagai bahan evaluasi, bukan dibuang.

Output bab ini: mahasiswa mampu menyimpan dan menelusuri structured output LLM di database secara rapi, bukan hanya menyimpan file JSON mentah.

Bab 7 — Otomatisasi Workflow Structured Output dengan n8n

Golden Rule: Python kuat untuk logika dan validasi kompleks; n8n kuat untuk orkestrasi, integrasi, retry, dan otomatisasi lintas sistem.

Bab ini yang menempatkan n8n sebagai lapisan orkestrasi setelah mahasiswa memahami JSON Schema, Pydantic, LLM lokal, *constrained generation*, dan penyimpanan database JSON. Dalam dokumen outline, Bab 7 memang diarahkan untuk menghubungkan webhook, LLM, parser, validator, dan database dalam workflow otomatis.

7.1 Posisi n8n dalam Pipeline AI

n8n adalah alat orkestrasi workflow. Dalam konteks buku ini, n8n bukan pengganti Python dan bukan pengganti database. n8n berperan sebagai **pengatur alur kerja**.

Pipeline besar:

```
Input teks bebas
↓
n8n menerima data
↓
LLM mengekstrak informasi
↓
Structured Output Parser / validasi schema
↓
Code Node / Python service bila perlu
↓
Database JSON
↓
log, error branch, retry, laporan
```

n8n cocok untuk menghubungkan banyak komponen:

- form input;
- webhook;
- HTTP API;
- LLM lokal;
- parser output;
- database;
- notifikasi;
- workflow retry;
- logging;
- cabang error.

Dokumentasi n8n menjelaskan bahwa data antar node bergerak sebagai array of objects dan node dapat dipahami seperti alur *Extract, Transform, Load*: mengambil data, mengubah data, lalu meneruskan data ke tujuan berikutnya. ([n8n Docs](#))

7.2 Mengapa n8n Penting untuk Mahasiswa

Mahasiswa sering bisa membuat script Python. Tetapi sistem nyata biasanya tidak hanya satu script. Sistem nyata butuh alur:

- menerima data dari luar;
- memanggil model;
- memeriksa hasil;
- menyimpan ke database;
- menangani error;
- memberi respons;
- mencatat log;
- mengulang proses jika gagal.

n8n membantu mahasiswa melihat pipeline AI sebagai **sistem berjalan**, bukan hanya kode lokal.

Nilai strategisnya:

- **mudah divisualisasikan;**
- **mudah diuji per node;**
- **cocok untuk demo proyek akhir;**
- **mempercepat integrasi;**
- **mengurangi kode perekat yang berulang;**
- **memudahkan debugging workflow.**

Tetapi prinsip kritis tetap sama: **n8n tidak boleh dipakai untuk melewati validasi**. Output LLM tetap harus distrukturkan, divalidasi, disimpan, dan diaudit. Dokumen utama buku menegaskan prinsip bahwa output LLM tidak boleh dipercaya sebelum melalui proses tersebut.

7.3 Kapan Memakai Python, Kapan Memakai n8n

Gunakan Python ketika:

- validasi data kompleks;
- butuh Pydantic;
- butuh transformasi detail;
- butuh query khusus;
- butuh pemrosesan batch besar;
- butuh unit test;
- butuh library AI/data tertentu;
- butuh kontrol penuh atas error.

Gunakan n8n ketika:

- butuh menerima input dari webhook;
- butuh mengatur urutan proses;
- butuh memanggil API;
- butuh menyimpan ke database;
- butuh cabang sukses/gagal;
- butuh retry sederhana;
- butuh integrasi antar sistem;
- butuh demo visual ke dosen atau tim.

Arsitektur terbaik untuk mahasiswa: hybrid.

n8n = orkestrasi

Python = logika kompleks

LLM = ekstraksi / klasifikasi

Database = penyimpanan

JSON Schema = kontrak

7.4 Node Penting dalam Workflow Structured Output

7.4.1 Webhook Node

Webhook menerima data dari luar melalui HTTP.

Contoh input:

```
{
  "teks": "Nama saya Siti Aulia, NIM 2301001, Prodi
  Informatika..."
}
```

Webhook cocok untuk:

- form mahasiswa;
- aplikasi kampus;
- input dari website;
- trigger dari sistem lain;
- pengujian API sederhana.

7.4.2 HTTP Request Node

HTTP Request dipakai untuk memanggil layanan lain.

Dalam buku ini, HTTP Request dapat dipakai untuk:

- memanggil Ollama lokal;
- memanggil service Python;
- mengambil berita online;
- mengirim data ke API internal;
- memanggil endpoint validasi.

7.4.3 AI Agent / LLM Node

AI Agent atau LLM node dipakai untuk memproses input teks. n8n menjelaskan AI Agent sebagai sistem yang menerima data, membuat keputusan, dan dapat memakai tool/API untuk mencapai tujuan tertentu. ([n8n Docs](#))

Dalam Bab 7, LLM dipakai secara terbatas:

- mengekstrak data;
- membuat ringkasan terstruktur;
- mengubah teks bebas menjadi JSON;
- melakukan klasifikasi sederhana.

Jangan biarkan AI Agent mengambil keputusan database tanpa validasi.

7.4.4 Structured Output Parser Node

Structured Output Parser adalah node penting untuk Bab 7. Dokumentasi n8n menyatakan node ini dipakai untuk mengembalikan field berdasarkan JSON Schema. ([n8n Docs](#))

Namun ada catatan kritis: dokumentasi n8n juga menegaskan bahwa Structured Output Parser dipakai untuk menstrukturkan **output akhir** dari AI Agent, bukan untuk menstrukturkan seluruh langkah perantara yang akan diteruskan ke tool lain. ([n8n Docs](#))

Artinya:

- gunakan parser untuk hasil akhir AI;
- jangan mengandalkan parser sebagai satu-satunya kontrol semua proses;
- untuk output antar langkah, menulis instruksi format di system message;
- untuk data penting, tetap validasi ulang dengan Python/Pydantic atau database constraint.

7.4.5 Code Node

Code Node dipakai untuk transformasi data ringan.

Contoh:

- membersihkan field;
- menambah timestamp;
- membuat `schema_version`;
- mengubah array menjadi string;
- membuat status validasi;
- menyiapkan payload database.

Catatan keamanan: dokumentasi perubahan n8n v2.0 menyebut akses environment variable dari Code Node diblokir secara default demi keamanan; untuk data sensitif, gunakan credentials atau metode aman lain. ([n8n Docs](#))

7.4.6 Database Node

Database node dipakai untuk menyimpan hasil.

Dalam workflow buku ini, database menyimpan:

- `source_text;`
- `schema_version;`
- `llm_model;`
- `json_output;`
- `validation_status;`
- `validation_error;`
- `created_at.`

n8n menyediakan node Postgres untuk menjalankan query serta insert/update row di database. ([n8n Docs](#)) Untuk MySQL/MariaDB, pendekatan yang aman adalah memakai node database yang tersedia di instalasi n8n atau memakai HTTP Request ke service Python kecil yang melakukan insert terkontrol.

7.5 Validasi Schema di Workflow

Ada tiga lapisan validasi yang dapat dipakai.

Lapisan 1 — Structured Output Parser

Dipakai di n8n untuk membuat output AI mengikuti JSON Schema.

Cocok untuk:

- output akhir AI Agent;
- ekstraksi field;
- format ringkasan;
- data form sederhana.

Lapisan 2 — Code Node

Dipakai untuk pengecekan ringan.

Contoh:

- field tidak kosong;
- array benar-benar array;
- status validasi;
- panjang teks;
- fallback default.

Lapisan 3 — Python/Pydantic Service

Dipakai untuk validasi kuat.

Cocok untuk:

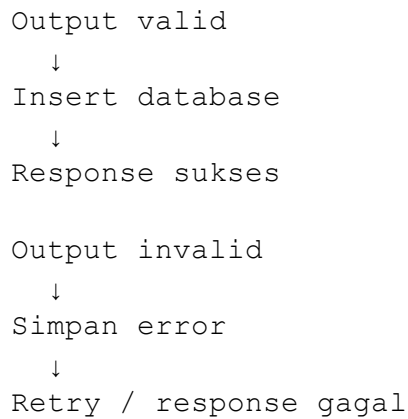
- email;
- enum;
- batas angka;
- validasi tanggal;
- field tambahan;
- error detail;
- schema versioning.

Prinsip: n8n mengatur alur, Python menjaga kualitas data.

7.6 Error Branch dan Retry

Workflow AI harus punya cabang error. Jangan hanya membuat jalur sukses.

Minimal ada dua jalur:



Error yang perlu ditangani:

- LLM tidak merespons;
- output bukan JSON;
- field wajib hilang;
- tipe data salah;
- parser gagal;
- database gagal insert;
- timeout;
- input kosong;
- schema tidak cocok.

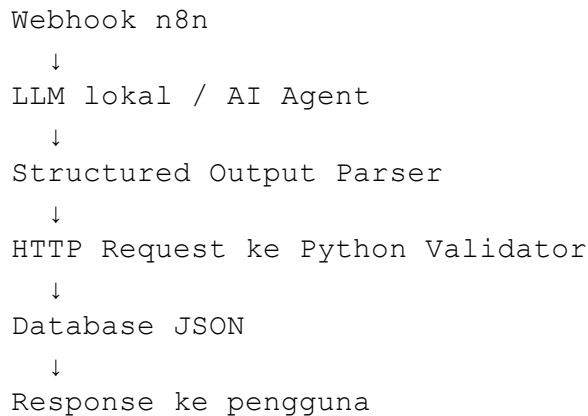
Retry boleh dipakai, tetapi harus dibatasi.

Contoh strategi:

- retry maksimal 3 kali;
- jika gagal, simpan ke log error;
- jangan simpan data invalid ke tabel utama;
- beri status `invalid`;
- simpan raw output untuk audit.

7.7 Hybrid Architecture: n8n + Python + Database

Arsitektur hybrid yang disarankan:



Python validator melakukan:

- validasi Pydantic;
- normalisasi data;
- penolakan field tambahan;
- pembuatan hash input;
- pencatatan error;
- insert database jika valid.

n8n melakukan:

- menerima input;
- memanggil LLM;
- memanggil validator;
- mengatur jalur sukses/gagal;
- retry;
- logging workflow;
- memberi respons.

Inilah pola yang paling sehat untuk mahasiswa: visual di n8n, kuat di Python, rapi di database.

Bagian Praktik

7.8 Contoh Praktik: Sistem Input Form Mahasiswa

Target sistem:

1. Mahasiswa mengisi teks bebas.
2. n8n menerima input lewat Webhook.
3. LLM mengekstrak data.
4. Structured Output Parser memaksa format JSON.
5. Python/Pydantic memvalidasi.
6. Database menyimpan hasil valid.
7. Error disimpan jika validasi gagal.

Input teks:

Nama saya Siti Aulia. NIM saya 2301001. Saya dari Prodi Informatika.
Email saya siti.aulia@example.ac.id. Saya masuk tahun 2023 dan tertarik pada AI, NLP, dan Database.

Output JSON:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP", "Database"]
}
```

7.9 JSON Schema untuk Data Mahasiswa

Gunakan schema sederhana:

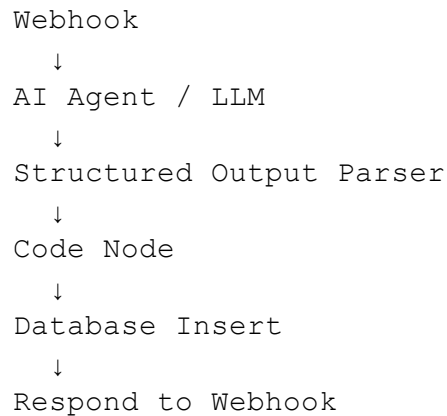
```
{
  "type": "object",
  "properties": {
    "nama": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "nim": {
      "type": "string",
      "minLength": 6,
      "maxLength": 20
    },
    "prodi": {
      "type": "string",
      "enum": ["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    },
    "email": {
      "type": "string"
    },
    "tahun_masuk": {
      "type": "integer",
      "minimum": 2020,
      "maximum": 2026
    },
    "minat_riset": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "maxItems": 5
    }
  },
  "required": ["nama", "nim", "prodi", "email",
    "tahun_masuk", "minat_riset"],
  "additionalProperties": false
}
```

Catatan:

- email akan divalidasi lebih kuat di Python;
- tahun masuk dibatasi;
- prodi memakai `enum`;
- field tambahan ditolak.

7.10 Workflow 1 — Webhook → LLM → JSON Schema → Database

Struktur node



Fungsi setiap node

Webhook

- menerima teks;
- menjadi pintu masuk workflow.

AI Agent / LLM

- membaca teks bebas;
- mengekstrak informasi mahasiswa;
- diarahkan mengikuti schema.

Structured Output Parser

- mengembalikan field berdasarkan JSON Schema;
- menstrukturkan output akhir AI Agent. ([n8n Docs](#))

Code Node

- menambahkan `schema_version`;
- menambahkan `llm_model`;
- menambahkan `validation_status`;
- menyiapkan payload database.

Database Insert

- menyimpan data valid ke tabel.

Respond to Webhook

- mengembalikan status ke pemanggil.

7.11 Contoh Payload Webhook

Kirim data ke Webhook:

```
{
  "teks": "Nama saya Siti Aulia. NIM saya 2301001. Saya dari Prodi Informatika. Email saya siti.aulia@example.ac.id. Saya masuk tahun 2023 dan tertarik pada AI, NLP, dan Database."
}
```

Response sukses:

```
{
  "status": "valid",
  "message": "Data mahasiswa berhasil diekstrak dan disimpan.",
  "data": {
    "nama": "Siti Aulia",
    "nim": "2301001",
    "prodi": "Informatika",
    "email": "siti.aulia@example.ac.id",
    "tahun_masuk": 2023,
    "minat_riset": ["AI", "NLP", "Database"]
  }
}
```

Response gagal:

```
{
  "status": "invalid",
  "message": "Output LLM gagal validasi.",
  "errors": [
    {
      "field": "email",
      "message": "Format email tidak valid."
    }
  ]
}
```

7.12 Code Node untuk Menyiapkan Payload

Contoh kode di Code Node:

```
const item = $input.first().json;

return [
  {
    json: {
      source_text: item.teks_asli || item.teks || "",
      schema_version: "mahasiswa_v1",
      llm_model: "llama3.2:3b",
      json_output: item,
      validation_status: "valid",
      created_at: new Date().toISOString()
    }
  }
];
```

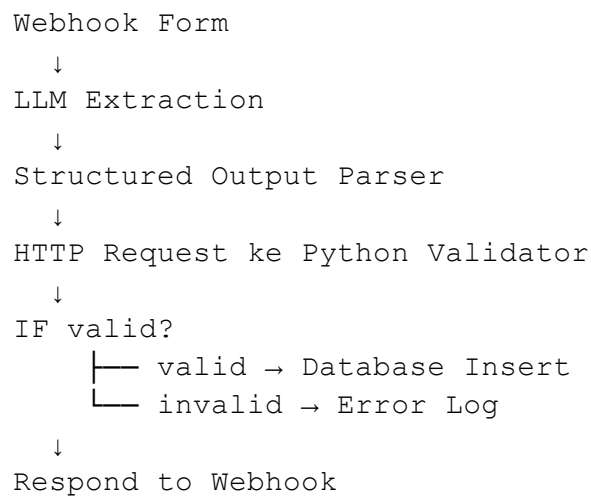
Penjelasan section-by-section

- `const item = $input.first().json;`
Mengambil data pertama dari input node sebelumnya.
- `return [...]`
Mengembalikan data ke node berikutnya.
- `source_text`
Menyimpan teks asli.
- `schema_version`
Menyimpan versi schema.
- `llm_model`
Menyimpan nama model.
- `json_output`
Menyimpan hasil ekstraksi.
- `validation_status`
Menyimpan status validasi.
- `created_at`
Menyimpan waktu proses.

Catatan: Code Node cukup untuk transformasi ringan. Untuk validasi serius, gunakan Python/Pydantic.

7.13 Workflow 2 — Input Form → Extraction → Validation → Database

Workflow kedua lebih kuat karena memakai validator Python.



Mengapa perlu Python Validator?

Karena Python/Pydantic bisa:

- memvalidasi email;
- menolak field tambahan;
- memberi error detail;
- menjaga model data;
- menghasilkan JSON Schema;
- menyimpan error terstruktur.

7.14 Python Validator Service Sederhana

Contoh memakai FastAPI.

Instalasi:

```
pip install fastapi uvicorn pydantic
```

Penjelasan baris

- `fastapi`
Framework API Python.
- `uvicorn`
Server untuk menjalankan FastAPI.
- `pydantic`
Validator data.

File `validator_service.py`:

```
from typing import Literal
from fastapi import FastAPI
from pydantic import BaseModel, ConfigDict, EmailStr, Field,
ValidationError

app = FastAPI()

class Mahasiswa(BaseModel):
    model_config = ConfigDict(extra="forbid")

    nama: str = Field(min_length=2, max_length=100)
    nim: str = Field(min_length=6, max_length=20)
    prodi: Literal["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    email: EmailStr
    tahun_masuk: int = Field(ge=2020, le=2026)
    minat_riset: list[str] = Field(min_length=1,
max_length=5)

@app.post("/validate/mahasiswa")
def validate_mahasiswa(payload: dict):
    try:
        data = Mahasiswa.model_validate(payload)

        return {
            "valid": True,
            "data": data.model_dump(),
            "errors": []
        }

    except ValidationError as error:
        return {
            "valid": False,
```

```
    "data": None,  
    "errors": error.errors()  
}
```

Jalankan:

```
uvicorn validator_service:app --host 0.0.0.0 --port 8000
```

Penjelasan kode

- `FastAPI()`
Membuat aplikasi API.
- `Mahasiswa(BaseModel)`
Model validasi.
- `extra="forbid"`
Menolak field tambahan.
- `Literal[...]`
Membatasi nilai prodi.
- `EmailStr`
Memvalidasi email.
- `@app.post("/validate/mahasiswa")`
Endpoint validasi.
- `payload: dict`
Input dari n8n.
- `Mahasiswa.model_validate(payload)`
Validasi payload.
- `ValidationError`
Error validasi dikembalikan sebagai JSON.

7.15 HTTP Request dari n8n ke Python Validator

Konfigurasi HTTP Request Node:

- Method: `POST`
- URL: `http://localhost:8000/validate/mahasiswa`
- Body: JSON
- Body content: output dari Structured Output Parser

Contoh body:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP", "Database"]
}
```

Output dari validator:

```
{
  "valid": true,
  "data": {
    "nama": "Siti Aulia",
    "nim": "2301001",
    "prodi": "Informatika",
    "email": "siti.aulia@example.ac.id",
    "tahun_masuk": 2023,
    "minat_riset": ["AI", "NLP", "Database"]
  },
  "errors": []
}
```

7.16 IF Node: Cabang Valid dan Invalid

Setelah HTTP Request ke validator, tambahkan IF Node.

Kondisi:

```
valid equals true
```

Jika true:

```
Database Insert
```

Jika false:

```
Error Log
```

Jalur valid:

- simpan ke tabel utama;
- status valid;
- respons sukses.

Jalur invalid:

- simpan ke tabel error;
- status invalid;
- simpan error detail;
- respons gagal.

7.17 Desain Tabel Database untuk Workflow

Tabel utama:

```
CREATE TABLE mahasiswa_extractions (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    source_text LONGTEXT NOT NULL,  
    schema_version VARCHAR(50) NOT NULL,  
    llm_model VARCHAR(100) NOT NULL,  
    json_output JSON NOT NULL,  
    validation_status VARCHAR(30) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabel error:

```
CREATE TABLE mahasiswa_extraction_errors (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    source_text LONGTEXT NOT NULL,  
    schema_version VARCHAR(50) NOT NULL,  
    llm_model VARCHAR(100) NOT NULL,  
    raw_output JSON,  
    validation_error JSON,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Catatan:

- gunakan tabel utama untuk data valid;
- gunakan tabel error untuk audit kegagalan;
- jangan campur data valid dan invalid tanpa status yang jelas.

7.18 Workflow 3 — Berita Online → Structured Summary → Database

Workflow ketiga memakai input berita.

Alur:

```
Manual Trigger / Schedule
↓
HTTP Request ambil halaman berita
↓
Code Node bersihkan teks
↓
LLM ringkas terstruktur
↓
Structured Output Parser
↓
Python Validator
↓
Database Insert
```

Schema berita:

```
{
  "type": "object",
  "properties": {
    "judul": { "type": "string" },
    "tanggal": { "type": "string" },
    "lokasi": { "type": "string" },
    "aktor": {
      "type": "array",
      "items": { "type": "string" }
    },
    "topik": {
      "type": "array",
      "items": { "type": "string" }
    },
    "ringkasan": { "type": "string" }
  },
  "required": ["judul", "tanggal", "lokasi", "aktor", "topik",
    "ringkasan"],
  "additionalProperties": false
}
```

Output:

```
{
  "judul": "Seminar AI Lokal di ITTS",
  "tanggal": "2026-05-12",
  "lokasi": "Tangerang Selatan",
  "aktor": ["ITTS", "Dr. Budi Santoso"],
  "topik": ["AI lokal", "JSON Schema", "Python"],
  "ringkasan": "ITTS mengadakan seminar tentang AI lokal, JSON Schema, dan Python."
}
```

7.19 Logging Workflow untuk Debugging

Workflow AI tanpa log akan sulit diperbaiki.

Minimal log:

- `execution_id`;
- `input_text`;
- `workflow_name`;
- `schema_version`;
- `llm_model`;
- `prompt_version`;
- `raw_output`;
- `validated_output`;
- `validation_status`;
- `validation_error`;
- `created_at`.

Tabel log:

```
CREATE TABLE workflow_logs (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    execution_id VARCHAR(100),  
    workflow_name VARCHAR(100),  
    schema_version VARCHAR(50),  
    llm_model VARCHAR(100),  
    input_text LONGTEXT,  
    raw_output JSON,  
    validated_output JSON,  
    validation_status VARCHAR(30),  
    validation_error JSON,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Log bukan beban tambahan. Log adalah alat belajar sistem.

7.20 Membuat Error Handling Path

Desain cabang error:

```
LLM / Parser gagal
↓
Set status invalid
↓
Simpan source_text, raw_output, error
↓
Optional retry
↓
Respond: invalid
```

Contoh error response:

```
{
  "status": "invalid",
  "message": "Data tidak disimpan karena gagal validasi.",
  "next_action": "Perbaiki input atau ulangi proses dengan
schema yang lebih sederhana."
}
```

Aturan penting:

- jangan menghapus output gagal;
- jangan memasukkan output gagal ke tabel utama tanpa status;
- jangan retry tanpa batas;
- jangan mengubah data diam-diam tanpa log;
- jangan percaya output AI hanya karena bentuknya JSON.

7.21 Strategi Retry

Retry dipakai jika error bersifat sementara.

Cocok untuk retry:

- timeout LLM;
- output JSON rusak;
- parser gagal sesekali;
- respons kosong.
-

Tidak cocok untuk retry:

- schema terlalu kompleks;
- prompt tidak jelas;
- input tidak punya informasi;
- nilai enum tidak sesuai;
- field wajib memang tidak ada.

Strategi praktis:

- retry maksimal 2–3 kali;
- temperature rendah;
- prompt lebih eksplisit;
- schema kecil dulu;
- simpan semua percobaan gagal.

7.22 Checklist Workflow n8n untuk Structured Output

Pertanyaan	Ya/Tidak
Apakah Webhook menerima input teks?	
Apakah input kosong ditolak?	
Apakah schema sudah jelas?	
Apakah Structured Output Parser dipakai untuk output akhir?	
Apakah output divalidasi ulang?	
Apakah Python validator dipakai untuk aturan penting?	
Apakah data valid masuk tabel utama?	
Apakah data invalid masuk log error?	
Apakah source_text disimpan?	
Apakah schema_version disimpan?	
Apakah llm_model disimpan?	
Apakah retry dibatasi?	
Apakah response ke pengguna jelas?	

7.23 Kesalahan Umum

1. Menganggap Parser Sama dengan Validasi Penuh

Structured Output Parser membantu struktur output, tetapi tidak menggantikan validasi bisnis, audit, dan database constraint.

2. Tidak Menyimpan Input Asli

Tanpa input asli, output AI sulit diverifikasi.

3. Tidak Membuat Jalur Error

Workflow yang hanya punya jalur sukses tidak siap untuk sistem nyata.

4. Terlalu Banyak Logika di Code Node

Code Node cocok untuk transformasi ringan. Logika kompleks lebih baik diletakkan di Python.

5. Tidak Memberi Versi Schema

Schema berubah. Tanpa versi, output lama sulit dibaca.

6. Menggunakan Schema Terlalu Kompleks

Schema kompleks membuat LLM lebih mudah gagal dan workflow lebih sulit di-debug.

7. Tidak Menguji dengan Data Buruk

Workflow harus diuji dengan input kosong, field hilang, email salah, prodi tidak valid, dan teks ambigu.

7.24 Latihan Mandiri

Latihan 1 — Form Mahasiswa

Buat workflow:

Webhook → LLM → Structured Output Parser → Python
Validator → Database

Field wajib:

- nama;
- nim;
- prodi;
- email;
- tahun_masuk;
- minat_riset.

Latihan 2 — Cabang Error

Tambahkan IF Node:

- jika `valid = true`, simpan ke tabel utama;
- jika `valid = false`, simpan ke tabel error.

Latihan 3 — Retry

Tambahkan retry maksimal 3 kali untuk output gagal parse.

Catat:

- percobaan ke berapa;
- error;
- output mentah;
- status akhir.

Latihan 4 — Workflow Berita

Buat workflow berita:

HTTP Request → LLM → Structured Output Parser →
Validator → Database

Field wajib:

- judul;
- tanggal;
- lokasi;
- aktor;
- topik;
- ringkasan.

Latihan 5 — Laporan Workflow

Buat query laporan:

- jumlah output valid;
- jumlah output invalid;
- error terbanyak;
- model yang dipakai;
- schema version;
- rata-rata waktu proses.

7.25 Ringkasan Bab

Bab ini menunjukkan cara mengubah pipeline *structured output* menjadi workflow otomatis.

Inti Bab 7:

- **n8n berperan sebagai orkestrator pipeline AI.**
- **Python tetap penting untuk validasi dan logika kompleks.**
- **Structured Output Parser membantu mengembalikan field berdasarkan JSON Schema.**
- **Parser sebaiknya dipakai untuk output akhir AI Agent, bukan semua langkah perantara.**
- **Webhook menerima input, LLM mengekstrak, parser menstrukturkan, validator memeriksa, database menyimpan.**
- **Error branch wajib ada agar data rusak tidak masuk database.**
- **Retry harus dibatasi dan dicatat.**
- **Hybrid architecture lebih sehat: n8n untuk alur, Python untuk validasi, database untuk penyimpanan.**
- **Logging workflow adalah fondasi debugging dan audit.**
- **Sistem AI yang baik bukan hanya bisa menjawab, tetapi bisa gagal dengan aman.**

Output bab ini: **mahasiswa mampu membuat workflow otomatis yang mengubah teks bebas menjadi data database menggunakan n8n, LLM, JSON Schema, Python validator, dan database JSON.**

Bab 8 — Studi Kasus Interaksi Database Berbasis Structured Output

Golden Rule: Portofolio AI yang kuat bukan hanya bisa menjawab pertanyaan, tetapi mampu mengubah teks bebas menjadi data valid yang bisa disimpan, dicari, dihitung, dan diaudit.

Bab ini menjadi bab paling *hands-on*. Semua teknik sebelumnya digabung: JSON, JSON Schema, Pydantic, LLM lokal, *constrained generation*, database JSON, dan workflow. Bab 8 sebagai studi kasus interaksi database berbasis *structured output*.

JSON Schema tetap menjadi kontrak utama karena ia mendeskripsikan struktur, constraint, dan tipe data JSON secara deklaratif. Pydantic dipakai di Python untuk memvalidasi object atau JSON string melalui `model_validate()` dan `model_validate_json()`. MySQL mendukung tipe data JSON native, validasi otomatis dokumen JSON, serta fungsi seperti `JSON_TABLE()` untuk memetakan JSON menjadi bentuk tabel. (json-schema.org)

8.1 Peta Besar Bab

Bab ini membangun lima mini-project:

- **Studi Kasus 1 — Ekstraksi Data Berita Kampus**
- **Studi Kasus 2 — Named Entity Recognition Bahasa Indonesia**
- **Studi Kasus 3 — Form Intelligence**
- **Studi Kasus 4 — Invoice / Kwitansi Sederhana**
- **Studi Kasus 5 — AI Agent dengan Tool Calling Berbasis JSON Schema**

Alur umum semua studi kasus:

```
input teks bebas
  ↓
LLM lokal
  ↓
JSON terstruktur
  ↓
Pydantic validation
  ↓
database
  ↓
query / laporan / audit
```

Target akhir Bab 8: mahasiswa memiliki portofolio mini-project yang membuktikan bahwa *structured output* dapat dipakai untuk membangun aplikasi AI berbasis database, bukan hanya demo chatbot.

8.2 Struktur Folder Proyek

Gunakan satu folder kerja:

```
bab08_studi_kasus/
├── db/
│   ├── create_tables.sql
│   └── reports.sql
├── schemas/
│   ├── berita.schema.json
│   ├── ner.schema.json
│   ├── form_mahasiswa.schema.json
│   ├── invoice.schema.json
│   └── tool_action.schema.json
├── src/
│   ├── config.py
│   ├── models.py
│   ├── llm_client.py
│   ├── db_client.py
│   ├── case1_berita.py
│   ├── case2_ner.py
│   ├── case3_form.py
│   ├── case4_invoice.py
│   └── case5_agent_action.py
└── output/
    ├── valid/
    └── error/
```

Penjelasan:

- `db/` berisi SQL tabel dan laporan.
- `schemas/` berisi JSON Schema.
- `src/` berisi kode Python.
- `output/valid/` menyimpan hasil valid.
- `output/error/` menyimpan output gagal validasi.

8.3 Tabel Audit Umum

Sebelum masuk studi kasus, buat tabel audit umum. Ini dipakai untuk menyimpan semua output LLM.

```
CREATE DATABASE structured_llm_portfolio;

USE structured_llm_portfolio;

CREATE TABLE llm_audit_logs (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  case_name VARCHAR(100) NOT NULL,
  source_text LONGTEXT NOT NULL,
  schema_version VARCHAR(50) NOT NULL,
  llm_model VARCHAR(100) NOT NULL,
  raw_output JSON,
  validated_output JSON,
  validation_status VARCHAR(30) NOT NULL,
  validation_error JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Penjelasan

- `case_name` membedakan studi kasus.
- `source_text` menyimpan input asli.
- `raw_output` menyimpan output mentah LLM.
- `validated_output` menyimpan output yang lolos validasi.
- `validation_error` menyimpan error jika gagal.
- `created_at` mencatat waktu proses.

Jangan hilangkan audit log. Data AI tanpa jejak proses sulit dipercaya.

Studi Kasus 1 — Ekstraksi Data Berita Kampus

8.4 Tujuan Studi Kasus

Input berupa artikel berita kampus. Output JSON berisi:

- `judul`
- `tanggal`
- `aktor`
- `organisasi`
- `lokasi`
- `isu`
- `ringkasan`

Output disimpan ke database. Lalu mahasiswa melakukan query berdasarkan organisasi dan lokasi.

8.5 Schema Berita Kampus

```
{
  "type": "object",
  "properties": {
    "judul": {
      "type": "string",
      "minLength": 5,
      "maxLength": 150
    },
    "tanggal": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
    },
    "aktor": {
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1
    },
    "organisasi": {
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1
    },
    "lokasi": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "isu": {
```

```

        "type": "array",
        "items": { "type": "string" },
        "minItems": 1,
        "maxItems": 10
    },
    "ringkasan": {
        "type": "string",
        "minLength": 20,
        "maxLength": 300
    }
},
"required": ["judul", "tanggal", "aktor", "organisasi",
"lokasi", "isu", "ringkasan"],
"additionalProperties": false
}

```

Catatan desain: `aktor`, `organisasi`, dan `isu` dibuat array karena berita bisa memiliki lebih dari satu entitas.

8.6 Tabel Berita Kampus

```

CREATE TABLE berita_kampus (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    source_text LONGTEXT NOT NULL,
    schema_version VARCHAR(50) NOT NULL,
    llm_model VARCHAR(100) NOT NULL,
    json_output JSON NOT NULL,
    validation_status VARCHAR(30) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    lokasi VARCHAR(100)
    GENERATED ALWAYS AS (
        JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.lokasi'))
    ) STORED,

    tanggal DATE
    GENERATED ALWAYS AS (
        CAST(JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.tanggal')) AS
DATE)
    ) STORED,

    INDEX idx_berita_lokasi (lokasi),
    INDEX idx_berita_tanggal (tanggal)
);

```

Generated column membantu query field JSON yang sering dipakai. MySQL menjelaskan bahwa indexing terhadap dokumen JSON dapat dilakukan melalui generated column yang mengekstrak scalar value dari JSON. ([MySQL Developer Zone](#))

8.7 Model Pydantic Berita

```
from pydantic import BaseModel, ConfigDict, Field

class BeritaKampus(BaseModel):
    model_config = ConfigDict(extra="forbid")

    judul: str = Field(min_length=5, max_length=150)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    aktor: list[str] = Field(min_length=1)
    organisasi: list[str] = Field(min_length=1)
    lokasi: str = Field(min_length=2, max_length=100)
    isu: list[str] = Field(min_length=1, max_length=10)
    ringkasan: str = Field(min_length=20, max_length=300)
```

Penjelasan

- `extra="forbid"` menolak field tambahan.
- `tanggal` dipaksa memakai format `YYYY-MM-DD`.
- `isu` dibatasi agar output tidak melebar.
- `ringkasan` dibatasi agar cocok untuk database dan dashboard.

8.8 Prompt Ekstraksi Berita

Ekstrak informasi dari berita kampus berikut.

Aturan:

- Jawab hanya JSON.
- Ikuti schema.
- Jangan menambah field.
- Tanggal wajib YYYY-MM-DD.
- Aktor, organisasi, dan isu harus array.

Berita:

{teks_berita}

Input contoh:

Pada 12 Mei 2026, Institut Teknologi Tangerang Selatan mengadakan seminar AI lokal di Tangerang Selatan. Acara ini menghadirkan Dr. Budi Santoso dan komunitas Python kampus. Seminar membahas JSON Schema, open source LLM, database JSON, dan otomasi workflow untuk pendidikan.

Output ideal:

```
{
  "judul": "Seminar AI Lokal di Institut Teknologi Tangerang Selatan",
  "tanggal": "2026-05-12",
  "aktor": ["Dr. Budi Santoso"],
  "organisasi": ["Institut Teknologi Tangerang Selatan", "Komunitas Python Kampus"],
  "lokasi": "Tangerang Selatan",
  "isu": ["AI lokal", "JSON Schema", "open source LLM", "database JSON", "otomasi workflow"],
  "ringkasan": "Institut Teknologi Tangerang Selatan mengadakan seminar AI lokal tentang JSON Schema, open source LLM, database JSON, dan otomasi workflow untuk pendidikan."
}
```

8.9 Query Berdasarkan Organisasi dan Lokasi

Query lokasi:

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    lokasi,
    tanggal
FROM berita_kampus
WHERE lokasi = 'Tangerang Selatan'
ORDER BY tanggal DESC;
```

Query organisasi:

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    json_output->'$.organisasi' AS organisasi
FROM berita_kampus
WHERE JSON_CONTAINS(
    json_output->'$.organisasi',
    JSON_QUOTE('Institut Teknologi Tangerang Selatan')
);
```

Query isu memakai JSON_TABLE():

```
SELECT
    i.isu,
    COUNT(*) AS jumlah
FROM berita_kampus AS b,
JSON_TABLE(
    b.json_output,
    '$.isu[*]'
    COLUMNS (
        isu VARCHAR(100) PATH '$'
    )
) AS i
GROUP BY i.isu
ORDER BY jumlah DESC;
```

`JSON_TABLE()` berguna untuk memetakan elemen JSON menjadi baris dan kolom sehingga bisa diproses dengan SQL biasa. ([MySQL Developer Zone](#))

Studi Kasus 2 — Named Entity Recognition Bahasa Indonesia

8.10 Tujuan Studi Kasus

Input berupa teks berita Indonesia. Output berupa JSON array berisi entitas.

Jenis entitas:

- `person`
- `organization`
- `location`
- `date`
- `topic`

Hasil disimpan ke tabel relasional agar mudah dihitung dan dicari.

8.11 Schema NER

```
{
  "type": "object",
  "properties": {
    "entities": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "text": { "type": "string" },
          "label": {
            "type": "string",
            "enum": ["person", "organization", "location", "date",
"topic"]
          },
          "confidence": {
            "type": "number",
            "minimum": 0,
            "maximum": 1
          }
        },
        "required": ["text", "label", "confidence"],
        "additionalProperties": false
      }
    },
    "required": ["entities"],
    "additionalProperties": false
  }
}
```

8.12 Tabel NER Relasional

```
CREATE TABLE ner_documents (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    source_text LONGTEXT NOT NULL,  
    schema_version VARCHAR(50) NOT NULL,  
    llm_model VARCHAR(100) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE ner_entities (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    document_id BIGINT NOT NULL,  
    entity_text VARCHAR(255) NOT NULL,  
    entity_label VARCHAR(50) NOT NULL,  
    confidence DECIMAL(5,4),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (document_id) REFERENCES ner_documents(id),  
    INDEX idx_entity_label (entity_label),  
    INDEX idx_entity_text (entity_text)  
);
```

Mengapa entitas disimpan ke tabel relasional?

Karena kita ingin query seperti:

- entitas paling sering muncul;
- organisasi terbanyak;
- lokasi terbanyak;
- entitas per dokumen;
- topik per periode.

8.13 Model Pydantic NER

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class Entity(BaseModel):
    model_config = ConfigDict(extra="forbid")

    text: str = Field(min_length=1, max_length=255)
    label: Literal["person", "organization", "location",
"date", "topic"]
    confidence: float = Field(ge=0, le=1)

class NERResult(BaseModel):
    model_config = ConfigDict(extra="forbid")

    entities: list[Entity] = Field(default_factory=list)
```

8.14 Contoh Output NER

Input:

Pada 12 Mei 2026, Institut Teknologi Tangerang Selatan mengadakan seminar AI lokal di Tangerang Selatan bersama Dr. Budi Santoso.

Output:

```
{
  "entities": [
    {
      "text": "12 Mei 2026",
      "label": "date",
      "confidence": 0.95
    },
    {
      "text": "Institut Teknologi Tangerang Selatan",
      "label": "organization",
      "confidence": 0.93
    },
    {
      "text": "Tangerang Selatan",
      "label": "location",
      "confidence": 0.92
    },
    {
      "text": "Dr. Budi Santoso",
      "label": "person",
      "confidence": 0.90
    },
    {
      "text": "AI lokal",
      "label": "topic",
      "confidence": 0.88
    }
  ]
}
```

Catatan kritis: `confidence` dari LLM bukan kebenaran ilmiah. Ia adalah skor internal yang harus dipakai hati-hati. Untuk evaluasi serius, perlu dataset label manual.

8.15 Insert NER ke Database

Pola insert:

```
def simpan_ner(connection, source_text, schema_version,
llm_model, ner_result):
    with connection.cursor() as cursor:
        cursor.execute(
            """
            INSERT INTO ner_documents (source_text,
schema_version, llm_model)
            VALUES (%s, %s, %s)
            """,
            (source_text, schema_version, llm_model)
        )

        document_id = cursor.lastrowid

        for entity in ner_result.entities:
            cursor.execute(
                """
                INSERT INTO ner_entities (
                    document_id,
                    entity_text,
                    entity_label,
                    confidence
                ) VALUES (%s, %s, %s, %s)
                """,
                (
                    document_id,
                    entity.text,
                    entity.label,
                    entity.confidence
                )
            )

        connection.commit()
```

Penjelasan

- Dokumen disimpan satu kali di `ner_documents`.
- Entitas disimpan banyak baris di `ner_entities`.
- `document_id` menghubungkan entitas ke dokumen asal.
- Ini pola normalisasi sederhana.

8.16 Query Top Entity

Entitas paling sering muncul:

```
SELECT
    entity_text,
    entity_label,
    COUNT(*) AS jumlah
FROM ner_entities
GROUP BY entity_text, entity_label
ORDER BY jumlah DESC
LIMIT 20;
```

Top organisasi:

```
SELECT
    entity_text,
    COUNT(*) AS jumlah
FROM ner_entities
WHERE entity_label = 'organization'
GROUP BY entity_text
ORDER BY jumlah DESC
LIMIT 10;
```

Top lokasi:

```
SELECT
    entity_text,
    COUNT(*) AS jumlah
FROM ner_entities
WHERE entity_label = 'location'
GROUP BY entity_text
ORDER BY jumlah DESC
LIMIT 10;
```

Studi Kasus 3 — *Form Intelligence*

8.17 Tujuan Studi Kasus

Input berupa jawaban form dalam bahasa bebas. LLM mengekstrak data sesuai schema. Pydantic memvalidasi field wajib. Database menyimpan hasil.

Contoh input:

```
Nama saya Siti Aulia. NIM 2301001. Saya dari Prodi
Informatika. Email saya siti.aulia@example.ac.id. Saya masuk
tahun 2023. Minat riset saya AI, NLP, dan Database.
```

Output:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "prodi": "Informatika",
  "email": "siti.aulia@example.ac.id",
  "tahun_masuk": 2023,
  "minat_riset": ["AI", "NLP", "Database"]
}
```

8.18 Tabel Form Mahasiswa

```
CREATE TABLE mahasiswa_forms (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  source_text LONGTEXT NOT NULL,
  nama VARCHAR(100) NOT NULL,
  nim VARCHAR(30) NOT NULL,
  prodi VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  tahun_masuk INT NOT NULL,
  minat_riset JSON NOT NULL,
  schema_version VARCHAR(50) NOT NULL,
  llm_model VARCHAR(100) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_nim (nim)
);
```

Mengapa tidak semua disimpan sebagai JSON?

Karena data mahasiswa relatif stabil. Field utama lebih baik menjadi kolom biasa:

- mudah dicari;
- mudah diindeks;
- mudah dicek duplikasi;
- mudah dipakai aplikasi.

`minat_riset` tetap JSON karena berbentuk array.

8.19 Model Pydantic Form Mahasiswa

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, EmailStr, Field

class FormMahasiswa(BaseModel):
    model_config = ConfigDict(extra="forbid")

    nama: str = Field(min_length=2, max_length=100)
    nim: str = Field(min_length=6, max_length=30)
    prodi: Literal["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    email: EmailStr
    tahun_masuk: int = Field(ge=2020, le=2026)
    minat_riset: list[str] = Field(min_length=1,
max_length=5)
```

Pydantic memakai model untuk validasi object atau JSON string, termasuk fungsi `model_validate_json()` untuk memvalidasi JSON langsung terhadap model. ([Pydantic](#))

8.20 Insert Form Mahasiswa

```
import json

def simpan_form_mahasiswa(connection, source_text, data,
                           schema_version, llm_model):
    with connection.cursor() as cursor:
        cursor.execute(
            """
            INSERT INTO mahasiswa_forms (
                source_text,
                nama,
                nim,
                prodi,
                email,
                tahun_masuk,
                minat_riset,
                schema_version,
                llm_model
            ) VALUES (%s, %s, %s, %s, %s, %s, CAST(%s AS
JSON), %s, %s)
            """,
            (
                source_text,
                data.nama,
                data.nim,
                data.prodi,
                data.email,
                data.tahun_masuk,
                json.dumps(data.minat_riset,
ensure_ascii=False),
                schema_version,
                llm_model
            )
        )

    connection.commit()
```

Penjelasan

- Field stabil disimpan sebagai kolom biasa.
- `minat_riset` disimpan sebagai JSON array.
- `schema_version` dan `llm_model` tetap dicatat.
- `source_text` tetap disimpan untuk audit.

8.21 Query Form Mahasiswa

Cari mahasiswa berdasarkan prodi:

```
SELECT
    nama,
    nim,
    prodi,
    email,
    tahun_masuk
FROM mahasiswa_forms
WHERE prodi = 'Informatika'
ORDER BY nama;
```

Cari mahasiswa dengan minat riset tertentu:

```
SELECT
    nama,
    nim,
    minat_riset
FROM mahasiswa_forms
WHERE JSON_CONTAINS(
    minat_riset,
    JSON_QUOTE('AI')
);
```

Laporan jumlah mahasiswa per prodi:

```
SELECT
    prodi,
    COUNT(*) AS jumlah
FROM mahasiswa_forms
GROUP BY prodi
ORDER BY jumlah DESC;
```

Studi Kasus 4 — Invoice / Kwitansi Sederhana

8.22 Tujuan Studi Kasus

Input berupa teks hasil OCR dari invoice atau kwitansi sederhana.

Output:

- `nomor_invoice`
- `tanggal`
- `vendor`
- `item`
- `total`
- `pajak`

Validasi:

- tanggal format benar;
- total angka;
- pajak angka;
- item minimal satu;
- jumlah item tidak negatif;
- harga tidak negatif.

8.23 Schema Invoice

```
{
  "type": "object",
  "properties": {
    "nomor_invoice": {
      "type": "string",
      "minLength": 3,
      "maxLength": 50
    },
    "tanggal": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
    },
    "vendor": {
      "type": "string",
      "minLength": 2,
      "maxLength": 150
    },
    "item": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
```

```

    "properties": {
      "nama_item": { "type": "string" },
      "jumlah": {
        "type": "integer",
        "minimum": 1
      },
      "harga_satuan": {
        "type": "number",
        "minimum": 0
      },
      "subtotal": {
        "type": "number",
        "minimum": 0
      }
    },
    "required": ["nama_item", "jumlah", "harga_satuan",
"subtotal"],
    "additionalProperties": false
  },
  "total": {
    "type": "number",
    "minimum": 0
  },
  "pajak": {
    "type": "number",
    "minimum": 0
  }
},
"required": ["nomor_invoice", "tanggal", "vendor", "item",
"total", "pajak"],
"additionalProperties": false
}

```

8.24 Model Pydantic Invoice

```
from pydantic import BaseModel, ConfigDict, Field,
model_validator

class InvoiceItem(BaseModel):
    model_config = ConfigDict(extra="forbid")

    nama_item: str = Field(min_length=1, max_length=150)
    jumlah: int = Field(ge=1)
    harga_satuan: float = Field(ge=0)
    subtotal: float = Field(ge=0)

class Invoice(BaseModel):
    model_config = ConfigDict(extra="forbid")

    nomor_invoice: str = Field(min_length=3, max_length=50)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    vendor: str = Field(min_length=2, max_length=150)
    item: list[InvoiceItem] = Field(min_length=1)
    total: float = Field(ge=0)
    pajak: float = Field(ge=0)

    @model_validator(mode="after")
    def cek_total(self):
        total_item = sum(i.subtotal for i in self.item)
        if self.total < total_item:
            raise ValueError("total tidak boleh lebih kecil
dari jumlah subtotal item")
        return self
```

Penjelasan

- `InvoiceItem` memvalidasi tiap item.
- `Invoice` memvalidasi dokumen invoice.
- `model_validator` mengecek logika antar field.
- JSON Schema kuat untuk struktur; Pydantic lebih nyaman untuk logika bisnis tambahan.

8.25 Tabel Invoice

```
CREATE TABLE invoices (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    source_text LONGTEXT NOT NULL,  
    nomor_invoice VARCHAR(50) NOT NULL,  
    tanggal DATE NOT NULL,  
    vendor VARCHAR(150) NOT NULL,  
    json_output JSON NOT NULL,  
    total DECIMAL(14,2) NOT NULL,  
    pajak DECIMAL(14,2) NOT NULL,  
    schema_version VARCHAR(50) NOT NULL,  
    llm_model VARCHAR(100) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE KEY uniq_invoice (nomor_invoice)  
);
```

8.26 Contoh Input OCR

INVOICE INV-2026-001
Tanggal: 2026-05-12
Vendor: Toko Buku Kampus

1. Buku AI Lokal 2 x 75000 = 150000
2. Modul Python 1 x 50000 = 50000

Pajak: 22000
Total: 222000

Output ideal:

```
{
  "nomor_invoice": "INV-2026-001",
  "tanggal": "2026-05-12",
  "vendor": "Toko Buku Kampus",
  "item": [
    {
      "nama_item": "Buku AI Lokal",
      "jumlah": 2,
      "harga_satuan": 75000,
      "subtotal": 150000
    },
    {
      "nama_item": "Modul Python",
      "jumlah": 1,
      "harga_satuan": 50000,
      "subtotal": 50000
    }
  ],
  "total": 222000,
  "pajak": 22000
}
```

8.27 Query Invoice

Cari invoice per vendor:

```
SELECT
    nomor_invoice,
    tanggal,
    vendor,
    total,
    pajak
FROM invoices
WHERE vendor = 'Toko Buku Kampus'
ORDER BY tanggal DESC;
```

Laporan total per vendor:

```
SELECT
    vendor,
    COUNT(*) AS jumlah_invoice,
    SUM(total) AS total_belanja
FROM invoices
GROUP BY vendor
ORDER BY total_belanja DESC;
```

Ekstrak item invoice memakai JSON_TABLE():

```
SELECT
    i.nomor_invoice,
    jt.nama_item,
    jt.jumlah,
    jt.harga_satuan,
    jt.subtotal
FROM invoices AS i,
JSON_TABLE(
    i.json_output,
    '$.item[*]'
    COLUMNS (
        nama_item VARCHAR(150) PATH '$.nama_item',
        jumlah INT PATH '$.jumlah',
        harga_satuan DECIMAL(14,2) PATH '$.harga_satuan',
        subtotal DECIMAL(14,2) PATH '$.subtotal'
    )
) AS jt;
```


Studi Kasus 5 — AI Agent dengan Tool Calling Berbasis JSON Schema

8.28 Tujuan Studi Kasus

LLM tidak langsung menjalankan perintah database. LLM hanya memilih aksi dalam JSON terstruktur.

Aksi yang diizinkan:

- `create_record`
- `update_record`
- `search_record`
- `summarize_record`

Prinsip kritis: LLM tidak boleh bebas membuat SQL. LLM hanya boleh menghasilkan rencana aksi yang dibatasi schema.

8.29 Schema Tool Action

```
{
  "type": "object",
  "properties": {
    "action": {
      "type": "string",
      "enum": ["create_record", "update_record", "search_record",
"summarize_record"]
    },
    "target_table": {
      "type": "string",
      "enum": ["berita_kampus", "mahasiswa_forms", "invoices"]
    },
    "filters": {
      "type": "object"
    },
    "payload": {
      "type": "object"
    },
    "reason": {
      "type": "string",
      "maxLength": 300
    }
  },
  "required": ["action", "target_table", "filters", "payload",
"reason"],
  "additionalProperties": false
}
```

8.30 Model Pydantic Tool Action

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class ToolAction(BaseModel):
    model_config = ConfigDict(extra="forbid")

    action: Literal[
        "create_record",
        "update_record",
        "search_record",
        "summarize_record"
    ]

    target_table: Literal[
        "berita_kampus",
        "mahasiswa_forms",
        "invoices"
    ]

    filters: dict = Field(default_factory=dict)
    payload: dict = Field(default_factory=dict)
    reason: str = Field(max_length=300)
```

8.31 Contoh Input dan Output Tool Calling

Input user:

Cari semua berita kampus yang lokasinya Tangerang Selatan dan membahas JSON Schema.

Output LLM:

```
{
  "action": "search_record",
  "target_table": "berita_kampus",
  "filters": {
    "lokasi": "Tangerang Selatan",
    "isu": "JSON Schema"
  },
  "payload": {},
  "reason": "Pengguna ingin mencari berita berdasarkan lokasi dan isu."
}
```

Python kemudian memvalidasi output ini. Jika valid, Python memilih query yang sudah disiapkan.

LLM tidak membuat SQL. Python yang memilih SQL aman.

8.32 Dispatcher Aman di Python

```
def jalankan_action(connection, action: ToolAction):
    if action.action == "search_record" and action.target_table ==
"berita_kampus":
        return cari_berita_kampus(connection, action.filters)

    if action.action == "summarize_record" and action.target_table ==
"invoices":
        return ringkas_invoice(connection, action.filters)

    raise ValueError("Aksi tidak diizinkan atau belum didukung.")
```

Contoh fungsi pencarian:

```
def cari_berita_kampus(connection, filters: dict):
    lokasi = filters.get("lokasi")
    isu = filters.get("isu")

    sql = """
SELECT
    id,
    json_output->>'$.judul' AS judul,
    lokasi,
    tanggal,
    json_output->'$.isu' AS isu
FROM berita_kampus
WHERE (%s IS NULL OR lokasi = %s)
    AND (%s IS NULL OR JSON_CONTAINS(json_output->'$.isu',
JSON_QUOTE(%s)))
ORDER BY tanggal DESC
    """

    with connection.cursor() as cursor:
        cursor.execute(sql, (lokasi, lokasi, isu, isu))
        return cursor.fetchall()
```

Penjelasan

- LLM hanya memilih `action`.
- Python memeriksa `action`.
- Query tetap dibuat oleh developer.
- Parameter dipakai agar input tidak digabung mentah ke SQL.
- Ini lebih aman daripada meminta LLM menulis query.

8.33 Query Summary Invoice

```
def ringkas_invoice(connection, filters: dict):
    vendor = filters.get("vendor")

    sql = """
    SELECT
        vendor,
        COUNT(*) AS jumlah_invoice,
        SUM(total) AS total_belanja,
        SUM(pajak) AS total_pajak
    FROM invoices
    WHERE (%s IS NULL OR vendor = %s)
    GROUP BY vendor
    ORDER BY total_belanja DESC
    """

    with connection.cursor() as cursor:
        cursor.execute(sql, (vendor, vendor))
        return cursor.fetchall()
```

8.34 Mengapa Tool Calling Harus Dibatasi Schema

Tanpa schema, LLM bisa membuat aksi liar:

```
{
  "action": "delete_all_records",
  "target_table": "all_tables"
}
```

Dengan schema, aksi hanya terbatas pada daftar yang diizinkan.

Manfaat:

- mengurangi risiko aksi berbahaya;
- membuat sistem mudah diaudit;
- memisahkan niat user dan eksekusi database;
- menjaga query tetap dikendalikan kode;
- membuat agent lebih aman untuk mini-project.

Ollama mendukung *structured outputs* dengan JSON Schema pada parameter `format`, sehingga pola seperti tool action berbasis schema dapat diarahkan agar output mengikuti bentuk tertentu. ([Ollama](#))

8.35 Logging Tool Action

Buat tabel:

```
CREATE TABLE tool_action_logs (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_input LONGTEXT NOT NULL,
  action_json JSON NOT NULL,
  validation_status VARCHAR(30) NOT NULL,
  execution_status VARCHAR(30),
  execution_result JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Simpan setiap aksi:

- input user;
- action JSON;
- status validasi;
- status eksekusi;
- hasil eksekusi.

Tool calling tanpa log berbahaya. Jika agent salah memilih aksi, kita harus tahu alasannya.

8.36 Checklist Mini-Project Bab 8

Komponen	Berita	NER	Form	Invoice	Tool Action
Schema tersedia					
Model Pydantic tersedia					
Prompt tersedia					
Validasi output tersedia					
Tabel database tersedia					
Insert data tersedia					
Query laporan tersedia					
Error log tersedia					
Audit input-output tersedia					

8.37 Strategi Demo Portofolio

Untuk demo kelas atau proyek akhir, gunakan alur sederhana:

1. Tampilkan input teks.
2. Jalankan LLM lokal.
3. Tampilkan output JSON.
4. Tampilkan validasi Pydantic.
5. Simpan ke database.
6. Jalankan query laporan.
7. Tampilkan error handling.
8. Jelaskan audit trail.

Narasi demo:

“Sistem ini tidak hanya meminta LLM menjawab. Sistem ini memaksa output mengikuti schema, memvalidasi dengan Python, menyimpan ke database, lalu membuat query dan laporan dari hasil ekstraksi.”

Itu yang membedakan mini-project serius dari demo chatbot biasa.

8.38 Kesalahan Umum

1. Langsung menyimpan output LLM

Output harus divalidasi dulu.

2. Tidak menyimpan input asli

Tanpa input asli, audit sulit.

3. Semua field disimpan sebagai JSON

Field yang stabil dan sering dicari sebaiknya menjadi kolom biasa.

4. LLM diminta membuat SQL bebas

Ini berisiko. Lebih aman memakai schema aksi dan query yang sudah disiapkan.

5. Tidak memisahkan data valid dan error

Error harus disimpan sebagai data evaluasi.

6. Tidak memberi versi schema

Schema akan berubah. Versi wajib dicatat.

7. Terlalu banyak field sejak awal

Mulai kecil. Uji. Baru perluas.

8.39 Latihan Mandiri

Latihan 1 — Berita Kampus

Buat 10 contoh berita kampus pendek. Ekstrak:

- judul;
- tanggal;
- aktor;
- organisasi;
- lokasi;
- isu;
- ringkasan.

Simpan ke database dan buat laporan top isu.

Latihan 2 — NER Bahasa Indonesia

Gunakan 10 teks berita Indonesia. Ekstrak entitas:

- person;
- organization;
- location;
- date;
- topic.

Simpan ke tabel relasional dan tampilkan top entity.

Latihan 3 — Form Intelligence

Buat 10 jawaban form bebas. Validasi:

- nama;
- NIM;
- prodi;
- email;
- tahun masuk;
- minat riset.

Tolak data yang tidak valid.

Latihan 4 — Invoice Sederhana

Buat 5 teks invoice hasil OCR. Ekstrak:

- nomor invoice;
- tanggal;
- vendor;
- item;
- total;
- pajak.

Validasi total dan tampilkan laporan total per vendor.

Latihan 5 — Tool Calling Aman

Buat agent sederhana yang hanya boleh memilih:

- `search_record`
- `summarize_record`

Uji dengan input:

Cari berita tentang JSON Schema di Tangerang Selatan.

Pastikan LLM menghasilkan action JSON, bukan SQL bebas.

8.40 Ringkasan Bab

Bab ini menggabungkan semua teknik sebelumnya menjadi mini-project portofolio.

Inti Bab 8:

- **Structured output dapat dipakai untuk berita, NER, form, invoice, dan tool calling.**
- **Schema membatasi bentuk output.**
- **Pydantic memvalidasi output sebelum database.**
- **Database membuat output AI bisa dicari, dihitung, dan diaudit.**
- **NER lebih baik disimpan ke tabel relasional jika ingin menghitung top entity.**
- **Form intelligence cocok memakai kolom biasa untuk field stabil.**
- **Invoice membutuhkan validasi angka, tanggal, item, pajak, dan total.**
- **Tool calling harus dibatasi JSON Schema agar LLM tidak bebas menjalankan aksi berbahaya.**
- **LLM tidak boleh menulis SQL bebas untuk sistem penting.**
- **Audit trail wajib ada untuk semua mini-project.**

Output bab ini: mahasiswa memiliki portofolio mini-project yang menunjukkan bahwa structured output dapat dipakai untuk membangun aplikasi AI berbasis database yang praktis, terukur, dan dapat dipertanggungjawabkan.

Bab 9 — Reliability, Error Handling, Audit, dan Evaluasi Kualitas Output

Golden Rule: Sistem AI yang baik bukan sistem yang selalu tampak benar, tetapi sistem yang tahu kapan harus menerima, menolak, mengulang, mencatat, dan meminta verifikasi manusia.

Bab ini menekankan bahwa **LLM tidak boleh dipercaya sebelum output-nya divalidasi, distrukturkan, disimpan, dan dapat diaudit**. Prinsip ini menjadi inti Bab 9: *structured output* bukan sekadar format JSON, tetapi fondasi *AI engineering* yang dapat diuji dan dipertanggungjawabkan.

9.1 Mengapa Reliability Lebih Penting daripada Demo yang Terlihat Pintar

Banyak aplikasi AI terlihat bagus saat demo. Model menjawab lancar, JSON terlihat rapi, dan output tampak meyakinkan. Masalahnya, sistem nyata tidak boleh hanya terlihat benar.

Sistem nyata harus mampu menjawab:

- Apakah JSON valid?
- Apakah field wajib lengkap?
- Apakah tipe data benar?
- Apakah nilai masuk akal?
- Apakah model mengarang field?
- Apakah data duplikat?
- Apakah output bisa diaudit?
- Apakah error disimpan?
- Apakah manusia perlu memeriksa?

Reliability berarti sistem tetap terkendali walaupun LLM salah. Sistem yang baik tidak panik saat output rusak. Sistem yang baik punya jalur aman.

Pipeline ideal:

```
input
  ↓
LLM
  ↓
structured output
  ↓
validasi
  ↓
jika valid → simpan
  ↓
jika invalid → retry / fallback / log error
  ↓
audit
```

Pydantic secara resmi akan menghasilkan `ValidationError` saat data tidak sesuai model; error tersebut berisi informasi tentang kesalahan yang ditemukan selama validasi. Ini penting karena error bukan sekadar pesan gagal, tetapi bahan audit dan perbaikan sistem. ([Pydantic](#))

9.2 Jenis Error dalam Structured Output

9.2.1 JSON Invalid

JSON invalid terjadi ketika output tidak bisa di parsing.

Contoh:

```
{
    "nama": "Siti Aulia",
    "nim": "2301001",
```

Masalah:

- kurung tidak lengkap;
- koma salah;
- tanda kutip hilang;
- model menambahkan paragraf di luar JSON.

Dampak:

- Python gagal membaca;
- database menolak;
- workflow berhenti.

9.2.2 Field Hilang

Contoh:

```
{
    "nama": "Siti Aulia",
    "nim": "2301001"
}
```

Jika schema mewajibkan `prodi`, `email`, `tahun_masuk`, dan `minat_riset`, output ini gagal.

Penyebab:

- prompt kurang tegas;
- input tidak memuat informasi;
- schema terlalu banyak field;
- model mengabaikan field yang dianggap tidak tersedia.

9.2.3 Tipe Data Salah

Contoh:

```
{
  "nama": "Siti Aulia",
  "tahun_masuk": "dua ribu dua puluh tiga",
  "minat_riset": "AI, NLP, Database"
}
```

Masalah:

- `tahun_masuk` seharusnya integer;
- `minat_riset` seharusnya array.

Pydantic dan JSON Schema membantu menangkap error tipe seperti ini. JSON Schema mendefinisikan constraint object melalui `properties`, dan field tambahan dapat dibatasi memakai `additionalProperties: false`. (json-schema.org)

9.2.4 Hallucinated Field

LLM kadang membuat field baru yang tidak diminta.

Contoh:

```
{
  "nama": "Siti Aulia",
  "nim": "2301001",
  "ramalan_karir": "akan menjadi pakar AI terkenal"
}
```

Masalah:

- field tidak ada di schema;
- bisa menyesatkan;
- bisa merusak struktur database;
- bisa menjadi sumber keputusan salah.

Solusi:

- gunakan `additionalProperties: false`;
- di Pydantic gunakan `extra="forbid"`;
- simpan error ke log.

9.2.5 Nilai Tidak Masuk Akal

Contoh:

```
{
  "tahun_masuk": 1800,
  "semester": 99,
  "total_invoice": -50000
}
```

JSON bisa valid. Tipe data bisa benar. Tetapi nilai tidak masuk akal.

Solusi:

- gunakan `minimum`;
- gunakan `maximum`;
- gunakan Pydantic `Field(ge=..., le=...)`;
- gunakan validator khusus untuk logika bisnis.

9.2.6 Duplikasi Data

Contoh:

- form mahasiswa yang sama masuk dua kali;
- invoice dengan nomor sama diproses ulang;
- berita yang sama diekstrak dari sumber berbeda;
- NER menghasilkan entitas berulang.

Solusi:

- gunakan hash input;
- gunakan unique key;
- gunakan normalisasi teks;
- simpan `source_hash`;
- cek sebelum insert.

Contoh hash:

```
import hashlib

source_hash = hashlib.sha256(
    source_text.encode("utf-8")
).hexdigest()
```

9.3 Strategi Retry

Retry berarti sistem mencoba ulang ketika output gagal.

Retry cocok untuk:

- JSON invalid;
- output kosong;
- timeout LLM;
- field hilang karena model tidak patuh;
- format rusak sesekali.

Retry tidak cocok untuk:

- input memang tidak lengkap;
- schema terlalu kompleks;
- nilai enum tidak ada dalam teks;
- field wajib tidak mungkin diisi;
- error bisnis yang harus diperiksa manusia.

Aturan praktis: retry maksimal 2–3 kali. Setelah itu, simpan error dan berhenti.

9.4 Praktik 1 — Retry Otomatis di Python

```
import json
from pydantic import BaseModel, ConfigDict, Field, ValidationError

class BeritaKampus(BaseModel):
    model_config = ConfigDict(extra="forbid")
    judul: str = Field(min_length=5, max_length=150)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    lokasi: str = Field(min_length=2, max_length=100)
    aktor: list[str] = Field(min_length=1)
    topik: list[str] = Field(min_length=1)
    ringkasan: str = Field(min_length=20, max_length=300)

def validasi_berita(raw_output: str):
    try:
        data = BeritaKampus.model_validate_json(raw_output)
        return True, data.model_dump(), None
    except ValidationError as error:
        return False, None, error.errors()

def ekstraksi_dengan_retry(panggil_llm, prompt: str, max_retry: int = 3):
    semua_error = []

    for percobaan in range(1, max_retry + 1):
        raw_output = panggil_llm(prompt)
        valid, data, errors = validasi_berita(raw_output)

        if valid:
            return {
                "status": "valid",
                "percobaan": percobaan,
                "data": data,
                "raw_output": raw_output,
                "errors": []
            }

        semua_error.append({
            "percobaan": percobaan,
            "raw_output": raw_output,
            "errors": errors
        })

    return {
        "status": "invalid",
        "percobaan": max_retry,
        "data": None,
        "raw_output": None,
        "errors": semua_error
    }
```

Penjelasan bagian penting

- `max_retry` membatasi jumlah percobaan.
- `validasi_berita()` memisahkan validasi dari pemanggilan LLM.
- `semua_error` menyimpan seluruh kegagalan.
- Jika semua retry gagal, sistem mengembalikan status `invalid`.

Jangan retry tanpa batas. Sistem AI harus tahu kapan berhenti.

9.5 Fallback Jika Output Gagal Validasi

Fallback adalah jalur cadangan.

Contoh fallback:

- simpan ke tabel error;
- minta input diperbaiki;
- turunkan schema menjadi lebih sederhana;
- jalankan model lain;
- kirim ke pemeriksa manusia;
- simpan sebagai “perlu review”.

Fallback yang aman:

```
LLM gagal
↓
validasi gagal
↓
retry maksimal 3 kali
↓
jika tetap gagal
↓
simpan error
↓
status = perlu_review
```

Fallback yang buruk:

```
LLM gagal
↓
simpan saja ke database
```

Itu berbahaya.

9.6 Schema Versioning

Schema akan berubah. Hari ini field berita hanya:

- judul;
- tanggal;
- lokasi;
- aktor;
- topik.

Besok field bertambah:

- sumber;
- bahasa;
- `tingkat_kepercayaan;`
- `kategori_berita.`

Jika tidak ada versi schema, data lama dan baru sulit dibedakan.

Gunakan format:

```
berita_v1
berita_v2
mahasiswa_v1
invoice_v1
ner_v1
tool_action_v1
```

Tabel schema:

```
CREATE TABLE schema_versions (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  schema_name VARCHAR(100) NOT NULL,
  schema_version VARCHAR(50) NOT NULL,
  schema_body JSON NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_schema_version (schema_name,
  schema_version)
);
```

Insert schema:

```
INSERT INTO schema_versions (  
    schema_name,  
    schema_version,  
    schema_body  
) VALUES (  
    'berita',  
    'berita_v1',  
    '{  
        "type": "object",  
        "properties": {  
            "judul": {"type": "string"},  
            "tanggal": {"type": "string"},  
            "lokasi": {"type": "string"}  
        },  
        "required": ["judul", "tanggal", "lokasi"],  
        "additionalProperties": false  
    }'  
);
```

Schema versioning adalah fondasi audit. Tanpa versi, kita tidak tahu aturan apa yang dipakai saat output dibuat.

9.7 Logging Prompt, Model, Parameter, Latency, dan Error

Log minimal untuk sistem *structured output*:

- `input_text`
- `prompt_version`
- `prompt_text`
- `schema_version`
- `llm_model`
- `temperature`
- `max_retry`
- `latency_ms`
- `raw_output`
- `validated_output`
- `validation_status`
- `validation_error`
- `created_at`

Tabel log error:

```
CREATE TABLE llm_error_logs (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    case_name VARCHAR(100) NOT NULL,  
    input_text LONGTEXT NOT NULL,  
    prompt_version VARCHAR(50) NOT NULL,  
    prompt_text LONGTEXT,  
    schema_version VARCHAR(50) NOT NULL,  
    llm_model VARCHAR(100) NOT NULL,  
    temperature DECIMAL(4,2),  
    max_retry INT,  
    attempt_number INT,  
    latency_ms INT,  
    raw_output JSON,  
    validation_error JSON,  
    error_type VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Mengapa perlu latency?

Latency menunjukkan waktu proses.

Manfaat:

- membandingkan model;
- mendeteksi workflow lambat;
- menghitung biaya komputasi;
- memilih model kecil atau besar;
- menilai apakah schema terlalu kompleks.

9.8 Praktik 2 — Logger Error di Python

```
import json
import time
import pymysql

def simpan_error_log(
    connection,
    case_name,
    input_text,
    prompt_version,
    prompt_text,
    schema_version,
    llm_model,
    temperature,
    max_retry,
    attempt_number,
    latency_ms,
    raw_output,
    validation_error,
    error_type
):
    with connection.cursor() as cursor:
        sql = """
        INSERT INTO llm_error_logs (
            case_name,
            input_text,
            prompt_version,
            prompt_text,
            schema_version,
            llm_model,
            temperature,
            max_retry,
            attempt_number,
            latency_ms,
            raw_output,
            validation_error,
            error_type
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
        CAST(%s AS JSON), CAST(%s AS JSON), %s)
        """

        cursor.execute(
            sql,
            (
                case_name,
                input_text,
                prompt_version,
```

```

        prompt_text,
        schema_version,
        llm_model,
        temperature,
        max_retry,
        attempt_number,
        latency_ms,
        json.dumps(raw_output, ensure_ascii=False),
        json.dumps(validation_error,
ensure_ascii=False),
        error_type
    )
)

connection.commit()

```

Penjelasan singkat

- `case_name` membedakan studi kasus.
- `prompt_version` melacak prompt.
- `schema_version` melacak schema.
- `llm_model` melacak model.
- `latency_ms` mencatat waktu.
- `raw_output` menyimpan output mentah.
- `validation_error` menyimpan error terstruktur.
- `error_type` memudahkan laporan.

9.9 Human-in-the-Loop untuk Data Sensitif

Tidak semua output AI boleh otomatis masuk keputusan akhir.

Butuh *human-in-the-loop* untuk:

- data akademik;
- data keuangan;
- data identitas;
- invoice bernilai besar;
- dokumen hukum;
- keputusan status mahasiswa;
- data yang berdampak pada hak orang.

Alur aman:

```
output valid secara schema
    ↓
risiko rendah → simpan otomatis
    ↓
risiko tinggi → status perlu_review
    ↓
manusia memeriksa
    ↓
approve / reject
```

Tambahkan field:

```
review_status VARCHAR(30) DEFAULT 'not_required',
reviewed_by VARCHAR(100),
reviewed_at TIMESTAMP NULL
```

Nilai `review_status`:

- `not_required`
- `pending`
- `approved`
- `rejected`

AI boleh mempercepat proses. Manusia tetap bertanggung jawab untuk keputusan sensitif.

9.10 Evaluasi Kualitas Output

Evaluasi tidak cukup hanya melihat satu contoh berhasil. Kita perlu metrik.

9.10.1 Valid JSON Rate

Mengukur berapa persen output yang bisa diparsing sebagai JSON.

```
valid_json_rate = jumlah_json_valid / total_output
```

Contoh:

- total output: 100
- JSON valid: 92
- valid JSON rate: 92%

9.10.2 Schema Compliance Rate

Mengukur berapa persen output yang lolos schema.

```
schema_compliance_rate = jumlah_lolos_schema /  
total_output
```

Contoh:

- total output: 100
- lolos schema: 85
- schema compliance rate: 85%

9.10.3 Extraction Accuracy

Mengukur apakah hasil ekstraksi benar dibanding label manual.

```
extraction_accuracy = jumlah_entitas_benar /  
jumlah_entitas_gold_standard
```

Catatan: butuh dataset kecil yang sudah diberi label manual.

9.10.4 False Positive

False positive berarti model mengekstrak sesuatu yang seharusnya tidak ada.

Contoh:

- model menandai “AI lokal” sebagai organisasi;
- model membuat nama orang yang tidak ada di teks;
- model menganggap lokasi sebagai topik.

9.10.5 False Negative

False negative berarti model gagal mengambil entitas yang seharusnya ada.

Contoh:

- teks menyebut “Tangerang Selatan”;
- output tidak memasukkan lokasi tersebut.

9.11 Tabel Evaluasi Output

```
CREATE TABLE output_evaluations (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  case_name VARCHAR(100) NOT NULL,  
  total_output INT NOT NULL,  
  json_valid_count INT NOT NULL,  
  schema_valid_count INT NOT NULL,  
  true_positive INT,  
  false_positive INT,  
  false_negative INT,  
  valid_json_rate DECIMAL(5,4),  
  schema_compliance_rate DECIMAL(5,4),  
  precision_score DECIMAL(5,4),  
  recall_score DECIMAL(5,4),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Rumus:

```
precision = true_positive / (true_positive + false_positive)  
recall = true_positive / (true_positive + false_negative)
```

Gunakan precision jika ingin mengukur **seberapa bersih hasil ekstraksi**.

Gunakan recall jika ingin mengukur **seberapa banyak entitas penting berhasil ditangkap**.

9.12 Praktik 3 — Menghitung Metrik Sederhana dengan Python

```
def hitung_metrik(total_output, json_valid, schema_valid):
    valid_json_rate = json_valid / total_output if total_output else 0
    schema_compliance_rate = schema_valid / total_output if
total_output else 0

    return {
        "total_output": total_output,
        "valid_json_rate": round(valid_json_rate, 4),
        "schema_compliance_rate": round(schema_compliance_rate, 4)
    }

hasil = hitung_metrik(
    total_output=100,
    json_valid=92,
    schema_valid=85
)

print(hasil)
```

Output:

```
{
  "total_output": 100,
  "valid_json_rate": 0.92,
  "schema_compliance_rate": 0.85
}
```

9.13 Praktik 4 — Precision dan Recall untuk NER

```
def hitung_precision_recall(true_positive, false_positive,
false_negative):
    precision = true_positive / (true_positive +
false_positive) \
        if (true_positive + false_positive) else 0

    recall = true_positive / (true_positive + false_negative)
\
        if (true_positive + false_negative) else 0

    return {
        "precision": round(precision, 4),
        "recall": round(recall, 4)
    }

hasil = hitung_precision_recall(
    true_positive=80,
    false_positive=10,
    false_negative=20
)

print(hasil)
```

Output:

```
{
  "precision": 0.8889,
  "recall": 0.8
}
```

Penjelasan

- **true_positive**: entitas benar ditemukan.
- **false_positive**: entitas salah ikut ditemukan.
- **false_negative**: entitas benar tetapi tidak ditemukan.
- **precision**: kualitas kebersihan output.
- **recall**: kemampuan menangkap entitas penting.

9.14 Dashboard Sederhana Valid vs Gagal

Dashboard sederhana bisa dibuat dari query SQL.

Jumlah output valid dan gagal:

```
SELECT
    validation_status,
    COUNT(*) AS jumlah
FROM llm_audit_logs
GROUP BY validation_status;
```

Jumlah error per jenis:

```
SELECT
    error_type,
    COUNT(*) AS jumlah
FROM llm_error_logs
GROUP BY error_type
ORDER BY jumlah DESC;
```

Rata-rata latency per model:

```
SELECT
    llm_model,
    AVG(latency_ms) AS rata_latency_ms,
    COUNT(*) AS jumlah
FROM llm_error_logs
GROUP BY llm_model
ORDER BY rata_latency_ms DESC;
```

Jumlah output per schema:

```
SELECT
    schema_version,
    validation_status,
    COUNT(*) AS jumlah
FROM llm_audit_logs
GROUP BY schema_version, validation_status
ORDER BY schema_version;
```

Untuk laporan JSON yang perlu diubah menjadi tabel, MySQL menyediakan `JSON_TABLE()` agar data JSON dapat dipetakan menjadi kolom relasional. ([MySQL Developer Zone](#))

9.15 Error Handling di Workflow n8n

Jika pipeline memakai n8n, workflow harus punya jalur error. Dokumentasi n8n menyarankan perancangan error handling agar workflow dapat menangani kegagalan dengan lebih terkendali, dan Error Trigger node dapat dipakai untuk membuat error workflow saat workflow lain gagal. ([n8n Docs](#))

Desain dasar:

```
Main Workflow
  ↓
LLM / Parser / Database
  ↓
Jika gagal
  ↓
Error Workflow
  ↓
Simpan error ke database
  ↓
Kirim status gagal
```

Data error yang perlu disimpan:

- nama workflow;
- execution id;
- node yang gagal;
- input;
- output mentah;
- pesan error;
- waktu gagal;
- status akhir.

9.16 Keamanan Data dan Etika Penggunaan LLM

Keamanan bukan tambahan akhir. Keamanan adalah bagian dari desain.

Prinsip dasar:

- jangan masukkan rahasia ke prompt jika tidak perlu;
- jangan simpan data sensitif tanpa alasan jelas;
- batasi akses database;
- simpan log seperlunya;
- hapus data uji yang tidak diperlukan;
- pisahkan data valid dan error;
- jangan menjadikan output LLM sebagai keputusan akhir untuk data sensitif;
- beri tahu pengguna jika data diproses AI.

Untuk output berbasis JSON, ingat: JSON valid tidak berarti aman. JWT, misalnya, memang membungkus JSON dalam format standar, tetapi signed JWT tidak menyembunyikan isi dari pihak yang memegang token; artinya data sensitif tidak boleh asal dimasukkan ke payload yang bisa dibaca.

9.17 Checklist Audit

Gunakan checklist ini sebelum sistem dianggap layak demo.

Pertanyaan	Ya/Tidak
Apakah setiap output punya <code>schema_version</code> ?	
Apakah setiap output punya <code>llm_model</code> ?	
Apakah prompt version disimpan?	
Apakah parameter penting disimpan?	
Apakah latency dicatat?	
Apakah raw output disimpan?	
Apakah output valid dipisahkan dari output error?	
Apakah error validasi disimpan?	
Apakah retry dibatasi?	
Apakah fallback tersedia?	
Apakah data sensitif butuh review manusia?	
Apakah metrik valid JSON rate dihitung?	
Apakah schema compliance rate dihitung?	
Apakah precision dan recall dihitung untuk NER?	
Apakah ada query dashboard valid vs gagal?	

9.18 Latihan Mandiri

Latihan 1 — Buat Tabel Error Log

Buat tabel `llm_error_logs` dan simpan minimal 10 error dari eksperimen Bab 8.

Wajib berisi:

- input;
- model;
- schema;
- raw output;
- error;
- latency.

Latihan 2 — Retry Otomatis

Buat fungsi retry maksimal tiga kali untuk kasus berita kampus.

Jika tetap gagal:

- simpan ke log error;
- beri status `invalid`;
- jangan simpan ke tabel utama.

Latihan 3 — Fallback Review Manual

Tambahkan field:

- `review_status`
- `reviewed_by`
- `reviewed_at`

Gunakan untuk invoice atau form mahasiswa.

Latihan 4 — Dashboard Valid vs Gagal

Buat query:

- jumlah valid;
- jumlah invalid;
- error terbanyak;
- schema paling sering gagal;
- model dengan latency tertinggi.

Latihan 5 — Evaluasi NER

Ambil 20 teks berita pendek. Buat label manual untuk entitas.

Hitung:

- true positive;
- false positive;
- false negative;
- precision;
- recall.

9.19 Ringkasan Bab

Bab ini menegaskan bahwa *structured output* adalah fondasi *AI engineering*, bukan sekadar format data.

Inti Bab 9:

- **Sistem AI yang baik harus tahu kapan menolak output.**
- **JSON invalid, field hilang, tipe salah, hallucinated field, nilai tidak masuk akal, dan duplikasi harus ditangani.**
- **Retry membantu, tetapi harus dibatasi.**
- **Fallback diperlukan ketika retry gagal.**
- **Schema versioning wajib agar output lama tetap bisa diaudit.**
- **Logging prompt, model, parameter, latency, dan error adalah fondasi debugging.**
- **Human-in-the-loop diperlukan untuk data sensitif.**
- **Valid JSON rate dan schema compliance rate mengukur stabilitas format.**
- **Precision dan recall mengukur kualitas ekstraksi.**
- **Keamanan dan etika harus masuk sejak desain awal.**

Output bab ini: mahasiswa memahami bahwa *structured output* bukan hanya cara membuat LLM menulis JSON, tetapi cara membangun sistem AI yang dapat diuji, gagal dengan aman, diperbaiki, dan dipertanggungjawabkan.

Bab 10 — Proyek Akhir: Membangun Sistem AI + Database yang Siap Didemokan

Golden Rule: Proyek akhir yang kuat bukan yang paling rumit, tetapi yang punya alur lengkap: **input → LLM → JSON Schema → validasi → database → audit → demo.**

Bab terakhir ini mengarahkan mahasiswa membuat proyek akhir yang dapat dipresentasikan, diuji, dan dikembangkan menjadi portofolio. Proyek wajib menggabungkan JSON Schema, Pydantic, open source LLM, database JSON, workflow atau script Python, logging, error handling, dokumentasi, screenshot, dan evaluasi kualitas output.

10.1 Tujuan Proyek Akhir

Proyek akhir bukan sekadar latihan teknis. Proyek ini harus membuktikan bahwa mahasiswa mampu membangun sistem kecil yang **berjalan end-to-end**.

Sistem harus mampu:

- menerima **teks bebas**;
- memproses teks dengan **open source LLM**;
- menghasilkan **JSON terstruktur**;
- memvalidasi output dengan **JSON Schema / Pydantic**;
- menyimpan data ke **database**;
- mencatat **log dan error**;
- menampilkan **query atau laporan**;
- menjelaskan hasil dalam demo.

JSON Schema dipakai karena ia adalah bahasa deklaratif untuk mendeskripsikan struktur, constraint, dan tipe data JSON. ([JSON Schema](#)) Pydantic dipakai karena dapat memvalidasi object dan JSON melalui model Python, termasuk `model_validate()` dan `model_validate_json()`. ([Pydantic](#)) Ollama dapat dipakai untuk *structured outputs* dengan JSON Schema agar respons model lebih konsisten. ([Ollama Dokumentasi](#))

10.2 Pilihan Proyek Akhir

Mahasiswa memilih **satu proyek**. Jangan mengambil terlalu banyak. Lebih baik satu proyek kecil tetapi selesai, valid, bisa diuji, dan bisa didemokan.

Pilihan 1 — Sistem Ekstraksi Berita Indonesia ke Database

Input:

- artikel berita;
- berita kampus;
- berita organisasi;
- berita kegiatan mahasiswa.

Output JSON:

- `judul`
- `tanggal`
- `aktor`
- `organisasi`
- `lokasi`
- `isu`
- `ringkasan`

Fitur demo:

- insert berita;
- query berdasarkan lokasi;
- query berdasarkan organisasi;
- laporan top isu.

Pilihan 2 — Sistem NER Otomatis untuk Dokumen Kampus

Input:

- pengumuman kampus;
- berita kampus;
- dokumen kegiatan;
- notulen pendek.

Output JSON:

- `person`
- `organization`
- `location`
- `date`
- `topic`

Fitur demo:

- ekstraksi entitas;
- simpan entitas ke tabel relasional;
- tampilkan top entity;
- hitung entity per dokumen.

Pilihan 3 — Sistem Input Form Pintar Berbasis LLM

Input:

- jawaban form dalam bahasa bebas.

Output JSON:

- `nama`
- `nim`
- `prodi`
- `email`
- `tahun_masuk`
- `minat_riset`

Fitur demo:

- input teks bebas;
- validasi email;
- validasi program studi;
- simpan data valid;
- tolak data invalid.

Pilihan 4 — Sistem Ekstraksi Invoice / Kwitansi Sederhana

Input:

- teks hasil OCR;
- salinan teks invoice sederhana;
- kwitansi pembelian.

Output JSON:

- `nomor_invoice`
- `tanggal`
- `vendor`
- `item`
- `total`
- `pajak`

Fitur demo:

- validasi angka;
- validasi tanggal;
- cek total;
- laporan total per vendor.

Pilihan 5 — Sistem Chatbot Database dengan Structured Query Intent

Input:

Cari berita kampus di Tangerang Selatan yang membahas JSON Schema.

Output LLM bukan SQL bebas, tetapi JSON intent:

```
{
  "intent": "search_record",
  "target_table": "berita_kampus",
  "filters": {
    "lokasi": "Tangerang Selatan",
    "isu": "JSON Schema"
  }
}
```

Fitur demo:

- LLM memilih intent;
- Python memvalidasi intent;
- Python menjalankan query aman;
- hasil ditampilkan ke user.

Catatan kritis: LLM tidak boleh menulis SQL bebas. LLM hanya boleh memilih aksi yang dibatasi schema.

Pilihan 6 — Sistem RAG Sederhana dengan Metadata JSON

Input:

- dokumen pendek;
- catatan kuliah;
- artikel kampus.

Output metadata JSON:

- `document_id`
- `judul`
- `topik`
- `sumber`
- `tanggal`
- `ringkasan`
- `tags`

Fitur demo:

- simpan metadata;
- simpan chunk;
- query berdasarkan tag;
- tampilkan ringkasan dokumen.

Pilihan 7 — Sistem Klasifikasi Keluhan Mahasiswa ke Database

Input:

Wi-Fi lab sering putus saat praktikum dan dosen sulit mengakses materi.

Output JSON:

- `kategori`
- `prioritas`
- `unit_terkait`
- `ringkasan`
- `sentimen`
- `rekomendasi`

Fitur demo:

- klasifikasi keluhan;
- prioritas otomatis;
- laporan keluhan per kategori;
- error handling untuk input kosong.

Pilihan 8 — Sistem Laporan Otomatis dari Data JSON

Input:

- data JSON hasil ekstraksi;
- tabel output LLM;
- data berita atau form.

Output:

- ringkasan laporan;
- statistik jumlah valid/gagal;
- top kategori;
- top lokasi;
- top isu.

Fitur demo:

- query database;
- buat ringkasan otomatis;
- tampilkan laporan harian/mingguan.

10.3 Komponen Wajib Proyek

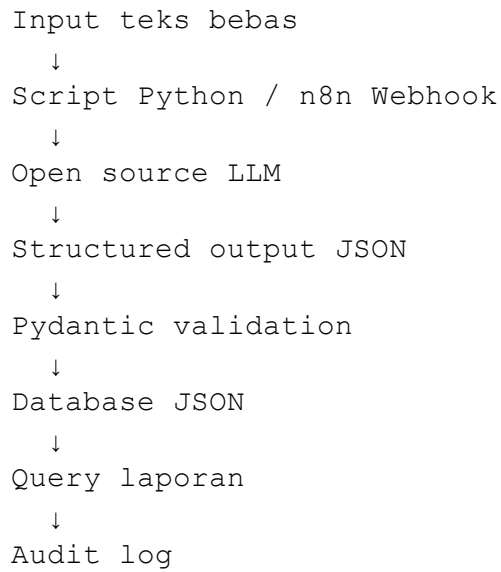
Setiap proyek wajib memiliki komponen berikut.

Komponen	Syarat Minimal
JSON Schema	Minimal 1 schema untuk output utama
Model Pydantic	Minimal 1 class validasi
Open source LLM	Minimal 1 model lokal / server lokal
Database	Minimal 1 tabel dengan kolom JSON
Workflow / Script	Minimal 1 workflow n8n atau script Python end-to-end
Logging	Menyimpan input, output, model, schema, status
Error Handling	Menangani output invalid
Dokumentasi	Cara instalasi dan cara menjalankan
Screenshot	Bukti input, output, validasi, database
Evaluasi	Menghitung kualitas output sederhana

n8n dapat dipakai karena menyediakan Structured Output Parser untuk mengembalikan field berdasarkan JSON Schema. Dokumentasinya juga menekankan bahwa parser ini dipakai untuk menstrukturkan output akhir AI Agent, bukan seluruh langkah perantara. ([n8n Docs](#))

10.4 Arsitektur Minimal Proyek

Gunakan arsitektur sederhana:



Untuk proyek kecil, script Python sudah cukup. Untuk demo visual, n8n dapat membantu.

Arsitektur hybrid lebih kuat:

- n8n = orkestrasi
- Python = validasi dan logika
- LLM = ekstraksi
- Database = penyimpanan
- JSON Schema = kontrak output

10.5 Struktur Folder Proyek Akhir

Gunakan struktur sederhana agar mudah dinilai.

```
final_project/
├── README.md
├── requirements.txt
├── data/
│   ├── sample_input.txt
│   └── sample_output.json
├── schemas/
│   └── output.schema.json
├── src/
│   ├── config.py
│   ├── models.py
│   ├── llm_client.py
│   ├── validator.py
│   ├── database.py
│   └── main.py
├── sql/
│   ├── create_tables.sql
│   └── sample_queries.sql
├── logs/
│   ├── valid/
│   └── error/
└── screenshots/
```

Isi penting:

- `README.md` menjelaskan proyek.
- `requirements.txt` daftar library Python.
- `schemas/` menyimpan JSON Schema.
- `models.py` menyimpan Pydantic model.
- `main.py` menjalankan pipeline.
- `sql/` menyimpan struktur tabel dan query.
- `logs/` menyimpan output valid dan error.
- `screenshots/` menyimpan bukti demo.

10.6 Template Model Pydantic

Contoh untuk proyek berita:

```
from pydantic import BaseModel, ConfigDict, Field

class BeritaOutput(BaseModel):
    model_config = ConfigDict(extra="forbid")

    judul: str = Field(min_length=5, max_length=150)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    aktor: list[str] = Field(min_length=1)
    organisasi: list[str] = Field(min_length=1)
    lokasi: str = Field(min_length=2, max_length=100)
    isu: list[str] = Field(min_length=1, max_length=10)
    ringkasan: str = Field(min_length=20, max_length=300)
```

Penjelasan:

- `BaseModel` menjadi dasar model.
- `extra="forbid"` menolak field tambahan.
- `Field` memberi constraint.
- `pattern` memaksa tanggal format `YYYY-MM-DD`.
- `list[str]` membuat field array string.
- `model_json_schema()` dapat menghasilkan JSON Schema dari model Pydantic. ([Pydantic](#))

10.7 Template Tabel Database

```
CREATE TABLE ai_outputs (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    project_name VARCHAR(100) NOT NULL,  
    source_text LONGTEXT NOT NULL,  
    schema_version VARCHAR(50) NOT NULL,  
    llm_model VARCHAR(100) NOT NULL,  
    json_output JSON NOT NULL,  
    validation_status VARCHAR(30) NOT NULL,  
    validation_error JSON,  
    latency_ms INT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tambahkan generated column untuk field penting jika sering dicari.

Contoh lokasi:

```
ALTER TABLE ai_outputs  
ADD COLUMN lokasi VARCHAR(100)  
GENERATED ALWAYS AS (  
    JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.lokasi'))  
) STORED;  
  
CREATE INDEX idx_ai_outputs_lokasi  
ON ai_outputs (lokasi);
```

10.8 Template Pipeline Python End-to-End

```
import json
import time
from pathlib import Path
from pydantic import ValidationError

from models import BeritaOutput
from llm_client import panggil_llm
from database import simpan_output

SCHEMA_VERSION = "berita_v1"
LLM_MODEL = "llama3.2:3b"

def validasi_output(raw_output: str):
    try:
        data = BeritaOutput.model_validate_json(raw_output)
        return True, data.model_dump(), []
    except ValidationError as error:
        return False, None, error.errors()

def main():
    source_text =
    Path("data/sample_input.txt").read_text(encoding="utf-8")

    prompt = f"""
    Ekstrak berita berikut menjadi JSON sesuai schema.
    Jawab hanya JSON. Jangan tambah field.

    Berita:
    {source_text}
    """

    start = time.time()
    raw_output = panggil_llm(prompt)
    latency_ms = int((time.time() - start) * 1000)

    valid, data, errors = validasi_output(raw_output)

    if valid:
        simpan_output(
            project_name="ekstraksi_berita",
            source_text=source_text,
            schema_version=SCHEMA_VERSION,
            llm_model=LLM_MODEL,
            json_output=data,
            validation_status="valid",
            validation_error=None,
            latency_ms=latency_ms
        )

    Path("logs/valid").mkdir(parents=True, exist_ok=True)
```

```

        Path("logs/valid/output.json").write_text(
            json.dumps(data, indent=2, ensure_ascii=False),
            encoding="utf-8"
        )

        print("VALID: data berhasil disimpan.")

    else:
        error_payload = {
            "raw_output": raw_output,
            "errors": errors
        }

        simpan_output(
            project_name="ekstraksi_berita",
            source_text=source_text,
            schema_version=SCHEMA_VERSION,
            llm_model=LLM_MODEL,
            json_output={},
            validation_status="invalid",
            validation_error=error_payload,
            latency_ms=latency_ms
        )

        Path("logs/error").mkdir(parents=True, exist_ok=True)
        Path("logs/error/error.json").write_text(
            json.dumps(error_payload, indent=2, ensure_ascii=False),
            encoding="utf-8"
        )

        print("INVALID: output gagal validasi.")

if __name__ == "__main__":
    main()

```

Penjelasan singkat

- `source_text` membaca input.
- `panggil_llm()` memanggil model.
- `model_validate_json()` memvalidasi output.
- Data valid disimpan ke database.
- Data invalid masuk log error.
- `latency_ms` mencatat waktu proses.
- Sistem tidak menyimpan output gagal sebagai data final.

10.9 Evaluasi Kualitas Output

Setiap proyek wajib memiliki evaluasi sederhana.

Minimal hitung:

Metrik	Rumus
Valid JSON Rate	jumlah JSON valid / total output
Schema Compliance Rate	jumlah lolos schema / total output
Error Rate	jumlah gagal / total output
Average Latency	total latency / jumlah output
Extraction Accuracy	jumlah ekstraksi benar / jumlah target manual

Untuk NER, gunakan:

```
precision = true_positive / (true_positive + false_positive)
recall = true_positive / (true_positive + false_negative)
```

Contoh tabel evaluasi:

Percobaan	JSON Valid	Schema Valid	Latency ms	Catatan
1	Ya	Ya	2100	Baik
2	Ya	Tidak	2300	Field hilang
3	Tidak	Tidak	1900	JSON rusak

10.10 Dokumentasi Demo

README.md minimal berisi:

- judul proyek;
- tujuan;
- arsitektur;
- cara instalasi;
- cara menjalankan;
- contoh input;
- contoh output;
- struktur database;
- screenshot hasil;
- hasil evaluasi;
- keterbatasan;
- rencana pengembangan.

Contoh bagian singkat:

```
# Sistem Ekstraksi Berita Indonesia ke Database

## Tujuan
Mengubah artikel berita Indonesia menjadi JSON terstruktur,
memvalidasi output dengan Pydantic, lalu menyimpan hasil ke
database JSON.

## Komponen
- Python
- Pydantic
- Ollama
- JSON Schema
- Database JSON
- n8n atau script Python

## Cara Menjalankan
1. Buat virtual environment.
2. Install dependency.
3. Jalankan database.
4. Jalankan model lokal.
5. Jalankan pipeline.
6. Cek hasil di database.
```


10.11 Screenshot yang Wajib Ada

Sertakan screenshot:

- struktur folder proyek;
- contoh input teks;
- output JSON dari LLM;
- hasil validasi sukses;
- hasil validasi gagal;
- isi tabel database;
- query laporan;
- dashboard sederhana atau output terminal;
- workflow n8n jika dipakai.

Screenshot bukan sekadar hiasan. Screenshot adalah **bukti sistem berjalan**.

10.12 Rubrik Penilaian

Komponen	Bobot
Desain JSON Schema	15%
Validasi Python / Pydantic	15%
Integrasi open source LLM	15%
Integrasi database	20%
Workflow n8n / otomasi	15%
Error handling dan audit	10%
Dokumentasi dan demo	10%
Total	100%

Detail Penilaian

Desain JSON Schema — 15%

Dinilai dari:

- field jelas;
- tipe data tepat;
- **required** benar;
- constraint dipakai;
- field tambahan ditolak;
- schema tidak terlalu kompleks.

Validasi Python / Pydantic — 15%

Dinilai dari:

- minimal 1 model Pydantic;
- validasi output LLM;
- error ditangani;
- data invalid tidak masuk tabel utama;
- model dapat menghasilkan JSON Schema.

Integrasi open source LLM — 15%

Dinilai dari:

- model lokal berjalan;
- prompt jelas;
- output terstruktur;
- temperature/parameter dicatat;
- schema dipakai jika memungkinkan.

Integrasi Database — 20%

Dinilai dari:

- tabel dibuat rapi;
- ada kolom JSON;
- data berhasil insert;
- ada query laporan;
- ada audit field seperti model, schema, waktu, status.

Workflow n8n / Otomasi — 15%

Dinilai dari:

- ada script end-to-end atau workflow n8n;
- alur input sampai database berjalan;
- error branch tersedia;
- retry atau fallback tersedia.

Error Handling dan Audit — 10%

Dinilai dari:

- raw output disimpan;
- error disimpan;
- schema version dicatat;
- model dicatat;
- latency dicatat;
- data sensitif dipertimbangkan.

Dokumentasi dan Demo — 10%

Dinilai dari:

- README jelas;
- screenshot lengkap;
- cara instalasi tersedia;
- cara menjalankan tersedia;
- keterbatasan dijelaskan.

10.13 Checklist Sebelum Demo

Pertanyaan	Ya/Tidak
Apakah proyek bisa dijalankan dari awal?	
Apakah input contoh tersedia?	
Apakah JSON Schema tersedia?	
Apakah Pydantic model tersedia?	
Apakah LLM lokal bisa dipanggil?	
Apakah output valid masuk database?	
Apakah output invalid ditolak?	
Apakah error disimpan?	
Apakah query laporan tersedia?	
Apakah screenshot lengkap?	
Apakah README jelas?	
Apakah hasil evaluasi tersedia?	

10.14 Pola Presentasi 5 Menit

Gunakan alur ini:

1. **Masalah**
“Teks bebas sulit langsung masuk database.”
2. **Solusi**
“Saya memakai LLM untuk ekstraksi, JSON Schema untuk kontrak, Pydantic untuk validasi, dan database untuk penyimpanan.”
3. **Demo input**
Tampilkan teks mentah.
4. **Demo output**
Tampilkan JSON hasil LLM.
5. **Demo validasi**
Tampilkan output valid dan contoh output gagal.
6. **Demo database**
Tampilkan data tersimpan dan query laporan.
7. **Evaluasi**
Tampilkan valid JSON rate, schema compliance rate, dan error terbanyak.
8. **Kesimpulan**
“Sistem ini tidak hanya menjawab, tetapi menghasilkan data terstruktur yang dapat diaudit.”

10.15 Ringkasan Bab

Bab terakhir ini menutup buku dengan proyek nyata.

Inti Bab 10:

- Proyek akhir harus end-to-end, bukan potongan kode terpisah.
- Minimal ada JSON Schema, Pydantic model, LLM lokal, database JSON, workflow/script, logging, dan error handling.
- Pilihan proyek dapat berupa ekstraksi berita, NER, form intelligence, invoice, chatbot database, RAG metadata, klasifikasi keluhan, atau laporan otomatis.
- LLM tidak boleh dipercaya langsung; output harus divalidasi.
- Database membuat output AI dapat dicari, dihitung, dan diaudit.
- Dokumentasi dan screenshot membuktikan sistem berjalan.
- Evaluasi kualitas output menunjukkan kematangan proyek.

Output bab ini: mahasiswa keluar dari buku dengan sistem nyata yang dapat dijadikan tugas akhir mini, portofolio, atau fondasi riset lanjutan.

Lampiran A — Cheat Sheet JSON Schema

Lampiran ini menempatkan *cheat sheet* JSON Schema, template Pydantic, template tabel database JSON, query SQL, workflow n8n, dan 30 prompt sebagai lampiran utama. JSON Schema dipakai sebagai **kontrak struktur data**: ia mendefinisikan field, tipe data, field wajib, constraint, dan aturan tambahan agar output LLM dapat divalidasi sebelum masuk database. JSON Schema juga menyediakan keyword seperti **properties** dan **additionalProperties** untuk mengatur field object dan mencegah field liar. ([JSON Schema](#))

A.1 Keyword Penting JSON Schema

Keyword	Fungsi	Contoh Singkat
type	Menentukan tipe data	"type": "object"
properties	Mendefinisikan field dalam object	"properties": {"nama": {"type": "string"}}
required	Menentukan field wajib	"required": ["nama", "nim"]
items	Menentukan tipe item dalam array	"items": {"type": "string"}
enum	Membatasi nilai ke daftar tertentu	"enum": ["Informatika", "Sistem Informasi"]
pattern	Membatasi string dengan pola regex	"pattern": "^\\d{4}-\\d{2}-\\d{2}\$"
minimum	Nilai angka minimum	"minimum": 2020
maximum	Nilai angka maksimum	"maximum": 2026
additionalProperties	Mengizinkan / menolak field tambahan	"additionalProperties": false

Prinsip cepat: gunakan **required** untuk field penting, **enum** untuk kategori resmi, **pattern** untuk format seperti tanggal, **minimum** dan **maximum** untuk angka, serta **additionalProperties: false** agar LLM tidak menambah field yang tidak diminta.

A.2 Contoh Schema Mahasiswa

```
{
  "type": "object",
  "properties": {
    "nama": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "nim": {
      "type": "string",
      "minLength": 6,
      "maxLength": 30
    },
    "prodi": {
      "type": "string",
      "enum": ["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    },
    "email": {
      "type": "string",
      "pattern": "^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]+$"
    },
    "tahun_masuk": {
      "type": "integer",
      "minimum": 2020,
      "maximum": 2026
    },
    "minat_riset": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "maxItems": 5
    }
  },
  "required": ["nama", "nim", "prodi", "email",
"tahun_masuk", "minat_riset"],
  "additionalProperties": false
}
```

A.3 Contoh Schema Berita

```
{
  "type": "object",
  "properties": {
    "judul": {
      "type": "string",
      "minLength": 5,
      "maxLength": 150
    },
    "tanggal": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
    },
    "aktor": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1
    },
    "organisasi": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "lokasi": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "isu": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "maxItems": 10
    },
    "ringkasan": {
      "type": "string",
      "minLength": 20,
      "maxLength": 300
    }
  },
  "required": ["judul", "tanggal", "aktor", "organisasi", "lokasi",
    "isu", "ringkasan"],
  "additionalProperties": false
}
```

A.4 Contoh Schema Invoice

```
{
  "type": "object",
  "properties": {
    "nomor_invoice": {
      "type": "string",
      "minLength": 3,
      "maxLength": 50
    },
    "tanggal": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
    },
    "vendor": {
      "type": "string",
      "minLength": 2,
      "maxLength": 150
    },
    "item": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "properties": {
          "nama_item": {
            "type": "string"
          },
          "jumlah": {
            "type": "integer",
            "minimum": 1
          },
          "harga_satuan": {
            "type": "number",
            "minimum": 0
          },
          "subtotal": {
            "type": "number",
            "minimum": 0
          }
        },
        "required": ["nama_item", "jumlah", "harga_satuan", "subtotal"],
        "additionalProperties": false
      }
    },
    "total": {
      "type": "number",
      "minimum": 0
    },
    "pajak": {
      "type": "number",
      "minimum": 0
    }
  },
  "required": ["nomor_invoice", "tanggal", "vendor", "item", "total", "pajak"],
  "additionalProperties": false
}
```


A.5 Contoh Schema NER

```
{
  "type": "object",
  "properties": {
    "entities": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "text": {
            "type": "string",
            "minLength": 1
          },
          "label": {
            "type": "string",
            "enum": ["person", "organization", "location",
"date", "topic"]
          },
          "confidence": {
            "type": "number",
            "minimum": 0,
            "maximum": 1
          }
        },
        "required": ["text", "label", "confidence"],
        "additionalProperties": false
      }
    },
    "required": ["entities"],
    "additionalProperties": false
  }
}
```

Lampiran B — Template Pydantic Model

Pydantic dipakai sebagai **lapisan validasi Python**. Pydantic dapat memvalidasi data, mengembalikan `ValidationError` jika data tidak sesuai, dan menghasilkan JSON Schema dari model melalui `model_json_schema()`. ([Pydantic](#)) Ini menempatkan Pydantic sebagai lapisan validasi antara LLM dan database.

B.1 Model Mahasiswa

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, EmailStr, Field

class Mahasiswa(BaseModel):
    model_config = ConfigDict(extra="forbid")

    nama: str = Field(min_length=2, max_length=100)
    nim: str = Field(min_length=6, max_length=30)
    prodi: Literal["Informatika", "Sistem Informasi", "Teknik
Elektro"]
    email: EmailStr
    tahun_masuk: int = Field(ge=2020, le=2026)
    minat_riset: list[str] = Field(min_length=1,
max_length=5)
```

B.2 Model Berita

```
from pydantic import BaseModel, ConfigDict, Field

class Berita(BaseModel):
    model_config = ConfigDict(extra="forbid")

    judul: str = Field(min_length=5, max_length=150)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    aktor: list[str] = Field(min_length=1)
    organisasi: list[str] = Field(default_factory=list)
    lokasi: str = Field(min_length=2, max_length=100)
    isu: list[str] = Field(min_length=1, max_length=10)
    ringkasan: str = Field(min_length=20, max_length=300)
```

B.3 Model Entitas

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class Entity(BaseModel):
    model_config = ConfigDict(extra="forbid")
    text: str = Field(min_length=1, max_length=255)
    label: Literal["person", "organization", "location", "date", "topic"]
    confidence: float = Field(ge=0, le=1)

class NEROutput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    entities: list[Entity] = Field(default_factory=list)
```

B.4 Model Invoice

```
from pydantic import BaseModel, ConfigDict, Field,
model_validator

class InvoiceItem(BaseModel):
    model_config = ConfigDict(extra="forbid")
    nama_item: str = Field(min_length=1, max_length=150)
    jumlah: int = Field(ge=1)
    harga_satuan: float = Field(ge=0)
    subtotal: float = Field(ge=0)

class Invoice(BaseModel):
    model_config = ConfigDict(extra="forbid")
    nomor_invoice: str = Field(min_length=3, max_length=50)
    tanggal: str = Field(pattern=r"^\d{4}-\d{2}-\d{2}$")
    vendor: str = Field(min_length=2, max_length=150)
    item: list[InvoiceItem] = Field(min_length=1)
    total: float = Field(ge=0)
    pajak: float = Field(ge=0)

    @model_validator(mode="after")
    def cek_total(self):
        total_item = sum(i.subtotal for i in self.item)
        if self.total < total_item:
            raise ValueError("total tidak boleh lebih kecil
dari jumlah subtotal item")
        return self
```

B.5 Model Intent Chatbot

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class ChatbotIntent(BaseModel):
    model_config = ConfigDict(extra="forbid")

    action: Literal[
        "create_record",
        "update_record",
        "search_record",
        "summarize_record"
    ]

    target_table: Literal[
        "berita_kampus",
        "mahasiswa_forms",
        "invoices",
        "ner_entities"
    ]

    filters: dict = Field(default_factory=dict)
    payload: dict = Field(default_factory=dict)
    reason: str = Field(max_length=300)
```

B.6 Contoh Validasi Error

```
from pydantic import ValidationError

data_salah = {
    "nama": "S",
    "nim": "123",
    "prodi": "AI Engineering",
    "email": "bukan-email",
    "tahun_masuk": 2010,
    "minat_riset": []
}

try:
    mahasiswa = Mahasiswa.model_validate(data_salah)
    print("VALID:", mahasiswa.model_dump())
except ValidationError as error:
    print("INVALID")
    print(error.errors())
```

Makna kode:

- `model_validate()` memvalidasi dictionary Python.
- `ValidationError` menangkap semua error validasi.
- `error.errors()` menghasilkan error terstruktur yang bisa disimpan ke log.

Lampiran C — Template Tabel Database JSON

Bagian ini mengikuti arah Bab 6 dan lampiran buku: database menjadi pusat pipeline, bukan hanya tempat menyimpan output mentah. Template tabel mencakup `llm_outputs`, `entities`, `documents`, `validation_logs`, dan `schema_versions`. MySQL mendukung fungsi pencarian JSON seperti `JSON_CONTAINS()` dan ekstraksi JSON; referensi *MySQL & JSON* juga menekankan validasi JSON Schema dapat dipakai sebelum dokumen diterima database. ([MySQL Developer Zone](#))

C.1 Tabel llm_outputs

```
CREATE TABLE llm_outputs (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  case_name VARCHAR(100) NOT NULL,  
  source_text LONGTEXT NOT NULL,  
  source_hash CHAR(64),  
  schema_version VARCHAR(50) NOT NULL,  
  llm_model VARCHAR(100) NOT NULL,  
  prompt_version VARCHAR(50),  
  json_output JSON NOT NULL,  
  validation_status VARCHAR(30) NOT NULL,  
  validation_error JSON,  
  latency_ms INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

C.2 Tabel documents

```
CREATE TABLE documents (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(200),  
  source_text LONGTEXT NOT NULL,  
  source_hash CHAR(64),  
  document_type VARCHAR(50),  
  schema_version VARCHAR(50),  
  llm_model VARCHAR(100),  
  metadata JSON,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE KEY uniq_source_hash (source_hash)  
);
```

C.3 Tabel entities

```
CREATE TABLE entities (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    document_id BIGINT NOT NULL,  
    entity_text VARCHAR(255) NOT NULL,  
    entity_label VARCHAR(50) NOT NULL,  
    confidence DECIMAL(5,4),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (document_id) REFERENCES documents(id),  
    INDEX idx_entity_label (entity_label),  
    INDEX idx_entity_text (entity_text)  
);
```

C.4 Tabel validation_logs

```
CREATE TABLE validation_logs (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    llm_output_id BIGINT,  
    validation_status VARCHAR(30) NOT NULL,  
    error_type VARCHAR(100),  
    error_detail JSON,  
    raw_output JSON,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (llm_output_id) REFERENCES llm_outputs(id)  
);
```

C.5 Tabel schema_versions

```
CREATE TABLE schema_versions (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    schema_name VARCHAR(100) NOT NULL,  
    schema_version VARCHAR(50) NOT NULL,  
    schema_body JSON NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE KEY uniq_schema_version (schema_name,  
    schema_version)  
);
```

Lampiran D — Contoh Query SQL untuk JSON

D.1 Insert JSON

```
INSERT INTO llm_outputs (  
    case_name,  
    source_text,  
    schema_version,  
    llm_model,  
    prompt_version,  
    json_output,  
    validation_status  
) VALUES (  
    'ekstraksi_berita',  
    'Pada 12 Mei 2026, ITTS mengadakan seminar AI lokal di  
Tangerang Selatan.',  
    'berita_v1',  
    'llama3.2:3b',  
    'prompt_berita_v1',  
    '{  
        "judul": "Seminar AI Lokal",  
        "tanggal": "2026-05-12",  
        "lokasi": "Tangerang Selatan",  
        "aktor": ["ITTS"],  
        "topik": ["AI lokal"],  
        "ringkasan": "ITTS mengadakan seminar AI lokal di  
Tangerang Selatan."  
    }',  
    'valid'  
);
```

D.2 Extract Field

```
SELECT  
    id,  
    json_output->>'$.judul' AS judul,  
    json_output->>'$.tanggal' AS tanggal,  
    json_output->>'$.lokasi' AS lokasi  
FROM llm_outputs  
WHERE case_name = 'ekstraksi_berita';
```

D.3 Filter Nested Field / Array

```
SELECT
    id,
    json_output->>'$.judul' AS judul,
    json_output->'$.topik' AS topik
FROM llm_outputs
WHERE JSON_CONTAINS(
    json_output->'$.topik',
    JSON_QUOTE('AI lokal')
);
```

D.4 Generated Column

```
ALTER TABLE llm_outputs
ADD COLUMN lokasi VARCHAR(100)
GENERATED ALWAYS AS (
    JSON_UNQUOTE(JSON_EXTRACT(json_output, '$.lokasi'))
) STORED;
```

D.5 Indexing Generated Column

```
CREATE INDEX idx_llm_outputs_lokasi
ON llm_outputs (lokasi);
```

Generated column berguna agar field JSON yang sering difilter dapat dibuat lebih mudah dicari. Praktik ini selaras dengan pola indexing JSON melalui kolom turunan / generated column.

D.6 Agregasi Entitas

```
SELECT
    entity_text,
    entity_label,
    COUNT(*) AS jumlah
FROM entities
GROUP BY entity_text, entity_label
ORDER BY jumlah DESC
LIMIT 20;
```


D.7 Agregasi Topik dari JSON Array

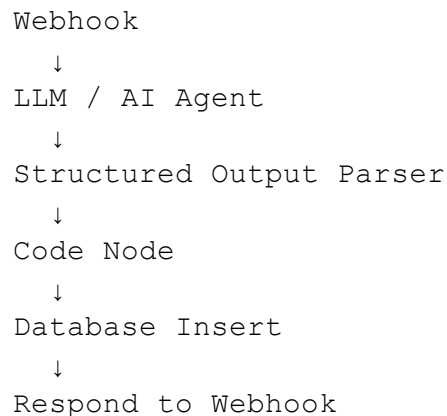
```
SELECT
    t.topik,
    COUNT(*) AS jumlah
FROM llm_outputs AS l,
JSON_TABLE(
    l.json_output,
    '$.topik[*]'
    COLUMNS (
        topik VARCHAR(100) PATH '$'
    )
) AS t
GROUP BY t.topik
ORDER BY jumlah DESC;
```

`JSON_TABLE()` mengubah data JSON menjadi format tabel agar bisa diproses dengan SQL biasa. ([MySQL Developer Zone](#))

Lampiran E — Template Workflow n8n

n8n menyediakan Structured Output Parser node untuk mengembalikan field berdasarkan JSON Schema. ([n8n Docs](#)) Dalam buku ini, n8n dipakai sebagai **orquestrator**, sedangkan Python tetap dipakai untuk validasi kuat dan logika kompleks.

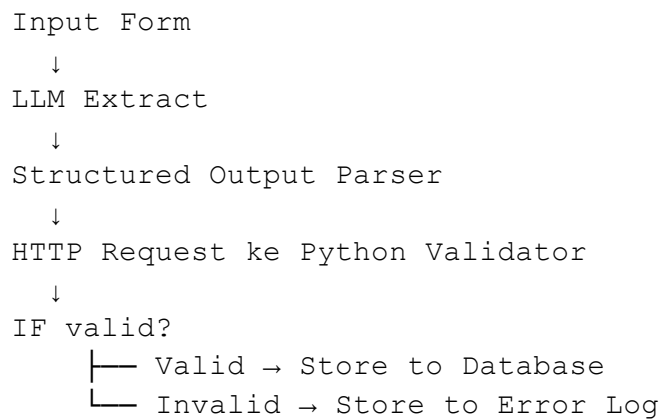
E.1 Workflow 1 — Webhook → LLM → Structured Output Parser → Database



Fungsi:

- **Webhook**: menerima input teks.
- **LLM / AI Agent**: mengekstrak data.
- **Structured Output Parser**: memaksa output akhir mengikuti JSON Schema.
- **Code Node**: menambah metadata seperti `schema_version`, `llm_model`, `validation_status`.
- **Database Insert**: menyimpan data valid.
- **Respond to Webhook**: memberi respons sukses/gagal.

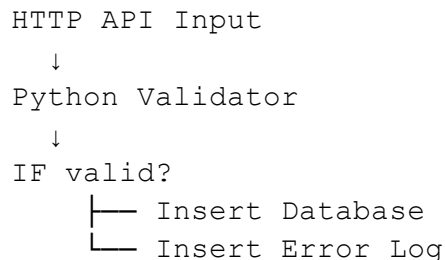
E.2 Workflow 2 — Email / Form Input → Extract → Validate → Store



Prinsip:

- input bebas boleh tidak rapi;
- output LLM wajib mengikuti schema;
- validasi final tetap dilakukan dengan Python;
- data invalid tidak boleh masuk tabel utama.

E.3 Workflow 3 — HTTP API → Python Validator → Database



Cocok untuk:

- sistem yang sudah menghasilkan JSON;
- validasi cepat sebelum database;
- workflow yang ingin memisahkan n8n dan logic Python.

E.4 Error Branch dan Retry

```
LLM / Parser gagal
↓
Retry maksimal 3 kali
↓
Jika masih gagal
↓
Simpan raw_output + error
↓
Status = invalid
↓
Response gagal
```

Aturan:

- retry maksimal 2–3 kali;
- simpan semua error;
- jangan simpan output gagal sebagai data final;
- jangan mengubah data diam-diam tanpa log.

E.5 Template Data Log Workflow

```
{
  "workflow_name": "form_mahasiswa_extraction",
  "schema_version": "mahasiswa_v1",
  "llm_model": "llama3.2:3b",
  "source_text": "Nama saya Siti Aulia...",
  "raw_output": {},
  "validated_output": {},
  "validation_status": "valid",
  "validation_error": null,
  "latency_ms": 1850
}
```

Lampiran F — 30 Prompt Structured Output untuk Mahasiswa

Gunakan prompt ini sebagai latihan. Selalu dikombinasikan dengan **JSON Schema**, **Pydantic validation**, dan **logging error**. Disini menekankan bahwa *structured output* bukan sekadar meminta LLM menulis JSON, melainkan membuat output stabil untuk sistem nyata.

F.1 Ekstraksi Biodata

1. Biodata mahasiswa dasar

Ekstrak biodata mahasiswa dari teks berikut menjadi JSON.

Field wajib: nama, nim, prodi, email, tahun_masuk, minat_riset.

Jawab hanya JSON. Jangan menambah field.

Teks:

{teks}

2. Biodata dengan minat riset

Ubah teks biodata berikut menjadi JSON sesuai schema mahasiswa.

minat_riset harus array string.

tahun_masuk harus integer.

Jawab hanya JSON.

Teks:

{teks}

3. Validasi data pendaftaran

Ekstrak data pendaftaran mahasiswa.

Jika informasi tidak tersedia, gunakan null hanya untuk field opsional.

Jangan mengarang data.

Jawab hanya JSON.

Teks:

{teks}

F.2 Ekstraksi Berita

4. Berita kampus

Ekstrak berita kampus menjadi JSON.

Field: judul, tanggal, aktor, organisasi, lokasi, isu, ringkasan.

Tanggal wajib format YYYY-MM-DD.

Jawab hanya JSON.

Berita:

{teks}

5. Berita dengan aktor dan organisasi

Ambil aktor dan organisasi dari berita berikut.

aktor dan organisasi harus array.

Jangan menambah field di luar schema.

Jawab hanya JSON.

Berita:

{teks}

6. Ringkasan berita terstruktur

Buat ringkasan terstruktur dari berita berikut.

Output wajib JSON dengan field judul, tanggal, lokasi, topik, ringkasan.

ringkasan maksimal 300 karakter.

Jawab hanya JSON.

Berita:

{teks}

F.3 NER

7. NER dasar

Lakukan Named Entity Recognition dari teks berikut.
Output JSON berisi entities array.
Setiap entitas memiliki text, label, confidence.
Label hanya boleh: person, organization, location, date, topic.
Jawab hanya JSON.

Teks:
{teks}

8. NER berita Indonesia

Ekstrak entitas dari berita Indonesia berikut.
Jangan mengarang entitas yang tidak ada di teks.
Gunakan label person, organization, location, date, topic.
Jawab hanya JSON.

Teks:
{teks}

9. NER dokumen kampus

Ekstrak entitas penting dari dokumen kampus berikut.
Output JSON mengikuti schema NER.
Jika tidak ada entitas, gunakan array kosong.
Jawab hanya JSON.

Dokumen:
{teks}

F.4 Invoice

10. Invoice sederhana

Ekstrak invoice berikut menjadi JSON.

Field: nomor_invoice, tanggal, vendor, item, total, pajak.

item harus array berisi nama_item, jumlah, harga_satuan, subtotal.

Jawab hanya JSON.

Teks invoice:

{teks}

11. Kwitansi sederhana

Ubah teks kwitansi berikut menjadi JSON terstruktur.

Validasi angka: total, pajak, harga_satuan, subtotal harus number.

Tanggal format YYYY-MM-DD.

Jawab hanya JSON.

Kwitansi:

{teks}

12. Invoice hasil OCR

Teks berikut berasal dari OCR dan mungkin berantakan.

Ekstrak informasi invoice hanya jika terlihat jelas.

Jangan mengarang vendor atau total.

Jawab hanya JSON sesuai schema.

OCR:

{teks}

F.5 Klasifikasi

13. Klasifikasi keluhan mahasiswa

Klasifikasikan keluhan mahasiswa berikut menjadi JSON.

Field: kategori, prioritas, unit_terkait, ringkasan, rekomendasi.

kategori hanya boleh: akademik, fasilitas, jaringan, administrasi, lainnya.

Jawab hanya JSON.

Keluhan:

{teks}

14. Klasifikasi berita

Klasifikasikan berita berikut.

Output JSON: kategori, topik, sentimen, ringkasan.

kategori hanya boleh: akademik, teknologi, riset, kegiatan, lainnya.

Jawab hanya JSON.

Berita:

{teks}

15. Klasifikasi urgensi

Tentukan urgensi teks berikut.

Output JSON: prioritas, alasan, tindakan_disarankan.

prioritas hanya boleh: rendah, sedang, tinggi.

Jawab hanya JSON.

Teks:

{teks}

F.6 Form Intelligence

16. Form bebas ke data mahasiswa

Input berikut adalah jawaban form bebas.

Ekstrak menjadi JSON mahasiswa.

Field wajib: nama, nim, prodi, email, tahun_masuk, minat_riset.

Jawab hanya JSON.

Input:

{teks}

17. Form event kampus

Ekstrak data event dari form bebas berikut.

Field: nama_event, tanggal, lokasi, penyelenggara, narahubung, topik.

Jawab hanya JSON.

Input:

{teks}

18. Form feedback kuliah

Ekstrak feedback kuliah menjadi JSON.

Field: mata_kuliah, dosen, sentimen, isu_utama, rekomendasi.

sentimen hanya boleh: positif, netral, negatif.

Jawab hanya JSON.

Feedback:

{teks}

F.7 Database Intent

19. Search record

Ubah permintaan user menjadi JSON intent.

action hanya boleh: search_record.

target_table hanya boleh: berita_kampus, mahasiswa_forms, invoices.

Jangan menulis SQL.

Jawab hanya JSON.

Permintaan:

{teks}

20. Summarize record

Ubah permintaan berikut menjadi JSON intent untuk meringkas data.

action: summarize_record.

target_table harus dipilih dari schema.

filters harus object.

Jangan menulis SQL.

Jawab hanya JSON.

Permintaan:

{teks}

21. Create record

Ubah input berikut menjadi JSON intent create_record.

target_table harus sesuai data.

payload berisi data yang akan dibuat.

Jangan menulis SQL.

Jawab hanya JSON.

Input:

{teks}

F.8 Validasi JSON

22. Cek field hilang

Periksa JSON berikut terhadap schema ringkas yang diberikan.
Laporkan field wajib yang hilang dalam JSON.
Output harus JSON dengan field valid, missing_fields, notes.

Schema:
{schema}

JSON:
{json}

23. Cek tipe data

Periksa apakah tipe data JSON berikut sesuai schema.
Output JSON: valid, type_errors, notes.
Jangan memperbaiki data, hanya laporkan.

Schema:
{schema}

JSON:
{json}

24. Perbaiki JSON rusak

Perbaiki JSON berikut agar valid secara sintaks.
Jangan menambah informasi baru.
Jawab hanya JSON hasil perbaikan.

JSON rusak:
{json}

F.9 Audit Output

25. Audit ekstraksi berita

Bandingkan teks sumber dan output JSON.

Cari apakah ada field yang dilarang, hilang, atau tidak didukung teks.

Output JSON: supported_fields, unsupported_fields, missing_information, audit_notes.

Teks sumber:

{teks}

Output JSON:

{json}

26. Audit NER

Audit hasil NER berikut terhadap teks sumber.

Laporkan false_positive dan possible_false_negative.

Jawab hanya JSON.

Teks:

{teks}

NER:

{json}

27. Audit invoice

Audit output invoice berikut.

Cek apakah subtotal dan total masuk akal.

Output JSON: valid, numeric_issues, audit_notes.

Invoice text:

{teks}

Output JSON:

{json}

F.10 Debugging Schema

28. Sederhanakan schema

Schema berikut terlalu kompleks untuk LLM pemula.

Sederhanakan schema tanpa menghilangkan field wajib utama.

Jawab hanya JSON Schema.

Schema:

```
{schema}
```

29. Cari field ambigu

Analisis schema berikut.

Cari nama field yang ambigu dan sarankan nama field lebih jelas.

Output JSON: `ambiguous_fields`, `suggestions`, `notes`.

Schema:

```
{schema}
```

30. Buat test case schema

Buat 5 test case untuk schema berikut:

1 valid case,

1 missing required field,

1 wrong type,

1 invalid enum,

1 extra field.

Jawab JSON array.

Schema:

```
{schema}
```

Ringkasan Lampiran

Lampiran ini memberi mahasiswa bahan siap pakai:

- **Lampiran A:** keyword dan contoh JSON Schema.
- **Lampiran B:** template Pydantic model.
- **Lampiran C:** template tabel database JSON.
- **Lampiran D:** query SQL untuk insert, extract, filter, generated column, indexing, dan agregasi.
- **Lampiran E:** template workflow n8n.
- **Lampiran F:** 30 prompt latihan *structured output*.

Dengan lampiran ini, mahasiswa dapat langsung membangun proyek akhir: dari teks bebas, menjadi JSON terstruktur, divalidasi Python, disimpan ke database, di otomasi workflow, dan diaudit sebagai sistem AI yang lebih reliable.

Glossary

Istilah	Definisi
Actor / Aktor	Entitas orang, lembaga, komunitas, atau pihak yang disebut dalam teks dan perlu diekstrak ke output terstruktur.
Additional Properties	Aturan JSON Schema untuk menentukan apakah field tambahan di luar schema boleh diterima atau harus ditolak. Dalam sistem LLM, biasanya disarankan <code>false</code> agar model tidak menambah field liar.
AI Agent	Sistem AI yang tidak hanya menjawab, tetapi dapat memilih aksi, memakai tool, membaca data, atau menjalankan workflow berdasarkan instruksi terstruktur.
API	<i>Application Programming Interface</i> , antarmuka agar satu aplikasi dapat berkomunikasi dengan aplikasi lain secara terstruktur.
Array	Struktur JSON berbentuk daftar nilai, ditulis dengan tanda <code>[]</code> , misalnya daftar topik, aktor, item invoice, atau minat riset.
Audit Log	Catatan proses yang menyimpan input, output, model, schema, parameter, waktu, status validasi, dan error agar sistem dapat ditelusuri ulang.
Audit Trail	Jejak lengkap perjalanan data dari input awal, prompt, output LLM, validasi, penyimpanan database, sampai hasil query.
BaseModel	Class utama Pydantic yang dipakai untuk membuat model data dan melakukan validasi otomatis terhadap input.

Boolean	Tipe data dengan dua nilai: <code>true</code> atau <code>false</code> . Cocok untuk status seperti aktif, valid, atau sudah diverifikasi.
Chatbot Database	Chatbot yang dapat memahami permintaan pengguna dan mengubahnya menjadi intent terstruktur untuk mencari, membuat, meringkas, atau memperbarui data.
Code Node	Node dalam n8n untuk menjalankan transformasi data ringan, seperti menambah metadata, mengubah format, atau menyiapkan payload ke database.
Constrained Generation	Teknik membatasi output LLM saat proses generasi agar mengikuti schema, grammar, atau struktur tertentu.
Database JSON	Database yang dapat menyimpan, mengekstrak, mencari, dan memproses data dalam format JSON.
Data Hygiene	Praktik menjaga kualitas data agar tidak ambigu, tidak rusak, tidak duplikat, dan dapat dipakai ulang. Buku referensi tentang data product menekankan risiko <i>ambiguity</i> , <i>knowledge gaps</i> , dan <i>blind spots</i> dalam pengelolaan data.
Determinisme	Tingkat kestabilan output LLM. Untuk ekstraksi data, parameter seperti <i>temperature</i> biasanya dibuat rendah agar output lebih konsisten.
Entity	Potongan informasi penting dalam teks, seperti nama orang, organisasi, lokasi, tanggal, atau topik.
Enum	Constraint JSON Schema atau Pydantic untuk membatasi nilai hanya pada daftar tertentu, misalnya prodi hanya boleh “Informatika”, “Sistem Informasi”, atau “Teknik Elektro”.

Error Branch	Jalur khusus dalam workflow untuk menangani output gagal, misalnya JSON rusak, field hilang, validasi gagal, atau database gagal insert.
Error Handling	Strategi menangani kegagalan sistem, termasuk retry, fallback, logging, dan penolakan data invalid.
Extraction Accuracy	Ukuran ketepatan ekstraksi informasi dari teks, biasanya dibandingkan dengan label manual atau <i>gold standard</i> .
Fail Fast	Prinsip agar sistem segera gagal ketika menemukan data rusak, daripada membiarkan data salah masuk database.
False Negative	Kondisi ketika sistem gagal menangkap entitas atau informasi yang seharusnya ada.
False Positive	Kondisi ketika sistem mengekstrak entitas atau informasi yang sebenarnya tidak ada atau salah kategori.
Fallback	Jalur cadangan jika output LLM gagal validasi, misalnya simpan ke error log, minta input diperbaiki, jalankan model lain, atau kirim ke manusia untuk review.
Field	Nama elemen data dalam JSON atau model, misalnya <code>nama</code> , <code>nim</code> , <code>tanggal</code> , <code>lokasi</code> , atau <code>total</code> .
Form Intelligence	Teknik mengubah jawaban form bahasa bebas menjadi data terstruktur yang bisa divalidasi dan disimpan ke database.
Generated Column	Kolom database yang nilainya dibuat otomatis dari ekspresi, misalnya mengambil <code>lokasi</code> dari field JSON agar bisa diindeks dan dicari lebih cepat.

Grammar	Aturan formal yang membatasi bentuk output LLM. Pada llama.cpp, grammar GBNF dapat digunakan untuk membatasi output agar mengikuti format tertentu.
Hallucinated Field	Field yang dibuat oleh LLM padahal tidak diminta dalam schema dan tidak didukung oleh input.
Human-in-the-Loop	Mekanisme melibatkan manusia untuk memeriksa data sensitif atau keputusan penting sebelum hasil AI dipakai sebagai keputusan final.
HTTP Request	Permintaan HTTP untuk mengirim atau mengambil data dari API, misalnya n8n memanggil LLM lokal atau service validator Python.
Indexing	Teknik database untuk mempercepat pencarian data, termasuk melalui generated column dari field JSON.
Input Text	Teks bebas yang menjadi sumber awal proses, seperti berita, form, invoice, dokumen kampus, atau keluhan mahasiswa.
Intent	Maksud terstruktur dari permintaan pengguna, misalnya <code>search_record</code> , <code>create_record</code> , <code>update_record</code> , atau <code>summarize_record</code> .
Invoice Extraction	Proses mengekstrak data invoice seperti nomor invoice, tanggal, vendor, item, total, dan pajak dari teks hasil OCR atau teks dokumen.
Items	Keyword JSON Schema untuk menentukan aturan isi array, misalnya setiap item dalam <code>minat_riset</code> harus bertipe string.
JSON	<i>JavaScript Object Notation</i> , format pertukaran data berbasis teks yang ringan, mudah dibaca manusia, dan mudah diproses program. Referensi MySQL & JSON menjelaskan JSON sebagai format yang

sederhana, mudah diparsing, dan banyak dipakai untuk pertukaran data antar sistem.

JSON Array	Struktur JSON berupa daftar, misalnya <code>["AI", "NLP", "Database"]</code> .
JSON Mode	Mode yang mengarahkan LLM agar menghasilkan JSON valid, tetapi belum tentu sesuai schema atau aturan bisnis.
JSON Object	Struktur JSON berbentuk pasangan key-value, ditulis dengan <code>{ }</code> .
JSON Schema	Bahasa deklaratif untuk mendeskripsikan struktur, tipe data, field wajib, dan aturan validasi dokumen JSON. Dalam buku ini, JSON Schema menjadi kontrak antara LLM, Python, n8n, dan database.
JSON_TABLE	Fungsi SQL untuk mengubah isi JSON menjadi bentuk tabel agar dapat di-query, di-join, dan diagregasi seperti data relasional.
LLM	<i>Large Language Model</i> , model AI yang dapat memproses dan menghasilkan teks, termasuk membuat output JSON jika diarahkan dengan schema.
llama.cpp	Tool open source untuk menjalankan model LLM, terutama model GGUF, dan mendukung grammar untuk membatasi output.
Logging	Proses mencatat semua informasi penting selama pipeline berjalan, seperti prompt, model, schema, output, error, latency, dan status validasi.
Machine-Readable	Format data yang dapat dibaca dan diproses langsung oleh mesin, bukan hanya dipahami manusia.

MariaDB	Database open source yang dapat dipakai untuk menyimpan dan memproses data JSON, dengan pendekatan dan detail teknis yang perlu diuji sesuai versi.
Metadata	Data tentang data, misalnya nama model, versi schema, waktu proses, sumber dokumen, atau status validasi.
Model Pydantic	Definisi struktur data dalam Python yang memakai <i>type hints</i> untuk validasi, serialisasi, dan pembuatan JSON Schema.
MySQL JSON	Fitur database MySQL untuk menyimpan dokumen JSON, melakukan validasi dokumen JSON, serta menyediakan fungsi pencarian dan ekstraksi nilai JSON. Referensi <i>MySQL & JSON</i> menekankan bahwa JSON native memungkinkan dokumen JSON disimpan dalam kolom database dan diproses dengan fungsi JSON.
Named Entity Recognition / NER	Teknik mengekstrak entitas dari teks, seperti nama orang, organisasi, lokasi, tanggal, dan topik.
n8n	Tool otomatisasi workflow open source untuk menghubungkan input, LLM, parser, validator, database, error branch, dan proses integrasi lain.
Nested Object	Object JSON di dalam object lain, misalnya field <code>alamat</code> berisi <code>kota</code> dan <code>provinsi</code> .
Null	Nilai JSON yang berarti kosong atau tidak tersedia. Harus dipakai hati-hati agar tidak menutupi field wajib yang hilang.
Ollama	Tool untuk menjalankan LLM lokal dan mendukung <i>structured output</i> dengan JSON Schema pada parameter format. Dalam outline buku, Ollama menjadi salah satu alat utama untuk eksperimen lokal.

Open Source LLM	Model bahasa besar yang dapat dijalankan atau dikembangkan dengan pendekatan terbuka, berguna untuk pembelajaran mandiri, riset, dan prototipe lokal.
Outlines	Library untuk <i>constrained generation</i> agar model menghasilkan output terstruktur, termasuk JSON sesuai schema.
Output Mentah	Jawaban asli LLM sebelum divalidasi. Output mentah sebaiknya disimpan untuk audit, tetapi tidak langsung dianggap sebagai data final.
Parser	Komponen yang membaca output teks atau JSON dan mengubahnya menjadi struktur data yang dapat diproses program.
Pattern	Constraint JSON Schema berbasis <i>regular expression</i> untuk memeriksa pola teks, misalnya format tanggal YYYY-MM-DD .
Pipeline	Rangkaian proses dari input, LLM, schema, validasi, database, workflow, sampai audit.
PostgreSQL	Database open source yang dapat dipakai dalam workflow data modern dan memiliki dukungan kuat untuk data semi-terstruktur melalui tipe JSON/JSONB.
Precision	Metrik evaluasi yang mengukur seberapa banyak hasil ekstraksi yang benar dari seluruh hasil yang diekstrak.
Prompt	Instruksi yang diberikan ke LLM agar model menghasilkan output sesuai tujuan.
Prompt Version	Versi prompt yang dicatat agar hasil eksperimen dapat dibandingkan dan diaudit.

Properties	Keyword JSON Schema untuk mendefinisikan field yang boleh ada dalam object.
Pydantic	Library Python untuk validasi dan serialisasi data berbasis <i>type hints</i> , serta dapat menghasilkan JSON Schema dari model Python. Outline buku menempatkan Pydantic sebagai lapisan validasi antara LLM dan database.
RAG	<i>Retrieval-Augmented Generation</i> , pendekatan yang menggabungkan pencarian dokumen atau data relevan dengan LLM agar jawaban lebih berbasis konteks.
Raw Output	Output asli dari LLM sebelum dibersihkan, diparsing, atau divalidasi.
Recall	Metrik evaluasi yang mengukur seberapa banyak informasi benar yang berhasil ditemukan dari seluruh informasi yang seharusnya ditemukan.
Required	Keyword JSON Schema untuk menentukan field yang wajib ada.
Retry	Strategi mencoba ulang proses jika output gagal, biasanya dibatasi maksimal 2–3 kali agar sistem tidak berputar tanpa akhir.
Schema Compliance Rate	Persentase output yang lolos validasi schema dibanding total output.
Schema Versioning	Praktik memberi versi pada schema agar perubahan struktur output dapat dilacak dan data lama tetap bisa diaudit.
Source Hash	Sidik jari digital dari input teks, biasanya dibuat dengan hash, untuk mendeteksi duplikasi dan mendukung audit.

Source Text	Teks asli yang diproses LLM. Wajib disimpan agar output dapat diverifikasi.
Structured Output	Output LLM yang mengikuti struktur formal seperti JSON Schema atau Pydantic model, sehingga dapat divalidasi, disimpan, dan diproses mesin.
Structured Output Parser	Node n8n untuk menstrukturkan output akhir AI berdasarkan JSON Schema, biasanya dipakai agar field hasil LLM lebih rapi dan konsisten.
Temperature	Parameter LLM yang memengaruhi variasi output. Untuk ekstraksi data, nilai rendah lebih disarankan agar output stabil.
Tool Calling	Pola ketika LLM memilih aksi atau tool dalam bentuk JSON terstruktur, misalnya <code>search_record</code> , bukan menjalankan perintah bebas.
Type	Keyword JSON Schema untuk menentukan tipe data, seperti <code>object</code> , <code>array</code> , <code>string</code> , <code>number</code> , <code>integer</code> , <code>boolean</code> , atau <code>null</code> .
Type Hints	Anotasi tipe data di Python, seperti <code>str</code> , <code>int</code> , <code>float</code> , <code>bool</code> , dan <code>list[str]</code> , yang dipakai Pydantic untuk validasi.
Validation Error	Error yang muncul ketika data tidak sesuai model atau schema. Error ini harus dicatat untuk debugging dan audit.
Validation Status	Status hasil validasi, biasanya <code>valid</code> , <code>invalid</code> , atau <code>pending_review</code> .
Valid JSON Rate	Persentase output LLM yang valid secara sintaks JSON dibanding total output.

Validator	Komponen yang memeriksa apakah data sesuai schema atau model. Dalam buku ini, validator utama dibangun dengan Python dan Pydantic.
Webhook	Endpoint HTTP yang menerima input dari luar dan memicu workflow, misalnya di n8n.
Workflow	Alur otomatis yang menghubungkan input, LLM, validasi, database, logging, dan respons.

Referensi

Bray, T. (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8259.html>

Dottxt AI. (n.d.). *JSON structured generation*. Diakses 27 Mei 2026, dari <https://dottxt-ai.github.io/outlines/reference/generation/json/>

Dottxt AI. (n.d.). *Outlines: Structured outputs*. Diakses 27 Mei 2026, dari <https://github.com/dottxt-ai/outlines>

FusionAuth Team. (2022). *Breaking down JSON Web Tokens*. FusionAuth. PDF terlampir.

FusionAuth. (n.d.). *Breaking down JSON Web Tokens*. Diakses 27 Mei 2026, dari <https://fusionauth.io/resources/breaking-down-json-web-tokens.pdf>

ggml-org. (n.d.). *GBNF guide: llama.cpp grammars*. Diakses 27 Mei 2026, dari <https://github.com/ggml-org/llama.cpp/blob/master/grammars/README.md>

Instructor. (n.d.). *Instructor: Structured LLM outputs*. Diakses 27 Mei 2026, dari <https://python.useinstructor.com/>

Instructor. (n.d.). *Structured outputs with Ollama*. Diakses 27 Mei 2026, dari <https://python.useinstructor.com/integrations/ollama/>

Itelman, R., & Viotti, J. C. (2024). *Unifying business, data, and code: Designing data products with JSON Schema*. O'Reilly Media. PDF terlampir.

Itelman, R., & Viotti, J. C. (2024). *Unifying business, data, and code: Designing data products with JSON Schema*. O'Reilly Media. <https://www.oreilly.com/library/view/unifying-business-data/9781098144999/>

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519.html>

JSON Schema. (n.d.). *JSON Schema documentation*. Diakses 27 Mei 2026, dari <https://json-schema.org/docs>

JSON Schema. (n.d.). *JSON Schema specification*. Diakses 27 Mei 2026, dari <https://json-schema.org/specification>

n8n. (n.d.). *Error handling*. Diakses 27 Mei 2026, dari <https://docs.n8n.io/flow-logic/error-handling/>

n8n. (n.d.). *Structured Output Parser node documentation*. Diakses 27 Mei 2026, dari <https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.outputparserstructured/>

Nuijten, A., & Barel, P. (2023). *Modern Oracle database programming: Level up your skill set to Oracle's latest and most powerful features in SQL, PL/SQL, and JSON*. Apress. PDF terlampir.

Nuijten, A., & Barel, P. (2023). *Modern Oracle database programming: Level up your skill set to Oracle's latest and most powerful features in SQL, PL/SQL, and JSON*. Apress.
<https://www.oreilly.com/library/view/modern-oracle-database/9781484291665/>

Ollama. (2024, December 6). *Structured outputs*. <https://ollama.com/blog/structured-outputs>

Ollama. (n.d.). *Structured outputs*. Diakses 27 Mei 2026, dari
<https://docs.ollama.com/capabilities/structured-outputs>

Oracle. (n.d.). *JSON data type*. MySQL Reference Manual. Diakses 27 Mei 2026, dari
<https://dev.mysql.com/doc/refman/9.1/en/json.html>

Oracle. (n.d.). *JSON search functions*. MySQL Reference Manual. Diakses 27 Mei 2026, dari
<https://dev.mysql.com/doc/refman/8.4/en/json-search-functions.html>

Oracle. (n.d.). *JSON table functions*. MySQL Reference Manual. Diakses 27 Mei 2026, dari
<https://dev.mysql.com/doc/refman/9.1/en/json-table-functions.html>

PostgreSQL Global Development Group. (n.d.). *JSON functions and operators*. PostgreSQL Documentation. Diakses 27 Mei 2026, dari
<https://www.postgresql.org/docs/current/functions-json.html>

Purbo, O. W. (2026). *Structured Outputs & JSON Schema untuk LLM: Teknik interaksi database modern dengan JSON, Python, n8n, dan open source LLM*. Institut Teknologi Tangerang Selatan, Dinas KOMINFO Tangerang Selatan, OnnoCenter. PDF terlampir.

Pydantic. (n.d.). *JSON*. Diakses 27 Mei 2026, dari
<https://pydantic.dev/docs/validation/latest/concepts/json/>

Pydantic. (n.d.). *JSON Schema*. Diakses 27 Mei 2026, dari
https://pydantic.dev/docs/validation/latest/concepts/json_schema/

Pydantic. (n.d.). *Models*. Diakses 27 Mei 2026, dari
<https://pydantic.dev/docs/validation/latest/concepts/models/>

Pydantic. (n.d.). *Validators*. Diakses 27 Mei 2026, dari
<https://pydantic.dev/docs/validation/latest/concepts/validators/>

Python Software Foundation. (n.d.). *typing — Support for type hints*. Python Documentation. Diakses 27 Mei 2026, dari <https://docs.python.org/3/library/typing.html>

python-jsonschema. (n.d.). *Schema validation*. Diakses 27 Mei 2026, dari
<https://python-jsonschema.readthedocs.io/en/stable/validate/>

Stokes, D. (2020). *MySQL & JSON: A practical programming guide* (2nd ed.). PDF terlampir.

Stokes, D. (2020). *MySQL & JSON: A practical programming guide* (2nd ed.).
<https://www.amazon.com/MySQL-JSON-Practical-Programming-Guide/dp/057878324X>

vLLM Project. (n.d.). *Structured outputs*. Diakses 27 Mei 2026, dari
https://docs.vllm.ai/en/latest/features/structured_outputs/

Tentang Penulis



Onno W. Purbo, saat ini bertugas sebagai rektor di Institut Teknologi Tangerang Selatan (ITTS). Onno memperoleh gelar Ph.D bidang Electrical Engineering dari University of Waterloo, Canada, adalah seorang copyleft, educator dan ICT evangelist. Dia sudah mempublikasikan 50++ buku, termasuk free ICT ebook untuk sekolah tahun 2008. Beberapa buku terakhirnya adalah "Internet-TCP/IP: Konsep Dan Implementasi", 2018; "Sistem Operasi, Konsep Dan Membuat Linux OpenWRT Dan ROM Android", 2019; "IPv6 Untuk Mendukung Operasi Jaringan Dan Domain Name System", 2019; "Kubernetes untuk Pemula", 2024 dan "Membuat Operator Seluler 5G Sendiri", 2024.

Dia memimpin sambungan pertama Internet di Institut Teknologi Bandung, tahun 1993-2000, dan menggunakannya untuk membuat jaringan Internet pendidikan yang pertama di Indonesia. Dia membebaskan frekuensi WiFi, memperkenalkan RT/RW-net, antenna Wajanbolic dan jaringan selular OpenBTS dan private 5G sendiri. Dia memimpin jaringan telepon pertama di atas Internet, VoIP Merdeka, yang kemudian hari dikenal sebagai VoIP Rakyat berbasis SIP dan menggunakan kode area +62520 dan +62521. Dia saat ini aktif memperkenalkan e-Learning, dan menjalankan server e-Learning di <http://lms.onnocenter.or.id/moodle/> 66,000++ siswa / mahasiswa dan <https://opencourse.itts.ac.id> 39.000+ mahasiswa secara gratis. Email: onno@indo.net.id